

HFT Ops Interview Preparation

Probability / Brainteasers / Code

Gabriel Kinshuk

August 24, 2026

Contents

1	Math / Probability Theory	3
1.1	Combinatorics	3
1.1.1	Permutations	3
1.1.2	Combinations	3
1.1.3	Distributing Indistinguishable Objects (Stars and Bars)	3
1.1.4	Inclusion-Exclusion	4
1.1.5	Select Worked Problems	5
1.2	Probability	6
1.2.1	Complement	6
1.2.2	Independence	7
1.2.3	Mutual Exclusivity	7
1.2.4	Select Worked Problems	7
1.3	Expected Value	10
1.3.1	Select Worked Problems	11
1.4	Conditional Probability & Bayes	15
1.4.1	Select Worked Problems	17
1.5	Distributions	19
1.5.1	Bernoulli Distribution	20
1.5.2	Binomial Distribution	20
1.5.3	Geometric Distribution	21
1.5.4	Negative Binomial Distribution	22
1.5.5	Poisson Distribution	22
1.5.6	Exponential Distribution	23
1.5.7	Uniform Distribution	24
1.5.8	Normal (Gaussian) Distribution	24
1.5.9	Lognormal Distribution	26
1.5.10	Reference Tables	26
1.5.11	Select Worked Problems	28
1.6	Markov Chains	30
1.6.1	Absorbing Chains and First-Step Analysis	30
1.6.2	Recurrent Chains and the Stationary Distribution	31
1.6.3	Select Worked Problems	32
2	Brainteasers	34
2.1	Strategies	34
2.1.1	Framing	34
2.1.2	Core Probabilistic Reductions	34

2.1.3	Combinatorial and Logical Tactics	35
2.1.4	Adversarial and Sequential Decisions	35
2.1.5	Counting Information	36
2.1.6	Estimation	36
2.1.7	Verification	37
2.2	Select Worked Problems	38
3	Code	56
3.1	SQL Patterns (PostgreSQL)	56
3.1.1	Filtering and Joins	56
3.1.2	Aggregation	57
3.1.3	NULL Handling	58
3.1.4	Window Functions	58
3.1.5	Deduplication	60
3.1.6	String Manipulation	61
3.1.7	Command Reference	64
3.2	Python Patterns	67
3.3	Algorithmic Paradigms	74
3.4	Recursion and Memoization	75
3.5	Heaps and Priority Queues	78
3.6	Sliding Window	79
3.7	Pandas	83
3.7.1	The Data Model	83
3.7.2	Selection and Filtering	85
3.7.3	Missing Data	86
3.7.4	Column Operations and Vectorization	87
3.7.5	String Matching with Regex	89
3.7.6	Grouping and Aggregation	90
3.7.7	Joining and Combining	92
3.7.8	Reshaping	94
3.7.9	Ordering and Row-Relative Logic	95
3.7.10	Consecutive Runs	98

1 Math / Probability Theory

1.1 Combinatorics

A counting problem is pinned down by two questions. Does the order of the chosen elements matter, and may an element be chosen more than once. The formulas below are organized around those two axes, plus a correction for identical elements and two escape hatches (the complement and inclusion-exclusion) for when a direct count fans out into many cases.

1.1.1 Permutations

Definition 1.1 (Permutation). A permutation is an ordered selection. Choosing k items from n distinct options where order matters gives

$$P(n, k) = \frac{n!}{(n-k)!}.$$

The formula counts position by position. The first slot has n candidates, the second has $n-1$ once one item is placed, and so on for k slots, giving the falling product $n(n-1)\cdots(n-k+1)$. Writing that product as $\frac{n!}{(n-k)!}$ just cancels the unused tail $(n-k)!$.

Definition 1.2 (Permutations with identical elements). When the n items being arranged include groups of identical copies with frequencies $n_1, n_2, \dots, n_k$, the number of distinguishable arrangements is

$$\frac{n!}{n_1! n_2! \cdots n_k!}.$$

Start from the $n!$ arrangements that would exist if every item were distinct. Each identical group of size n_i has $n_i!$ internal orderings that all produce the same visible arrangement, so dividing by each $n_i!$ removes exactly those duplicates.

1.1.2 Combinations

Definition 1.3 (Combination). A combination is an unordered selection. Choosing k items from n distinct options where order does not matter gives

$$C(n, k) = \binom{n}{k} = \frac{n!}{k!(n-k)!}.$$

Combinations come from permutations by discarding the ordering. Each unordered set of k items can be arranged in $k!$ ways, and $P(n, k)$ counts all of those as distinct, so

$$C(n, k) = \frac{P(n, k)}{k!}.$$

1.1.3 Distributing Indistinguishable Objects (Stars and Bars)

Theorem 1.4 (Stars and bars). *The number of ways to place n identical objects into k distinct bins, where a bin may hold any number of objects including zero, is*

$$\binom{n+k-1}{k-1} = C(n+k-1, k-1).$$

Proof. Lay the n objects in a row as stars and insert $k - 1$ dividers (bars) to cut the row into k groups. Reading left to right, objects before the first bar fall in bin 1, objects between the first and second bar fall in bin 2, and so on through bin k . Every placement of the bars produces one distribution, and every distribution corresponds to exactly one placement. The row has $n + (k - 1)$ positions, and a distribution is fixed by choosing which $k - 1$ of them hold bars, which is $\binom{n+k-1}{k-1}$. $\square$

1.1.4 Inclusion-Exclusion

Theorem 1.5 (Inclusion-Exclusion). *The size of a union is the alternating sum of the sizes of all intersections. For two and three sets,*

$$|A \cup B| = |A| + |B| - |A \cap B|,$$

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|.$$

The mechanism is bookkeeping on how many times each element gets counted. Summing the individual set sizes counts an element that lies in two sets twice, so subtracting the pairwise intersections corrects it back to one. An element in all three sets is now counted $3 - 3 = 0$ times, so adding the triple intersection restores it to one. The pattern continues to any number of sets: terms over an odd number of sets are added and terms over an even number are subtracted, which leaves every element counted exactly once no matter how many sets contain it.

Which counting tool

Order matters?	Repeats allowed?	Count
yes	no	$P(n, k) = \frac{n!}{(n-k)!}$
no	no	$C(n, k) = \binom{n}{k}$
yes	yes	n^k
no	yes	$\binom{n+k-1}{k-1}$ (stars and bars)

Two escape hatches handle counts that fan out into many subcases. Use the **complement** ($1 - P$ of the unwanted case) on “at least one” phrasing, and use **inclusion-exclusion** on “divisible by a or b ” style overlaps.

1.1.5 Select Worked Problems

Problem 1.1

How many ways can you arrange the letters in the word BANANA?

Solution. There are $n = 6$ elements. Repeated items have frequency 3 and 2 (Ns and As). Then the number of arrangements are $\frac{6!}{3! \times 2!} = \frac{720}{12} = 60$.

Problem 1.2

You roll 2 dice. How many distinct outcomes are there? What if you can't tell the dice apart?

Solution. With two dice, there are $6 \times 6 = 36$ outcomes if we can tell the dice apart. If we *can't* tell the dice apart, then we can solve either logically or formulaically:

Logically, we can see that from the original 36 combinations, there are 6 outcomes where the dice are the same, leaving 30 outcomes using mixed pairs of dice. Of these, half are duplicate when we can't tell the dice apart, leaving 15 outcomes. We can then add the 6 double outcomes back in for a total of 21 outcomes.

Formulaically, we can solve using a similar approach, taking the sum of mixed pair outcomes and double outcomes as $C(6, 2) + C(6, 1) = 15 + 6 = 21$.

Lastly, we can consider this problem using the stars and bars approach, where we notice that there are $n = 2$ identical dice and 6 distinct bins with $k = 5$ dividers between them. Then we can count the outcomes by $C(2 + 6 - 1, 6 - 1) = C(7, 5) = 21$

Problem 1.3

How many ways can you distribute 8 identical balls into 3 distinct boxes?

Solution. This is a direct application of stars and bars with $n = 8$ and $k = 3$. The number of outcomes is $C(8 + 3 - 1, 3 - 1) = C(10, 2) = \frac{10 \times 9 \times 8 \times \dots \times 3}{2} = 45$.

Problem 1.4

A deck has 52 cards. How many 5-card hands contain at least one ace?

Solution. Since this solution asks for the number of hands containing at least one ace, it will be simpler to compute the complement. This is the number of hands containing no aces, we then subtract from the total number of 5 card hands: $C(52, 5) - C(48, 5) = 886,656$.

Problem 1.5

You have 5 books on a shelf. In how many ways can you choose 3 books and arrange them left-to-right on a new shelf?

Solution. This is simply $C(5, 3) \times P(3, 3) = \frac{5 \times 4 \times 3}{3!} \times \frac{3!}{0!} = \frac{60 \times 3!}{3!} = 60$.

Problem 1.6

A pizza place offers 8 toppings. How many different pizzas can you make if you can choose any number of toppings (including none)?

Solution. We can solve this by taking the summation of $C(8, i)$ from $i = 0$ to $i = 8$:

$$\text{Total outcomes} = \sum_{i=0}^8 C(8, i) = 256$$

There is a more succinct solution we can find if we approach the problem seeing each topping as the binary choice of being either on or off of the pizza, then the total outcomes are $2^8 = 256$.

Problem 1.7

How many ways can 12 people be split into 3 groups of 4?

Solution. We choose each group independently with the remaining members of the pool, and then divide by the reorderings since the groups are indistinguishable:

$$\frac{C(12, 4) \times C(8, 4) \times C(4, 4)}{3!} = \frac{495 \times 70 \times 1}{6} = 34,650.$$

1.2 Probability

Definition 1.6 (Sample Space). The sample space Ω is the set of all possible outcomes of an experiment.

Most problems begin by fixing the sample space and then identifying the subset of outcomes that count as favourable. Getting the sample space right, which means choosing the correct unit of observation and the correct notion of “equally likely”, settles most of the difficulty before any arithmetic starts.

Definition 1.7 (Event). An event is a subset of the sample space, the collection of outcomes of interest. Its probability is the total weight of the outcomes it contains.

1.2.1 Complement

Complement

For any event A ,

$$P(A) = 1 - P(A^c),$$

where A^c is the event that A does not occur.

The complement is the first move whenever the direct event splits into many cases while its negation is a single clean case. “At least one” is the signature cue: the direct count runs over one, two, three or more successes, while the negation is the single event of zero successes. The birthday and “at least one head” problems below both collapse this way.

1.2.2 Independence

Definition 1.8 (Independence). Two events are independent when the occurrence of one leaves the probability of the other unchanged. Formally,

$$P(A \cap B) = P(A) P(B).$$

The multiplication rule is the practical test. Drawing *with* replacement keeps every draw independent because the pool is restored each time. Drawing without replacement makes the draws dependent, since removing an item changes both the size and the composition of the pool for the next draw. Independence is a statement about probabilities and is a separate idea from mutual exclusivity, which is a statement about outcomes.

1.2.3 Mutual Exclusivity

Definition 1.9 (Mutual exclusivity). Two events are mutually exclusive when they cannot both occur, so $A \cap B = \emptyset$. For such events the union probability adds:

$$P(A \cup B) = P(A) + P(B).$$

This is the special case of inclusion-exclusion where the intersection is empty. The general identity $P(A \cup B) = P(A) + P(B) - P(A \cap B)$ carries the overlap term $P(A \cap B)$, and mutual exclusivity sets that term to zero. A common error is to add probabilities for events that can co-occur, which double counts the overlap.

1.2.4 Select Worked Problems

Problem 1.8

How many people do you need in a room for a >50% chance that two share a birthday?

Solution. Computing this directly is impractical, since there are many ways for multiple people to share a birthday. Instead, using the complement, we simply subtract the probability that *no* people share a birthday from 1:

$$P(\text{all different}) = \prod_{i=1}^n \frac{365 - i + 1}{365}$$

We multiply additional factors until the quantity drops *below* 0.5 (because then $1 - P$ is greater than 0.5). This turns out to be $n = 23$, where $P(\text{all different}) \approx 0.493$ and thus $P(\text{birthday is shared}) \approx 0.507$.

Problem 1.9

You flip a fair coin 3 times. What's the probability of getting exactly 2 heads?

Solution. Flipping a coin three times has eight outcomes. Of these, three involve exactly two heads, thus the probability is $\frac{3}{8} = 0.375$.

If we don't want to enumerate each of the eight options and count directly, we could use $C(3, 2) = 3$ to find the number of outcomes involving two heads.

Problem 1.10

Roll two dice. What's the probability the sum is 7?

Solution. There are 36 outcomes of rolling two dice. Of these, the pairs that equal 7 are $(1, 6), (2, 5), (3, 4), (4, 3), (5, 2), (6, 1)$. Then the probability is $\frac{6}{36} \approx 0.167$.

Problem 1.11

You draw 2 cards from a standard 52-card deck (without replacement). What's the probability both are hearts?

Solution. Without replacement, the draws are dependent. Then, the probability is:

$$\frac{13}{52} \times \frac{12}{51} \approx 0.0588$$

Note that *both* the numerator and the denominator shrank after we removed the first card.

We could also solve this by counting, if we divide the number of ways to pick two hearts by the total number of ways to pick two cards:

$$\frac{C(13, 2)}{C(52, 2)} = \frac{78}{1326} \approx 0.0588$$

Problem 1.12

You flip a fair coin 5 times. What's the probability of getting at least one head?

Solution. We can use the complement, the probability of getting *no* heads, subtracted from 1:

$$1 - \frac{1}{2^5} = 0.96875$$

Problem 1.13

You roll a fair die twice. What's the probability the second roll is strictly greater than the first?

Solution. There are 36 total ways to roll two dice. If the first roll is a 1, then there are 5 ways to roll higher on the second die. Similarly, there are 4 ways to roll higher if we rolled a 2, and so on.

Thus the number of favourable outcomes $1 + 2 + \cdots + 5$ or the triangle number of 5, $T_5 = 15$. The final answer is:

$$\frac{15}{36} \approx 0.417$$

Alternatively, we could solve by symmetry, calling roll 1 and 2 as A and B , respectively. If we notice that $P(A > B) = P(B < A)$, and $P(A = B) = \frac{6}{36}$, then we evenly divide the remaining outcomes by 2 to get $\frac{15}{36}$.

Problem 1.14

A box has 5 white and 3 black balls. You draw 3 without replacement. What's the probability of drawing exactly 2 white and 1 black?

Solution. The total ways to draw 3 balls are $C(8, 3) = 56$. We then find the ways to choose 2 white balls and 1 black ball individually, which is $C(5, 2)$ and $C(3, 1)$, then the answer is:

$$\frac{C(5, 2) \times C(3, 1)}{C(8, 3)} = \frac{10 \times 3}{56} \approx 0.536$$

Since we remove the balls without replacement, this problem follows the hypergeometric distribution.

Problem 1.15

You randomly pick a number from 1 to 100. What's the probability it's divisible by 3 or 5?

Solution. There are 33 numbers divisible by 3, and 20 numbers divisible by 5. The numbers divisible by both are the multiples of 15 of which there are 6 (15, 30, 45, 60, 75, 90). Thus:

$$P_{Div}(A \cup B) = \frac{33}{100} + \frac{20}{100} - \frac{6}{100} = 0.47$$

Problem 1.16

A family has 3 children. Assuming each child is equally likely to be a boy or girl, what's the probability that all 3 are the same gender?

Solution. There are eight outcomes, of those there are two in which all children are the same gender, so the answer is $\frac{2}{8} = 0.25$.

Problem 1.17

You throw 3 dice. What's the probability ALL three show different numbers?

Solution. There are $6^3 = 216$ total ways to throw three dice. After we throw the first dice, there are 5 options remaining, and 4 after the second dice. Then the answer is:

$$\frac{6 \times 5 \times 4}{6^3} = \frac{120}{216} \approx 0.556$$

Problem 1.18

You toss a fair coin until you get heads. What's the probability you need exactly 3 tosses?

Solution. We stop at the first heads, so needing exactly 3 tosses means the first two tosses are tails and the third is heads: T, T, H. Since tosses are independent and each outcome has probability $\frac{1}{2}$,

$$P(\text{exactly 3}) = \frac{1}{2} \times \frac{1}{2} \times \frac{1}{2} = \frac{1}{8}.$$

We do not enumerate anything else, because exactly one sequence produces this event.

Remark (Two ways probability gets counted). The reason we can pin down this single case without listing every outcome is that this problem lives in a different kind of sample space than counting problems like the birthday question.

When every outcome is equally likely, probability is combinatorial. You divide favorable outcomes by total outcomes, so you have to characterize the entire sample space. Each outcome carries no probability on its own and earns weight only by being counted against the total.

When outcomes are not equally likely, each outcome carries its own probability directly, usually a product of independent step probabilities. Here the sequence TTH has probability $\frac{1}{2} \cdot \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{8}$ by itself, with no denominator to divide by. That is why one case can be solved in isolation.

These are the same axioms applied to different spaces. Counting favorable over total is the special case that appears when all per-outcome probabilities are equal, so the sum of probabilities collapses into a plain count. The coin problem is the general version, where the per-outcome probabilities differ and you take the one you want.

Problem 1.19

In a room of 5 people, what's the probability that at least two share a birthday? (Assume 365 days, equally likely.)

Solution. Again, we use the complement to find 1 minus the probability that no one shares a birthday:

$$1 - \left(\frac{365}{365} \times \frac{364}{365} \times \frac{363}{365} \times \frac{362}{365} \times \frac{361}{365} \right) \approx 1 - 0.9729 \approx 0.0271$$

1.3 Expected Value

Definition 1.10 (Expected Value). The expected value of a random variable is the probability-weighted average of its outcomes:

$$\mathbb{E}[X] = \sum_i x_i P(X = x_i).$$

It is the long-run mean over many repetitions. The expected value need not be an attainable outcome (a fair die has $\mathbb{E}[X] = 3.5$) and need not be the most likely one.

Linearity of expectation

For any random variables X and Y and any scalars a, b ,

$$\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y].$$

This holds with no independence assumption. Paired with indicator variables it is the highest-yield tool for expectation brainteasers, since it turns a hard expected count into a sum of easy probabilities.

Definition 1.11 (Indicator Random Variable). An indicator $\mathbf{1}\{A\}$ equals 1 when event A occurs and 0 otherwise. Its expectation is the event's probability,

$$\mathbb{E}[\mathbf{1}\{A\}] = 1 \cdot P(A) + 0 \cdot P(A^c) = P(A).$$

The variable itself is trivial. The leverage is the decomposition it enables. To find the expected *count* of how many events in a family occur, write that count as a sum of indicators, one per event, then apply linearity:

$$\mathbb{E}\left[\sum_i \mathbf{1}\{A_i\}\right] = \sum_i \mathbb{E}[\mathbf{1}\{A_i\}] = \sum_i P(A_i).$$

Because linearity ignores dependence, this works even when the events are entangled, which is what collapses an otherwise messy count into a sum of easy probabilities. The shuffle and fixed-point problems below are the standard instances.

Theorem 1.12 (Law of Total Expectation). *If the cases $B_1, \dots, B_n$ partition the sample space, meaning they are mutually exclusive and cover every outcome, then*

$$\mathbb{E}[X] = \sum_{i=1}^n \mathbb{E}[X \mid B_i] \cdot P(B_i).$$

This regroups the definition of expected value by cases. Rather than averaging over individual outcomes, average the conditional mean inside each case and weight it by the case probability. Expanding each $\mathbb{E}[X \mid B_i]$ recovers the flat sum, so the identity adds no new information, but it changes what is computable. It is the tool of choice in three situations:

- you are handed conditional averages and case probabilities rather than the full outcome distribution
- X is continuous, so the flat sum $\sum_x x P(X = x)$ does not apply while conditioning on discrete cases still does
- $\mathbb{E}[X \mid B_i]$ refers back to $\mathbb{E}[X]$ itself, which turns the conditioning into a solvable equation

The cases must be disjoint and exhaustive. Overlapping or missing cases break the weighting.

Example 1.13 (Conditioning on cases). A commute time depends on the weather. Sunny days are 60% of days with a 20 minute average, and rainy days are 40% with a 35 minute average. The two cases partition every day, so

$$\mathbb{E}[\text{commute}] = 0.6(20) + 0.4(35) = 12 + 14 = 26 \text{ minutes.}$$

Example 1.14 (Recursive expectation). Take the expected number of rolls of a die until the first 6, and call it E . One roll always happens. With probability $\frac{1}{6}$ it is a 6 and the process stops, and with probability $\frac{5}{6}$ it is not, which leaves the same problem ahead with the same expectation E :

$$E = 1 + \frac{5}{6}E \longrightarrow \frac{1}{6}E = 1 \longrightarrow E = 6.$$

This is the mean of the geometric distribution, the expected number of trials until the first success, which is $\frac{1}{p}$ with $p = \frac{1}{6}$.

1.3.1 Select Worked Problems

Problem 1.20

A game costs \$1. You flip a coin: heads you win \$3, tails you win nothing. What's $\mathbb{E}[\text{Profit}]$?

Solution. $EV = 0.5 \cdot 2 + 0.5 \cdot -1 = +0.5$

Problem 1.21

Roll a die. If it shows 6, you roll again and add. Otherwise, your score is the number shown. What's the expected score?

Solution.

$$EV = 3.5 + \frac{1}{6} \cdot EV \longrightarrow \frac{5}{6} \cdot EV = 3.5 \longrightarrow EV = 4.2$$

Problem 1.22

Shuffle a standard deck. What's the expected number of cards that are in their "correct" position (card k is in position k)?

Solution. This is linearity of expectation applied to indicators. Let $I_k = 1$ if card k lands in position k , and 0 otherwise. In a random shuffle each card is equally likely to occupy any of the 52 positions, so $\mathbb{E}[I_k] = P(I_k = 1) = \frac{1}{52}$. The number of correctly placed cards is $M = \sum_{k=1}^{52} I_k$, and by linearity

$$\mathbb{E}[M] = \sum_{k=1}^{52} \mathbb{E}[I_k] = 52 \cdot \frac{1}{52} = 1.$$

The I_k are dependent, but linearity does not require independence, so that never matters.

Problem 1.23

You flip a coin until you get heads. How many flips do you expect?

Solution.

$$EV = 1 + 0.5 \cdot EV \longrightarrow 0.5 \cdot EV = 1 \longrightarrow EV = 2$$

Problem 1.24

A cereal box contains 1 of 4 different toys (equally likely). How many boxes must you buy to expect to collect all 4?

Solution. The total number of boxes is a sum of four phase waits, so we use linearity of expectation. In phase k we already hold $k - 1$ toys and buy boxes until a new one appears. The per-trial success probability is $p = \frac{\text{favourable}}{\text{total}}$, the number of missing toys over 4. We want the reciprocal, the expected number of trials per success, which is $\frac{1}{p}$:

$$\begin{aligned} 0 \text{ toys, } p &= \frac{4}{4} \longrightarrow \mathbb{E}[T_1] = \frac{1}{p} = 1 \\ 1 \text{ toy, } p &= \frac{3}{4} \longrightarrow \mathbb{E}[T_2] = \frac{1}{p} = \frac{4}{3} \\ 2 \text{ toys, } p &= \frac{2}{4} \longrightarrow \mathbb{E}[T_3] = \frac{1}{p} = 2 \\ 3 \text{ toys, } p &= \frac{1}{4} \longrightarrow \mathbb{E}[T_4] = \frac{1}{p} = 4 \end{aligned}$$

$$\mathbb{E}[T] = \mathbb{E}[T_1] + \mathbb{E}[T_2] + \mathbb{E}[T_3] + \mathbb{E}[T_4] = 1 + \frac{4}{3} + 2 + 4 = \frac{25}{3} \approx 8.33$$

Problem 1.25

I roll a die repeatedly and keep a running sum. What's the expected number of rolls until the sum first reaches or exceeds 10?

Solution. We solve recursively. Let $\mathbb{E}[s]$ be the expected number of additional rolls needed to first reach a sum of at least 10, starting from current sum s . The answer we want is $\mathbb{E}[0]$.

Base case: $\mathbb{E}[s] = 0$ for all $s \geq 10$, since the target is already met. For $s < 10$, one roll costs 1 and moves to $s + 1, \dots, s + 6$ with equal probability $\frac{1}{6}$:

$$\mathbb{E}[s] = 1 + \frac{1}{6} \sum_{j=1}^6 \mathbb{E}[s + j], \quad \mathbb{E}[s] = 0 \text{ for } s \geq 10.$$

Working downward from the base case:

$$\begin{aligned} \mathbb{E}[9] &= 1 + \frac{1}{6}(\mathbb{E}[10] + \dots + \mathbb{E}[15]) = 1 \\ \mathbb{E}[8] &= 1 + \frac{1}{6}\mathbb{E}[9] = \frac{7}{6} \\ \mathbb{E}[7] &= 1 + \frac{1}{6}(\mathbb{E}[8] + \mathbb{E}[9]) = \frac{49}{36} \\ &\vdots \\ \mathbb{E}[0] &\approx 3.6 \end{aligned}$$

Problem 1.26

You roll a fair six-sided die. After seeing the result, you may keep it as your score, or reroll. If you reroll, the new value replaces the old one (you cannot go back to a discarded roll). You are allowed up to three rolls total, and your score is the value you are holding when you stop. Playing optimally to maximize your expected score, what is that expected score, and what is the stopping rule at each stage?

Solution. Let $\mathbb{E}[r]$ be the expected score with r rerolls remaining. Solve backwards.

With no rerolls left, you keep whatever you roll, so $\mathbb{E}[0] = 3.5$.

With one reroll left, you keep the current face only if it beats the continuation value $\mathbb{E}[0] = 3.5$, i.e. keep on $\{4, 5, 6\}$ and reroll otherwise:

$$\mathbb{E}[1] = \frac{1}{6}(4) + \frac{1}{6}(5) + \frac{1}{6}(6) + \frac{3}{6}(3.5) = \frac{17}{4} = 4.25.$$

With two rerolls left, the continuation value is now $\mathbb{E}[1] = 4.25$, so you keep only faces that beat it, $\{5, 6\}$, and reroll on $\{1, 2, 3, 4\}$:

$$\mathbb{E}[2] = \frac{1}{6}(6) + \frac{1}{6}(5) + \frac{4}{6}(4.25) = \frac{14}{3} \approx 4.67.$$

Problem 1.27

You pick a random number uniformly from $[0, 1]$. Then you pick another. Then another. You stop when the running total exceeds 1. How many numbers do you expect to pick?

Solution. Let N be the number of draws until the running sum first exceeds 1, and let $X_1, X_2, \dots$ be the draws. We want $\mathbb{E}[N]$.

Step 1: a tail probability. The *tail probability* $\mathbb{P}(N > n)$ is the chance N exceeds a cutoff n , i.e. the chance we still need more draws after n of them. Needing more than n draws means the first n have not yet crossed 1:

$$N > n \iff X_1 + \dots + X_n \leq 1.$$

Step 2: geometry The point $(X_1, \dots, X_n)$ lands uniformly in the unit hypercube $[0, 1]^n$, which has volume 1. Since the draws are uniform, the probability of landing in any region equals that region's volume. The region where the coordinates are nonnegative and sum to at most 1 is a *simplex* (the n -dimensional triangle: a segment in 1D, triangle in 2D, tetrahedron in 3D). Its volume is $\frac{1}{n!}$ ($V_n(t) = \int_0^t V_{n-1}(t-x) dx$, starting from $V_1(t) = t$. Then $V_n(t) = \frac{t^n}{n!}$, evaluated at $t = 1$, $V_n(1) = \frac{1}{n!}$), so

$$\mathbb{P}(N > n) = \mathbb{P}(X_1 + \dots + X_n \leq 1) = \frac{1}{n!}.$$

Step 3: sum the tails. For a nonnegative integer variable, the mean equals the sum of its tail probabilities:

$$\mathbb{E}[N] = \sum_{n=0}^{\infty} \mathbb{P}(N > n).$$

This holds because an outcome $N = k$ clears the cutoffs $n = 0, 1, \dots, k-1$, so it is counted exactly k times across the sum, contributing its own value. Substituting the tails:

$$\mathbb{E}[N] = \sum_{n=0}^{\infty} \frac{1}{n!} = \frac{1}{0!} + \frac{1}{1!} + \frac{1}{2!} + \dots = e \approx 2.718.$$

1.4 Conditional Probability & Bayes

Definition 1.15 (Conditional Probability). The probability of A given that B has occurred is

$$P(A | B) = \frac{P(A \cap B)}{P(B)}, \quad P(B) > 0.$$

Conditioning on B restricts attention to the outcomes inside B and rescales their probabilities to sum to 1, which is the role of the denominator.

Theorem 1.16 (Bayes' Theorem). *Bayes' theorem reverses the direction of conditioning, updating a hypothesis as evidence arrives:*

$$P(A | B) = \frac{P(B | A) P(A)}{P(B)}.$$

The four pieces are worth memorizing by name:

- $P(A | B)$ is the posterior, the updated belief in A after seeing B
- $P(B | A)$ is the likelihood, how well A explains the evidence
- $P(A)$ is the prior, the belief in A before the evidence
- $P(B)$ is the marginal, the total probability of the evidence

Odds form skips the marginal

Dividing Bayes' theorem for two competing hypotheses cancels the shared marginal $P(B)$:

$$\underbrace{\frac{P(A | B)}{P(A^c | B)}}_{\text{posterior odds}} = \underbrace{\frac{P(B | A)}{P(B | A^c)}}_{\text{likelihood ratio}} \times \underbrace{\frac{P(A)}{P(A^c)}}_{\text{prior odds}}.$$

Each piece of evidence multiplies the prior odds by its likelihood ratio, so repeated independent evidence just multiplies again. This avoids computing $P(B)$ and is the fastest route through two-hypothesis update problems.

Theorem 1.17 (Law of Total Probability). *If $B_1, B_2, \dots, B_n$ partition the sample space (mutually exclusive and exhaustive), then*

$$P(A) = \sum_i P(A | B_i) P(B_i).$$

This is how the marginal in Bayes' theorem is usually assembled. When $P(A)$ is not known directly but the conditional probabilities on each case are, summing those conditionals weighted by the case probabilities builds it. The two-way form $P(A) = P(A | C)P(C) + P(A | C^c)P(C^c)$ covers most interview uses.

Example 1.18 (Monty Hall Problem). Assume the contestant chooses Door 1 and Monty opens Door 2 to reveal a goat. Let A_i be the event the car is behind Door i (prior $P(A_i) = \frac{1}{3}$), and B be the event Monty opens Door 2.

The likelihoods $P(B|A_i)$ are:

- $P(B|A_1) = \frac{1}{2}$ (Monty randomly picks Door 2 or 3)
- $P(B|A_2) = 0$ (Monty cannot reveal the car)
- $P(B|A_3) = 1$ (Monty is forced to open Door 2)

The marginal probability of B is:

$$P(B) = \sum_{i=1}^3 P(B|A_i)P(A_i) = \left(\frac{1}{2}\right)\left(\frac{1}{3}\right) + (0)\left(\frac{1}{3}\right) + (1)\left(\frac{1}{3}\right) = \frac{1}{2}$$

By Bayes' Theorem, the posterior probabilities of winning by staying (A_1) or switching (A_3) are:

$$P(A_1|B) = \frac{P(B|A_1)P(A_1)}{P(B)} = \frac{\left(\frac{1}{2}\right)\left(\frac{1}{3}\right)}{\frac{1}{2}} = \frac{1/6}{1/2} = \frac{1}{3}$$
$$P(A_3|B) = \frac{P(B|A_3)P(A_3)}{P(B)} = \frac{(1)\left(\frac{1}{3}\right)}{\frac{1}{2}} = \frac{1/3}{1/2} = \frac{2}{3}$$

Switching doubles the probability of winning.

1.4.1 Select Worked Problems

Problem 1.28

I roll two dice. Given that the sum is 8, what's the probability that one of the dice shows a 3?

Solution. The possible ways to roll a sum of 8 are: (2, 6), (3, 5), (4, 4), (5, 3), (6, 2). Of these, two options have a 3 in them, so the answer is $\frac{2}{5} = 0.4$.

Problem 1.29

A bag has 3 red and 2 blue balls. You draw one, don't replace it, then draw another. What's $P(\text{2nd is red} \mid \text{1st was blue})$?

Solution. Once we drew one blue on, the remaining bag composition is 3 red balls and 1 blue one. Then, there is a $\frac{3}{4}$ probability that the second will be red.

Problem 1.30

60% of emails are spam. A filter catches 95% of spam but also flags 3% of legitimate emails. An email is flagged. What's the probability it's actually spam?

Solution. We want $P(\text{Spam} \mid \text{Flagged})$. The likelihood $P(\text{Flagged} \mid \text{Spam})$ is given as 0.95, the prior $P(\text{Spam})$ is 0.6, and the marginal probability $P(\text{Flagged})$ is $0.95 \cdot 0.6 + 0.03 \cdot 0.4 = 0.582$.

Then the posterior is

$$P(\text{Spam} \mid \text{Flagged}) = \frac{P(\text{Flagged} \mid \text{Spam}) \times P(\text{Spam})}{P(\text{Flagged})} = \frac{0.95 \times 0.6}{0.582} \approx 0.979$$

Problem 1.31

You have 2 coins. Coin A is fair (50% heads). Coin B is biased (75% heads). You pick a coin at random and flip it. It lands heads. What's the probability you picked coin B?

Solution. We are looking for the posterior, $P(B \mid H)$.

The priors are $P(A) = P(B) = 0.5$. The likelihoods are $P(H \mid A) = 0.5$ and $P(H \mid B) = 0.75$. Then $P(H) = P(H \mid A) \cdot P(A) + P(H \mid B) \cdot P(B) = 0.5 \cdot 0.5 + 0.75 \cdot 0.5 = 0.625$. Then finally

$$P(B \mid H) = \frac{P(H \mid B) \cdot P(B)}{P(H)} = \frac{0.75 \cdot 0.5}{0.625} = 0.6$$

After flipping and landing on heads, our belief that we picked coin B increased from 0.5 to 0.6.

Problem 1.32

Same as 1.31. The chosen coin is flipped twice and get heads both times. Now what's $P(B)$?

Solution. The only difference is our priors are $P(A) = 0.4$ and $P(B) = 0.6$. Then we update $P(H) = P(H \mid A) \cdot P(A) + P(H \mid B) \cdot P(B) = 0.5 \cdot 0.4 + 0.75 \cdot 0.6 = 0.65$. Finally:

$$P(B \mid H) = \frac{P(H \mid B) \cdot P(B)}{P(H)} = \frac{0.75 \cdot 0.6}{0.65} = \frac{9}{13} \approx 0.692$$

Problem 1.33

A family has 2 children. Given that at least one child is a boy, what's the probability both are boys?

Solution. We can answer this by considering the restriction over the sample space caused by the new information. Originally, for two children there would be four possibilities, of which 1 has two boys. After we know that at least one child is a boy, only three options remain $(B, G), (B, B), (G, B)$. Thus, the probability both are boys is $\frac{1}{3}$.

Problem 1.34

Three cards: one is red on both sides, one is blue on both sides, one is red on one side and blue on the other. You pick a card at random and see a red side. What's the probability the other side is also red?

Solution. The natural first instinct is to treat the cards as the sample space, as with the births problem: restrict to cards consistent with seeing a red face, then count outcomes among those. This fails here because a card is not a single unit of observation. When a card has two sides of the same color, "seeing red" can happen in more than one way for that card. There's no such internal multiplicity in the births problem.

So take the six sides, not the three cards, as the equally likely outcomes. Three of these six are red: two belong to the RR card, one belongs to the RB card. Since each red side was equally likely to be the one we observed, and two of the three red sides have a red back:

$$P(\text{other side red} \mid \text{observed red}) = \frac{2}{3}.$$

Problem 1.35

You have a 6-sided die. You don't know if it's fair or loaded. A fair die shows each number with probability $1/6$. The loaded die shows 6 with probability $1/2$ and each other number with probability $1/10$. You pick a die at random (50-50) and roll a 6. What's $P(\text{loaded})$?

Solution. By Bayes' theorem, with $P(\text{loaded}) = P(\text{fair}) = 0.5$:

$$P(\text{loaded} \mid 6) = \frac{P(6 \mid \text{loaded}) P(\text{loaded})}{P(6)} = \frac{0.5 \times 0.5}{0.5 \times 0.5 + \frac{1}{6} \times 0.5} = \frac{0.25}{1/3} = 0.75$$

Alternatively, work in odds instead of probabilities. Dividing the Bayes' theorem expressions for $P(\text{loaded} \mid 6)$ and $P(\text{fair} \mid 6)$ cancels $P(6)$ from both, leaving:

$$\underbrace{\frac{P(\text{loaded} \mid 6)}{P(\text{fair} \mid 6)}}_{\text{posterior odds}} = \underbrace{\frac{P(6 \mid \text{loaded})}{P(6 \mid \text{fair})}}_{\text{likelihood ratio}} \times \underbrace{\frac{P(\text{loaded})}{P(\text{fair})}}_{\text{prior odds}}$$

So the posterior odds are just the prior odds scaled by the likelihood ratio, no need to compute $P(6)$ at all. Here, the prior odds are $\frac{P(\text{loaded})}{P(\text{fair})} = \frac{0.5}{0.5} = 1$, and the likelihood ratio is $\frac{P(6 \mid \text{loaded})}{P(6 \mid \text{fair})} = \frac{0.5}{1/6} = 3$. So the posterior odds are $1 \times 3 = 3$, i.e. $3 : 1$ in favor of loaded, giving $P(\text{loaded} \mid 6) = 3/4 = 0.75$.

In short, a six is three times more likely to appear if the die is loaded, so each 6 observed multiplies the odds by 3. If we observed *another* 6, the odds would update again: $3 : 1 \rightarrow 9 : 1$, giving $P(\text{loaded}) = \frac{9}{10}$.

1.5 Distributions

A named distribution is a compressed description of a random process. What makes the catalogue useful is not the formulas in isolation but the mapping from a verbal cue in the problem statement to a process, and from that process to a mean, a variance, and a tail. The organizing question for any problem is therefore which process generated the randomness rather than which formula to apply.

Definition 1.19 (Random variable and distribution). A random variable X is a function assigning a number to each outcome in the sample space Ω . Its distribution specifies how probability is spread across the values X can take, and it is described by one of three equivalent objects:

- the probability mass function (PMF) $P(X = x)$ in the discrete case, with $\sum_x P(X = x) = 1$
- the probability density function (PDF) $f(x)$ in the continuous case, with $\int_{-\infty}^{\infty} f(x) dx = 1$
- the cumulative distribution function (CDF) $F(x) = P(X \leq x)$, defined in both cases

The set of values with nonzero probability is the support.

In the continuous case $P(X = x) = 0$ for every single point, so density is not probability. Only integrals of the density over an interval carry probability, which is why continuous questions are answered through the CDF as $P(a < X \leq b) = F(b) - F(a)$.

Recognize the process first

A named distribution (Bernoulli, Binomial, Geometric, Poisson, and so on) is a family arising from a common underlying random process, indexed by one or more parameters such as n and p for the Binomial. Once a problem is matched to its process, the PMF or PDF, mean, and variance follow immediately from the known formulas for the family. Three cues carry most of the discrete cases:

- a *fixed* number of trials with a counted number of successes points to the Binomial
- a *fixed* number of successes with a counted number of trials points to the Geometric or Negative Binomial
- a count over a window of time or space with no natural trial count points to the Poisson

Definition 1.20 (Moment). The n -th moment of a random variable X is $\mathbb{E}[X^n]$, and its n -th central moment is $\mathbb{E}[(X - \mu)^n]$, taken about the mean $\mu = \mathbb{E}[X]$. A moment exists only when the defining expectation converges absolutely, meaning $\mathbb{E}[|X|^n] < \infty$.

Moments are summary numbers describing the shape of a distribution, each one weighting outcomes by a higher power and so responding more strongly to values far from the center. The first four carry standard names and interpretations:

- the first moment $\mathbb{E}[X]$ is the mean, locating the distribution
- the second central moment $\mathbb{E}[(X - \mu)^2]$ is the variance, measuring spread
- the third standardized moment is the skewness, measuring asymmetry
- the fourth standardized moment is the kurtosis, measuring tail weight

Standardizing divides the central moment by σ^n , which makes skewness and kurtosis dimensionless and therefore comparable across distributions. A distribution is not guaranteed to possess any given moment, since sufficiently heavy tails make the integral diverge, and moments fail from the top down in the sense that a finite n -th moment forces every lower moment to be finite as well.

1.5.1 Bernoulli Distribution

Definition 1.21 (Bernoulli). A random variable X follows a Bernoulli distribution with parameter $p \in [0, 1]$, written $X \sim \text{Bernoulli}(p)$, if it takes exactly two values:

$$P(X = 1) = p, \quad P(X = 0) = 1 - p.$$

It models a single trial with two outcomes, conventionally labeled success (1) and failure (0).

$$\mu = p \quad \sigma^2 = p(1 - p)$$

Both moments follow in one line. Since X only takes the values 0 and 1, the identity $X^2 = X$ holds pointwise, so $\mathbb{E}[X^2] = \mathbb{E}[X] = p$ and

$$\text{Var}(X) = \mathbb{E}[X^2] - \mathbb{E}[X]^2 = p - p^2 = p(1 - p).$$

The variance is largest at $p = \frac{1}{2}$, where the outcome is most uncertain, and shrinks to zero as p approaches 0 or 1, where the result becomes nearly certain.

Building block for other distributions

The Bernoulli is rarely the distribution of interest on its own. It is the atomic unit that other named distributions are built from. A Binomial random variable is a sum of n i.i.d. $\text{Bernoulli}(p)$ trials, and a Geometric random variable counts trials until the first Bernoulli success. Recognizing a Bernoulli inside a larger problem, often through an indicator variable $\mathbf{1}\{\cdot\}$, is frequently the first step in applying linearity of expectation to more complex sums.

1.5.2 Binomial Distribution

Definition 1.22 (Binomial). $X \sim \text{Binomial}(n, p)$ counts the number of successes in n independent trials, each succeeding with the same probability p . Its PMF is

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}, \quad k = 0, 1, \dots, n.$$

Proof. Fix one particular sequence of outcomes with k successes and $n - k$ failures. Independence makes its probability the product $p^k (1 - p)^{n-k}$, and that value depends only on the counts, not on the positions. The number of distinct sequences with k successes is the number of ways to choose which trials succeed, namely $\binom{n}{k}$. Summing the common probability over those disjoint sequences gives the stated PMF. $\square$

$$\mu = np \quad \sigma^2 = np(1 - p)$$

Writing $X = \sum_{i=1}^n X_i$ as a sum of i.i.d. $\text{Bernoulli}(p)$ variables gives both moments without touching the PMF. Linearity of expectation yields $\mathbb{E}[X] = np$ with no independence needed. Variance requires independence, which zeroes every covariance cross term and leaves $\text{Var}(X) = n \cdot p(1 - p)$.

Sampling with replacement against without

The Binomial requires that p stay constant across trials, which holds when sampling with replacement. Sampling without replacement changes the composition of the pool after each draw and produces the Hypergeometric distribution instead. When the population is large relative to the sample, the shift in p is negligible and the Binomial is the standard approximation.

Definition 1.23 (Hypergeometric). Drawing n items without replacement from a population of N containing K successes, the number of successes drawn has PMF

$$P(X = k) = \frac{\binom{K}{k} \binom{N-K}{n-k}}{\binom{N}{n}}.$$

Its mean is $\mu = n \frac{K}{N}$, matching the Binomial with $p = K/N$, and its variance is

$$\sigma^2 = np(1-p) \frac{N-n}{N-1},$$

where the factor $\frac{N-n}{N-1}$ is the finite population correction.

The mean is unchanged because linearity of expectation ignores the dependence between draws. The variance is lower because the draws are negatively correlated: each draw removes an item from the pool, so a success on one draw leaves fewer successes available and makes a success on the next slightly less likely. That downward pull is exactly the correction factor, which is below 1, so sampling without replacement is strictly less variable than sampling with replacement. It collapses to 1 as N grows relative to n , which recovers the Binomial.

1.5.3 Geometric Distribution

Definition 1.24 (Geometric). $X \sim \text{Geometric}(p)$ counts the number of independent trials up to and including the first success, where each trial succeeds with probability p . Its PMF is

$$P(X = k) = (1-p)^{k-1}p, \quad k = 1, 2, 3, \dots$$

The PMF is a direct product. Reaching the first success on trial k requires $k-1$ failures followed by one success, and independence multiplies those probabilities. The tail probability is equally compact, since needing more than k trials means the first k all failed:

$$P(X > k) = (1-p)^k.$$

$$\mu = \frac{1}{p} \quad \sigma^2 = \frac{1-p}{p^2}$$

Two conventions

Some texts define the Geometric as the number of *failures before* the first success, with support $k = 0, 1, 2, \dots$ and PMF $(1-p)^k p$. That version has mean $\frac{1-p}{p}$, exactly one less, while the variance $\frac{1-p}{p^2}$ is identical since shifting a variable by a constant does not change its spread. The two differ by the single trial on which the success occurs, so the convention in force must be established before any numerical comparison. The trials convention with mean $\frac{1}{p}$ is the one used throughout these notes.

Theorem 1.25 (Memorylessness). *The Geometric is the only discrete distribution with the memoryless property:*

$$P(X > m + n \mid X > m) = P(X > n).$$

Past failures carry no information about how many further trials are required.

Proof. Apply the definition of conditional probability. The event $\{X > m + n\}$ is contained in $\{X > m\}$, so their intersection is the former:

$$P(X > m + n \mid X > m) = \frac{P(X > m + n)}{P(X > m)} = \frac{(1 - p)^{m+n}}{(1 - p)^m} = (1 - p)^n = P(X > n). \quad \square$$

Memorylessness is the reason the recursive expectation argument works. A coin that has landed tails ten times in a row still needs $\frac{1}{p}$ further flips in expectation, so conditioning on the first trial reproduces the original problem and gives $\mathbb{E}[X] = 1 + (1 - p)\mathbb{E}[X]$, which solves to $\frac{1}{p}$.

1.5.4 Negative Binomial Distribution

Definition 1.26 (Negative Binomial). $X \sim \text{NegBin}(r, p)$ counts the number of independent trials up to and including the r -th success. Its PMF is

$$P(X = k) = \binom{k-1}{r-1} p^r (1-p)^{k-r}, \quad k = r, r+1, \dots$$

The binomial coefficient counts the arrangements of the first $r-1$ successes among the first $k-1$ trials. Trial k is forced to be a success, which is what pins the r -th success to the final position rather than anywhere in the sequence.

$$\mu = \frac{r}{p} \quad \sigma^2 = \frac{r(1-p)}{p^2}$$

Both follow from writing $X = G_1 + G_2 + \dots + G_r$, a sum of r independent $\text{Geometric}(p)$ waiting times. Memorylessness is what guarantees each waiting period restarts fresh after a success, so the G_i are i.i.d. The mean is r times the Geometric mean by linearity of expectation. The variance is r times the Geometric variance by *independence*, not by linearity, since variances add only when covariance terms vanish.

1.5.5 Poisson Distribution

Definition 1.27 (Poisson). $X \sim \text{Poisson}(\lambda)$ counts discrete events occurring in a fixed window of time or space, when events arrive independently at a constant average rate λ per window. Its PMF is

$$P(X = k) = e^{-\lambda} \frac{\lambda^k}{k!}, \quad k = 0, 1, 2, \dots$$

$$\mu = \lambda \quad \sigma^2 = \lambda$$

The equality of mean and variance serves as a diagnostic. Count data whose sample variance materially exceeds its mean is overdispersed and is not Poisson. This occurs routinely in market data, where order arrivals cluster rather than arriving at a constant rate, and it motivates the mixture and doubly stochastic models that generalize the Poisson.

Theorem 1.28 (Poisson limit of the Binomial). *Fix $\lambda > 0$ and let $n \rightarrow \infty$ with $p = \lambda/n$, so that the expected count $np = \lambda$ stays constant. Then*

$$\binom{n}{k} p^k (1-p)^{n-k} \rightarrow e^{-\lambda} \frac{\lambda^k}{k!}.$$

Proof. Substitute $p = \lambda/n$ and split the expression into three factors:

$$\underbrace{\frac{n(n-1)\cdots(n-k+1)}{n^k}}_{\rightarrow 1} \cdot \frac{\lambda^k}{k!} \cdot \underbrace{\left(1 - \frac{\lambda}{n}\right)^n}_{\rightarrow e^{-\lambda}} \cdot \underbrace{\left(1 - \frac{\lambda}{n}\right)^{-k}}_{\rightarrow 1}.$$

The first factor is a ratio of k terms each approaching n , the third is the standard limit defining $e^{-\lambda}$, and the fourth has a fixed exponent with a base approaching 1. The surviving product is the Poisson PMF. $\square$

The interpretation matters more than the algebra. The Poisson is the law of rare events, meaning many opportunities for an event to occur with a small probability at each one. This is why it models message counts, quote updates, and defaults, where no natural trial count exists but a rate does.

Superposition and thinning

Superposition. Independent Poisson streams merge into a Poisson stream with the summed rate, so $X \sim \text{Poisson}(\lambda_1)$ and $Y \sim \text{Poisson}(\lambda_2)$ independent give $X + Y \sim \text{Poisson}(\lambda_1 + \lambda_2)$. Combining order flow across venues is the standard application.

Thinning. If each event in a $\text{Poisson}(\lambda)$ stream is independently retained with probability q , the retained events form a $\text{Poisson}(\lambda q)$ stream, and the discarded ones form an independent $\text{Poisson}(\lambda(1-q))$ stream. Filtering a stream to buy orders only is the standard application.

1.5.6 Exponential Distribution

Definition 1.29 (Exponential). $X \sim \text{Exponential}(\lambda)$ is the continuous waiting time until the next event in a Poisson process of rate λ . Its density and CDF are

$$f(x) = \lambda e^{-\lambda x}, \quad F(x) = 1 - e^{-\lambda x}, \quad x \geq 0.$$

$$\mu = \frac{1}{\lambda} \quad \sigma^2 = \frac{1}{\lambda^2}$$

The Exponential is the continuous counterpart of the Geometric, and the pairing runs deep. The Geometric counts discrete trials until a success while the Exponential measures continuous time until an arrival, and each is the unique memoryless distribution on its domain:

$$P(X > s + t \mid X > s) = P(X > t).$$

The proof is the same cancellation as before, using the tail $P(X > x) = e^{-\lambda x}$. The consequence is that the expected remaining wait is $\frac{1}{\lambda}$ regardless of how long the wait has already lasted, so elapsed time carries no information about the time still to come.

Counts against waiting times

Poisson and Exponential describe one process from two directions. If events arrive at rate λ , the *count* in a unit window is $\text{Poisson}(\lambda)$ and the *gap* between consecutive events is $\text{Exponential}(\lambda)$. The identity linking them is that zero arrivals in $[0, t]$ is the same event as a wait longer than t :

$$P(N_t = 0) = e^{-\lambda t} = P(X > t).$$

Minimum of exponentials. If $X_i \sim \text{Exponential}(\lambda_i)$ independently, then $\min_i X_i \sim \text{Exponential}(\sum_i \lambda_i)$, and the probability that stream j arrives first is $\lambda_j / \sum_i \lambda_i$. This is the central result for competing arrivals, such as determining which of several order flows reaches the book first.

1.5.7 Uniform Distribution

Definition 1.30 (Continuous Uniform). $X \sim \text{Uniform}(a, b)$ places equal density across an interval:

$$f(x) = \frac{1}{b-a}, \quad F(x) = \frac{x-a}{b-a}, \quad a \leq x \leq b.$$

$$\mu = \frac{a+b}{2} \quad \sigma^2 = \frac{(b-a)^2}{12}$$

Probability for a continuous Uniform is proportional to length, so $P(c < X < d) = \frac{d-c}{b-a}$ for any subinterval. The variance depends only on the width $b-a$ and not on the location, which is the general behavior of variance under a shift.

Do not mix the discrete and continuous forms

The discrete Uniform on $\{1, 2, \dots, n\}$ has mean $\frac{n+1}{2}$ and variance $\frac{n^2-1}{12}$, which is not the continuous formula with $a = 1$ and $b = n$. A fair die has $\mu = 3.5$ and $\sigma^2 = \frac{35}{12} \approx 2.92$. Substituting into $\frac{(b-a)^2}{12}$ would give $\frac{25}{12}$, which is wrong.

Universality of the uniform

For any continuous random variable X with CDF F , the transformed variable $F(X)$ is $\text{Uniform}(0, 1)$. Running this backwards, $F^{-1}(U)$ with $U \sim \text{Uniform}(0, 1)$ has distribution F . This inverse transform is the basis for generating arbitrary distributions from a uniform random number generator, which is the foundation of Monte Carlo simulation. For the Exponential it gives the explicit sampler $X = -\frac{1}{\lambda} \ln(U)$.

1.5.8 Normal (Gaussian) Distribution

Definition 1.31 (Normal). $X \sim \mathcal{N}(\mu, \sigma^2)$ has density

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right).$$

The two parameters are exactly its mean and its variance, so the distribution is fully specified by its first two moments.

Two structural properties account for most applications.

Standardization. Any Normal reduces to the standard Normal through $Z = \frac{X-\mu}{\sigma} \sim \mathcal{N}(0, 1)$, which is why a single table of Z values suffices for every Normal question.

Closure under addition. A sum of independent Normals is Normal, with means and variances adding:

$$X_1 \sim \mathcal{N}(\mu_1, \sigma_1^2), X_2 \sim \mathcal{N}(\mu_2, \sigma_2^2) \implies X_1 + X_2 \sim \mathcal{N}(\mu_1 + \mu_2, \sigma_1^2 + \sigma_2^2).$$

Standard deviations do not add. This is the source of the $\sqrt{T}$ scaling of volatility, since variance grows linearly in time while volatility grows with the square root.

Standard tail masses

Range	Mass inside	Mass outside
$\mu \pm 1\sigma$	68.3%	≈ 1 in 3
$\mu \pm 1.645\sigma$	90%	5% per tail
$\mu \pm 1.96\sigma$	95%	2.5% per tail
$\mu \pm 2\sigma$	95.4%	≈ 1 in 22
$\mu \pm 2.576\sigma$	99%	0.5% per tail
$\mu \pm 3\sigma$	99.7%	≈ 1 in 370

The one-tailed values 1.645 and 2.326 correspond to the 95th and 99th percentiles and are the multipliers underlying standard value-at-risk calculations.

Remark (Why the Normal is the default). The Normal is the maximum entropy distribution among all distributions on the real line with a given mean and variance. Constraining only the first two moments and otherwise assuming as little as possible produces the Normal, which is the formal version of the claim that it encodes minimal structural assumptions. The Central Limit Theorem gives the second, more operational reason for its ubiquity.

Theorem 1.32 (Central Limit Theorem). *Let $X_1, \dots, X_n$ be i.i.d. with finite mean μ and finite variance σ^2 . Then the standardized sample mean converges in distribution to a standard Normal:*

$$\frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \xrightarrow{d} \mathcal{N}(0, 1)$$

The standardization is essential to the statement. The sample mean $\bar{X}_n$ on its own converges to the constant μ by the law of large numbers, and its variance σ^2/n shrinks to zero, so it is the rescaled deviation that stabilizes into a Normal shape rather than the mean itself.

Where the CLT fails

The common rule that $n \geq 30$ suffices is a heuristic, not a theorem, and it breaks in three ways that matter for trading:

- **Finite variance is required.** Heavy-tailed distributions with infinite variance, such as the Cauchy, never converge. The sample mean of Cauchy draws has the same Cauchy distribution as a single draw, so averaging buys nothing.
- **Skewness slows convergence.** Strongly asymmetric or rare-event distributions may need n in the thousands before the Normal approximation is usable in the tails, which is exactly where risk questions live.
- **Independence is required.** Financial returns are volatility clustered and serially dependent, so effective sample size is far below nominal sample size.

Empirical asset returns are leptokurtic, meaning excess kurtosis above the Normal value of 3, so extreme moves occur far more often than a Gaussian model predicts.

1.5.9 Lognormal Distribution

Definition 1.33 (Lognormal). X is lognormal when $\ln X \sim \mathcal{N}(\mu, \sigma^2)$, so $X = e^Y$ for a Normal Y . Its support is $x > 0$, and

$$\mathbb{E}[X] = e^{\mu + \sigma^2/2}, \quad \text{median}(X) = e^\mu, \quad \text{Var}(X) = (e^{\sigma^2} - 1) e^{2\mu + \sigma^2}.$$

The lognormal is the standard model for asset prices, for two structural reasons. Prices are bounded below by zero while a Normal would assign positive probability to negative prices, and returns compound multiplicatively, so the logarithm of the price is a sum of log returns and therefore Normal by the same CLT argument.

The mean sits above the median because the distribution is right-skewed. Exponentiating a Normal stretches the upper values far more than the lower ones, producing a long right tail. The median only tracks the middle draw, which is e^μ , but the mean is dragged upward by that tail to $e^{\mu + \sigma^2/2}$. The two differ by the factor $e^{\sigma^2/2}$, which sits at 1 when there is no variability and grows as the volatility σ increases. More spread in the log means a heavier right tail and a wider gap between the average outcome and the typical one.

This gap is a concrete case of a general rule: averaging after applying a curved function is not the same as applying the function to the average. Here the function is e^x , which curves upward, so $\mathbb{E}[e^Y]$ comes out larger than $e^{\mathbb{E}[Y]}$. The excess is exactly the $e^{\sigma^2/2}$ term. The same correction reappears in the drift of geometric Brownian motion, the standard continuous-time price model, for the same reason.

1.5.10 Reference Tables

Discrete distributions

Distribution	Cue in the problem	PMF	Mean	Variance	Support
Bernoulli(p)	one trial, success or failure	$p^x(1-p)^{1-x}$	p	$p(1-p)$	$\{0, 1\}$
Binomial(n, p)	successes in n fixed trials	$\binom{n}{k} p^k (1-p)^{n-k}$	np	$np(1-p)$	$0, \dots, n$
Hypergeom.	successes, no replacement	$\frac{\binom{K}{k} \binom{N-K}{n-k}}{\binom{N}{n}}$	np	$np(1-p) \frac{N-n}{N-1}$	$0, \dots, n$
Geometric(p)	trials until 1st success	$(1-p)^{k-1} p$	$\frac{1}{p}$	$\frac{1-p}{p^2}$	$1, 2, \dots$
NegBin(r, p)	trials until r -th success	$\binom{k-1}{r-1} p^r (1-p)^{k-r}$	$\frac{r}{p}$	$\frac{r(1-p)}{p^2}$	$r, r+1, \dots$
Poisson(λ)	count at a rate, no trial count	$e^{-\lambda} \frac{\lambda^k}{k!}$	λ	λ	$0, 1, 2, \dots$
Uniform $\{1, n\}$	equally likely integers	$\frac{1}{n}$	$\frac{n+1}{2}$	$\frac{n^2-1}{12}$	$1, \dots, n$

Continuous distributions

Distribution	Cue in the problem	PDF	Mean	Variance	Support
Uniform(a, b)	equally likely over a range	$\frac{1}{b-a}$	$\frac{a+b}{2}$	$\frac{(b-a)^2}{12}$	$[a, b]$
Exponential(λ)	wait until next arrival	$\lambda e^{-\lambda x}$	$\frac{1}{\lambda}$	$\frac{1}{\lambda^2}$	$x \geq 0$
Normal(μ, σ^2)	sum of many small effects	$\frac{1}{\sqrt{2\pi\sigma^2}} e^{-(x-\mu)^2/2\sigma^2}$	μ	σ^2	$\mathbb{R}$
Lognormal(μ, σ^2)	multiplicative growth	$\frac{1}{x\sqrt{2\pi\sigma^2}} e^{-(\ln x - \mu)^2/2\sigma^2}$	$e^{\mu + \sigma^2/2}$	$(e^{\sigma^2} - 1) e^{2\mu + \sigma^2}$	$x > 0$

Moment generating functions

The MGF is $M_X(t) = \mathbb{E}[e^{tX}]$. It determines the distribution uniquely, generates moments through $\mathbb{E}[X^n] = M_X^{(n)}(0)$, and multiplies across independent sums, which is the fastest proof that Binomials and Poissons and Normals are each closed under addition.

Distribution	$M_X(t)$	Distribution	$M_X(t)$
Bernoulli(p)	$1 - p + pe^t$	Poisson(λ)	$e^{\lambda(e^t-1)}$
Binomial(n, p)	$(1 - p + pe^t)^n$	Exponential(λ)	$\frac{\lambda}{\lambda - t}, t < \lambda$
Geometric(p)	$\frac{pe^t}{1 - (1 - p)e^t}$	Normal(μ, σ^2)	$e^{\mu t + \sigma^2 t^2 / 2}$
NegBin(r, p)	$\left(\frac{pe^t}{1 - (1 - p)e^t} \right)^r$	Lognormal	undefined for $t > 0$

As a note on the lognormal entry, the right tail is heavy enough that $\mathbb{E}[e^{tX}]$ diverges for every $t > 0$, which is a compact example of a distribution that has all moments yet no MGF.

How the families connect

From	Relationship	To
Bernoulli	sum of n i.i.d. trials	Binomial
Bernoulli	count trials to 1st success	Geometric
Geometric	sum of r i.i.d. waits	Negative Binomial
Binomial	$n \rightarrow \infty, p \rightarrow 0, np = \lambda$ fixed	Poisson
Binomial	n large, $np(1 - p)$ large	Normal
Poisson	gap between arrivals	Exponential
Geometric	continuous time limit	Exponential
Exponential	minimum of independent draws	Exponential (rates add)
Normal	exponentiate	Lognormal
Any distribution	standardized mean, finite variance	Normal (CLT)
Any continuous	apply own CDF	Uniform(0, 1)

Memorylessness is the thread through the waiting-time column. The Geometric and the Exponential are the unique discrete and continuous distributions with that property, which is why both admit one-line recursive expectation arguments.

1.5.11 Select Worked Problems

Problem 1.36

A basketball player makes 70% of free throws. In 10 attempts, what's $P(\text{exactly 8 makes})$?

Solution. We are given a target out of 10 independent trials, each with the same probability, so we can use the Binomial distribution:

$$P(X = 8) = \binom{10}{8} \cdot 0.7^8 \cdot 0.3^2 \approx 0.233$$

Problem 1.37

A website gets 2 signups per hour on average. What's $P(0 \text{ signups in the next hour})$?

Solution. The problem gives a rate over a timeframe, so this uses the Poisson or Exponential, this time we can use either. Working in hours, the rate is $\lambda = 2$ per hour and the window is $t = 1$.

Poisson. Count the signups directly and ask for zero of them:

$$P(X = 0) = e^{-\lambda} \frac{\lambda^0}{0!} = e^{-2} \approx 0.135.$$

Exponential. Zero signups in the hour is the same event as the wait for the first signup exceeding one hour, which is the tail probability $P(X > t) = 1 - F(t)$:

$$P(X > 1) = 1 - (1 - e^{-\lambda t}) = e^{-\lambda t} = e^{-2} \approx 0.135.$$

The two agree, which is the identity $P(N_t = 0) = P(X > t)$ between a Poisson count of zero and an Exponential wait past t .

Problem 1.38

You keep drawing cards (with replacement) from a deck until you get an ace. Expected number of draws?

Solution. We want to know the expected trials until the first success with replacement, which is the mean of the Geometric distribution:

$$\mathbb{E}[X] = \mu = \frac{1}{p} = \frac{1}{4/52} = 13$$

Problem 1.39

A factory produces items with a 5% defect rate. In a batch of 20, what's the expected number of defective items and $P(\text{at most 1 defective})$?

Solution. We want the successes (defects) in 20 fixed trials, which calls for the Binomial distribution:

$$\mathbb{E}[X] = \mu = np = 20 \cdot 0.05 = 1$$

$$P(X \leq 1) = P(0) + P(1) = \left(\binom{20}{0} \cdot 0.05^0 \cdot 0.95^{20} \right) + \left(\binom{20}{1} \cdot 0.05^1 \cdot 0.95^{19} \right) = 0.358 + 0.377 = 0.736$$

Problem 1.40

On average, a bus arrives every 10 minutes. What's $P(\text{waiting more than 15 minutes})$?

Solution. We want the tail probability of the Exponential distribution, making our unit of time 1 minute, then $\lambda = 0.1$.

$$P(X > 15) = 1 - (1 - e^{-0.1 \cdot 15}) = e^{-1.5} \approx 0.223$$

Problem 1.41

Test scores are Normal with mean 70 and standard deviation 10. What fraction of students score above 90?

Solution. This is simply asking what proportion of a normal distribution exists in the upper right tail two standard deviations from the mean. We know that 95.4% of the data lies within 2σ , then the portion only in the right tail is $\frac{1-0.954}{2} = 0.023 = 2.3\%$

Problem 1.42

You roll a die until you've gotten two 6's total. Expected number of rolls?

Solution. We can use linearity of expectation along with the Geometric distribution (or more directly, the Negative Binomial). Then

$$\mathbb{E}[X] = \frac{r}{p} = \frac{2}{1/6} = 12$$

Problem 1.43

Errors on a page occur at a rate of 0.5 per page (Poisson). A book has 200 pages. What's the expected total errors? What's the approximate probability the book has fewer than 90 errors?

Solution. Scaling the rate to the full book, the total is Poisson with

$$\mathbb{E}[X] = 200 \cdot 0.5 = 100 = \lambda, \quad \sigma = \sqrt{\lambda} = 10.$$

The book total is a sum of 200 independent per-page Poisson counts, so by the CLT it is approximately Normal, and with $\lambda = 100$ well above the $\lambda \approx 10$ threshold the approximation is safe: $X \approx \mathcal{N}(100, 100)$.

Fewer than 90 errors is one standard deviation below the mean, so by the 68% rule

$$P(X < 90) \approx \frac{1 - 0.68}{2} = 0.16.$$

Problem 1.44

You flip a coin with $P(H) = p$. What's the variance of the number of heads in n flips? For what value of p is the variance maximized?

Solution. This is a binary outcome with a fixed number of flips, which calls for the Binomial distribution. The variance is therefore $\sigma^2 = np(1-p)$. This is maximized at $p = \frac{1}{2}$.

1.6 Markov Chains

A Markov chain models a system that moves between states in steps, where the rule for the next move depends only on where the system is now.

Definition 1.34 (Markov Property). A process has the Markov property when the distribution of the next state depends only on the current state. The path taken to reach the current state carries no additional information.

This is memorylessness generalized from waiting times to states. The Geometric and Exponential forgot how long they had already waited. A Markov chain forgets its entire past trajectory and keeps only its current position, which is what makes the current state a sufficient summary for predicting the future.

Definition 1.35 (States and Transitions). A *state* is a complete description of the system at one step. Each state carries *transition probabilities* giving the chance of moving to each other state in one step. A *transient* state is one the chain eventually leaves, and an *absorbing* (terminal) state is one it can never leave, such as “win” or “lose”. Collecting the transition probabilities into a matrix P , with entry P_{ij} the probability of moving from state i to state j , gives the *transition matrix*. Each row sums to 1, since the chain must go somewhere.

Two questions a Markov chain answers

Problems split by whether the chain can get stuck.

- **Absorbing chains** contain terminal states the process eventually lands in. The questions are which absorbing state it reaches, a *hitting probability*, and how long it takes to get there, a *hitting time*. Both are solved by first-step analysis.
- **Recurrent chains** keep moving with no absorbing state. The question is the long-run fraction of time spent in each state, given by the stationary distribution.

1.6.1 Absorbing Chains and First-Step Analysis

A problem with terminal states is solved by **first-step analysis**, which is the same conditioning-on-the-next-move idea behind the recursive expectation examples earlier. Condition on the single step the chain takes from its current state, and write the quantity at that state as a weighted average of the same quantity at the neighboring states, with the transition probabilities as weights. The recipe is:

1. enumerate the states and mark each as transient or absorbing
2. for each transient state, write the quantity of interest as a weighted average over the states reachable in one step
3. use the absorbing states to set the boundary conditions
4. solve the resulting system of equations

The quantity of interest is whichever the problem asks for. Filling it with a probability yields a hitting probability, and filling it with an expected step count yields a hitting time.

Example 1.36 (Gambler's Ruin). A gambler starts with k dollars and bets \$1 at a time, winning a dollar with probability p and losing one with probability $q = 1 - p$. Play continues until the stake reaches \$0 (ruin) or a target N . The state space is $\mathcal{S} = \{0, 1, \dots, N\}$, with 0 and N absorbing.

Let $u_i = P(\text{ruin} \mid \text{start at } i)$. Conditioning on the first bet gives the first-step equation

$$u_i = p u_{i+1} + q u_{i-1},$$

with boundary conditions set by the absorbing states,

$$u_0 = 1 \quad (\text{already ruined}), \quad u_N = 0 \quad (\text{already at target}).$$

Writing $r = \frac{q}{p}$, the solution for a starting stake k is

$$u_k = \frac{r^k - r^N}{1 - r^N}.$$

In the fair case $p = q = \frac{1}{2}$ this is undefined as written, and the equation instead solves to

$$u_k = \frac{N - k}{N},$$

so the probability of reaching the target is $\frac{k}{N}$, simply the stake as a fraction of the goal.

Expected time to absorption

The expected number of steps until the chain is absorbed, starting from state k , is another first-step analysis. Writing D_k for that expected duration,

$$D_k = 1 + p D_{k+1} + q D_{k-1}, \quad D_0 = D_N = 0,$$

where the leading 1 counts the current step. For the fair Gambler's Ruin ($p = q = \frac{1}{2}$) this solves to

$$D_k = k(N - k).$$

The product form matches intuition. Duration is longest when the gambler starts in the middle, far from both barriers, and shrinks to zero at either edge.

1.6.2 Recurrent Chains and the Stationary Distribution

When a chain has no absorbing states and keeps moving indefinitely, the relevant question is not where it ends but how its time is divided among the states over the long run.

Definition 1.37 (Stationary Distribution). For a chain with transition matrix P , a stationary distribution is a probability vector π satisfying

$$\pi P = \pi, \quad \sum_i \pi_i = 1.$$

It is the long-run fraction of time the chain spends in each state. The equation $\pi P = \pi$ says that a chain already distributed as π reproduces that same distribution after one step.

For a chain that is irreducible and aperiodic, meaning every state is reachable from every other and the chain does not cycle in a fixed period, this π is unique and the chain settles into it regardless of where it started.

1.6.3 Select Worked Problems

Problem 1.45

A gambler has \$4 and wants to reach \$7, betting \$1 each round on a fair game. What's the probability of reaching \$7 before going broke?

Solution. Since the game is fair, we can assume that $p = \frac{1}{2}$. Then the probability of reaching the target is $\frac{k}{N} = \frac{4}{7}$

Problem 1.46

You flip a coin. What's the expected number of flips to get 3 heads in a row?

Solution. $\mathcal{S} = \{S_0, S_1, S_2, S_3\}$, where the subscript indicates the number of consecutive heads in the state. Then the only absorbing state is S_3 .

We then create a system of transition equations, where each state has a 50% chance to progress or to go back to zero consecutive heads:

$$E_0 = 1 + \frac{1}{2}E_0 + \frac{1}{2}E_1$$

$$E_1 = 1 + \frac{1}{2}E_0 + \frac{1}{2}E_2$$

$$E_2 = 1 + \frac{1}{2}E_0 + \frac{1}{2}E_3$$

$$E_3 = 0$$

We then backsubstitute to solve for E_0 :

$$E_2 = 1 + \frac{1}{2}E_0 + \frac{1}{2}0 = 1 + \frac{1}{2}E_0$$

$$E_1 = 1 + \frac{1}{2}E_0 + \frac{1}{2}\left(1 + \frac{1}{2}E_0\right) = \frac{3}{2} + \frac{3}{4}E_0$$

$$E_0 = 1 + \frac{1}{2}E_0 + \frac{1}{2}\left(\frac{3}{2} + \frac{3}{4}E_0\right) = \frac{7}{4} + \frac{7}{8}E_0$$

$$\rightarrow E_0 = 14$$

Problem 1.47

You roll a die repeatedly. What's the expected number of rolls until the running sum is a multiple of 3?

Solution. Divisibility by three makes the relevant states the residue classes of the running sum modulo 3: $\mathcal{S} = \{[0], [1], [2]\}$, with $[0]$ the target. A die value is uniform mod 3, since the faces $1, \dots, 6$ give residues $1, 2, 0, 1, 2, 0$, two of each. Adding a uniform residue to any current sum lands on a uniform residue, so each roll moves to one of the three classes with probability $\frac{1}{3}$. This folds the six equally likely rolls into three equally likely transitions.

The state $[0]$ plays two roles that must be separated. As the *target* it is absorbing, so no further rolls are needed once the sum is a multiple of 3, i.e., $E_0 = 0$.

As the *start* the running sum is 0, but the target requires arriving there by a roll, so the first move is forced and the start does not inherit the absorbing value. It gets its own equation below. Let E_i be the expected additional rolls to reach a multiple of 3 from residue i . The transient states [1] and [2] give

$$E_1 = 1 + \frac{1}{3}E_0 + \frac{1}{3}E_1 + \frac{1}{3}E_2$$

$$E_2 = 1 + \frac{1}{3}E_0 + \frac{1}{3}E_1 + \frac{1}{3}E_2$$

By symmetry $E_1 = E_2 = E$, and with $E_0 = 0$ each reduces to

$$E = 1 + \frac{2}{3}E \longrightarrow \frac{1}{3}E = 1 \longrightarrow E = 3$$

Finally the forced first roll from the start:

$$R = 1 + \frac{1}{3} \underbrace{E_0}_{=0} + \frac{1}{3}E_1 + \frac{1}{3}E_2 = 1 + \frac{1}{3}(3) + \frac{1}{3}(3) = 3$$

Checks. Each roll independently lands on a multiple of 3 with probability $\frac{1}{3}$ regardless of the current sum, so the count is Geometric($\frac{1}{3}$) with mean $\frac{1}{p} = 3$. Equivalently, this is the expected return time to state [0], which for an irreducible chain is $\frac{1}{\pi_0}$. The residue chain is symmetric with $\pi = (\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$, giving $\frac{1}{\pi_0} = 3$.

Problem 1.48

A particle starts at position 0 on a number line. Each step: +1 with probability $\frac{2}{3}$, -1 with probability $\frac{1}{3}$. What's the probability it ever reaches -3?

Solution. With no upper barrier, this is one-sided ruin: against an upward drift, what is the chance the walk ever gets 3 levels down. For a biased walk that steps toward the target with probability p and away with probability p' , where $p < p'$, the probability of ever reaching a point n levels below the start is $(p/p')^n$. The ratio is below 1 since the walk drifts away from the target, and it is raised to the n because the n levels are cleared one at a time, each an independent copy of the same descent. Here the walk moves down with probability $p = \frac{1}{3}$ and up with probability $p' = \frac{2}{3}$, and $n = 3$, so

$$P(\text{ever reach } -3) = \left(\frac{1/3}{2/3}\right)^3 = \left(\frac{1}{2}\right)^3 = \frac{1}{8}$$

This ratio is the same $r = q/p$ that drives Gambler's Ruin, here with the upper barrier sent to infinity. When the walk is fair the ratio equals 1 and every level is reached with certainty.

2 Brainteasers

2.1 Strategies

Most of these problems are solved by recognizing a recurring structure rather than by extended calculation. A given problem usually reduces to one of a small set of standard techniques, and the central skill is identifying which one applies. The approach proceeds in three phases: frame the problem precisely, map it to a known reduction, and verify the result against limiting cases.

2.1.1 Framing

Much of the work happens at the setup stage. An error introduced while framing a problem propagates through every later step and cannot be repaired by correct algebra, so the setup warrants explicit attention before any technique is chosen.

Framing checklist

- State every implicit assumption: fairness of coins and dice, independence of trials, sampling with or without replacement, and the exact meaning of “random”, meaning which distribution over which set
- Fix the sample space and the unit of observation. Counting the wrong objects as equally likely is the most common source of probability errors
- Classify the target quantity as a probability, an expectation, a count, a ratio, an optimal strategy, or a worst-case guarantee. The class selects the toolset

The choice of observational unit deserves particular care. A single experiment supports several sample spaces, and only some of them make the outcomes equally likely. A direct count is valid only at the level where that symmetry holds, so the first task is to identify that level and count there.

2.1.2 Core Probabilistic Reductions

Most probability puzzles reduce to one of the techniques below:

Cue to reduction

Cue in the problem	Reduction
“at least one”	complement, $1 - P(\text{none})$
“expected number of ...”	linearity of expectation with indicators
exchangeable items, “ A before B ”	symmetry
“until the first ...”, repeated identical trials	conditioning on the first step
waiting for a pattern, random walk, absorbing states	Markov states
updating on evidence, false positives	Bayes, odds form
fixed trial count, rate, or waiting structure	recognize the named distribution
sum or average of many independent pieces	normal approximation, CLT

Three techniques recur often enough to warrant restating:

Linearity with indicators. The expected size of a count equals the sum of the probabilities of its parts, whether or not those parts are independent. The method is to write the count as a sum of indicator variables, one per event, and take expectations term by term. Its power comes from sidestepping dependence entirely, since linearity holds without any independence assumption, which reduces an intractable joint distribution to a list of individual probabilities.

Symmetry. When several items are exchangeable, every statement about their relative order is uniform over the possible orderings. This converts a probability into a counting ratio with no further work. Two consequences appear repeatedly: a specific item A precedes a specific item B with probability $\frac{1}{2}$, and among k competing items each is equally likely to be first. When the items instead compete at rates λ_i , the uniform $\frac{1}{k}$ generalizes to the rate share $\lambda_j / \sum_i \lambda_i$.

Conditioning and self-similarity. A process is self-similar when its state after one step is a scaled copy of the original problem. Conditioning on that first step then expresses the unknown in terms of itself, producing a solvable equation. This is recursive expectation for waiting times and first-step analysis for Markov chains, and it is the standard route for any “expected time until” quantity.

2.1.3 Combinatorial and Logical Tactics

These tactics address counting problems and questions of the form “is a configuration reachable” or “must something exist”.

Small cases and induction. Compute the answer for $n = 1, 2, 3$, conjecture the pattern, then confirm it by induction or a direct argument. Beyond producing the formula, small cases expose the recursive structure that conditioning later exploits.

Invariants. An invariant is a quantity left unchanged by every legal move. If two states differ in the value of some invariant, no sequence of moves connects them, which resolves reachability and “is it possible” questions without any search. The usual construction assigns each configuration a value in a small set, often through a parity or coloring argument, and shows that every move preserves it.

Pigeonhole and extremal arguments. If more items than categories are distributed among the categories, some category holds at least two. The extremal variant examines the largest or smallest element of a configuration to force a contradiction or an existence claim. Both establish that something must occur without exhibiting it, which is what makes them suited to existence questions.

Working backwards. When the goal state is simple and the starting state is complex, reason from the end. The value of the final position is known outright, and each earlier position is defined in terms of positions closer to the goal, which is the structure underlying optimal-stopping and game problems.

Generating functions. Encoding a sequence as the coefficients of a power series turns operations on the sequence into algebra on the series. Convolution becomes multiplication, so counting problems with additive constraints and distributions of sums of independent variables both reduce to expanding a product.

2.1.4 Adversarial and Sequential Decisions

These problems involve a decision maker choosing actions over time, sometimes against an opponent, to optimize an outcome.

Backward induction. In a finite game of alternating moves with perfect information, solve from the last move outward. At each position a player selects the move that optimizes their outcome assuming optimal play thereafter. Applied recursively, this fixes the value of every position together with the equilibrium strategy.

Dominant strategies and minimax. A dominant strategy is optimal regardless of the opponent’s choice, and when one exists the analysis reduces to it. When none exists, a zero-sum game is solved by minimax, in which each player minimizes their maximum loss. With

mixed strategies permitted, the two players' values coincide at a single value of the game, which is von Neumann's minimax theorem.

Optimal stopping. When observations arrive in sequence and each accept-or-reject decision is final, the problem is one of optimal stopping, and its solution is typically a threshold rule. Observe without committing for a calibrated initial portion of the sequence to establish a baseline, then accept the first later observation that exceeds it. In the canonical case of selecting the best of n candidates presented in random order, the initial portion is a fraction $1/e$ and the resulting success probability also approaches

$$\frac{1}{e} \approx 0.368$$

Fair games and martingales. A fair bet has expected value zero, and a sequence of fair bets forms a martingale, whose expected future value equals its present value regardless of history. No stopping rule applied with finite capital converts a fair game into a favorable one in expectation, which is the formal reason betting systems fail. Gambler's Ruin is the standard model, and its ruin and duration formulas quantify the outcome.

Remark (Kelly sizing). For a favorable repeated bet, the wealth fraction that maximizes long-run growth is the edge divided by the odds, which for an even-money bet at win probability p is $2p - 1$. Staking more than this fraction raises variance without raising growth, and staking well beyond it drives the long-run growth rate negative despite the positive edge.

2.1.5 Counting Information

A separate class of problems asks for the minimum number of measurements that guarantee a result in the worst case. The governing tool is outcome counting. A measurement with b distinguishable results partitions the remaining possibilities into b groups, so k measurements distinguish at most b^k cases. Resolving M possibilities therefore requires

$$k \geq \log_b M$$

Worst-case guarantees by outcome counting

Measurement	Outcomes per step	k steps resolve
yes/no question	2	2^k
balance weighing	3	3^k

A balance produces three outcomes, left heavy, right heavy, or level, so k weighings resolve 3^k cases, and locating one heavy item among N of known direction needs $\lceil \log_3 N \rceil$. If the odd item may be either heavier or lighter, each candidate splits into two hypotheses and the requirement becomes $3^k \geq 2N$.

The same bound gives $\lceil \log_2 M \rceil$ yes/no questions, achieved by binary search. When the possibilities are not equally likely, the bound sharpens from a raw count to the entropy: the expected number of yes/no questions needed is at least $H(X)$ bits.

2.1.6 Estimation

Order-of-magnitude questions are handled by decomposition. Break the unknown into a product of factors, each estimable to within a small multiple, anchor on a known reference quantity, estimate each factor independently, and combine. Because the factor errors are multiplicative and roughly independent, they tend to partially cancel rather than accumulate, so a chain of rough estimates usually lands within a factor of two or three of the true value.

2.1.7 Verification

A closed-form answer should survive a set of inexpensive checks before it is trusted. Each check tests the answer against a case where the correct value is already known or constrained.

Sanity checks

- **Boundary.** Send a parameter to an extreme, such as $p \rightarrow 0$, $p \rightarrow 1$, $n = 1$, or $n \rightarrow \infty$, and confirm the formula reduces to the obvious value
- **Range.** A probability lies in $[0, 1]$, an expected number of trials is at least 1, and a variance is nonnegative
- **Monotonicity.** The answer should move in the correct direction as each parameter increases
- **Symmetry.** If the setup is symmetric in two variables, the answer must be symmetric in them as well
- **Small case.** Evaluate by hand at $n = 2$ or 3 and match the result against the formula

2.2 Select Worked Problems

Problem 2.1 [Easy]

Five pirates need to divide 100 Gold Coins. Pirates have a hierarchy, from Level 1 to level 5. The highest level pirate is the leader. The leader proposes a plan to distribute the gold and all the pirates vote on it (including the leader). If at least 50% of the pirates agree on the plan, the gold is split according to the proposal. If not, level-5 pirate is kicked from the ship, and the level-4 pirate now proposes a new plan. This process continues until a proposal is accepted. All pirates are extremely smart and extremely greedy. How does level-5 Pirate divide the treasure?

Solution. We approach this problem game-theoretically and dynamically:

- 2 pirates → Pirate 2 proposes 100 coins for themselves and zero for Pirate 1
- 3 pirates → Pirate 3 bribes Pirate 1 with one coin and keeps 99 for themselves, which they accept because they get zero coins if Pirate 3 is booted off.
- 4 pirates → Pirate 4 bribes Pirate 2 with one coin and keeps 99 for themselves, in the same manner as the 3-pirate case.
- 5 pirates → Pirate 5 bribes pirates 1 and 3 with one coin each, keeping 98 for themselves.

Thus, the final division is [98, 0, 1, 0, 1].

Problem 2.2 [Easy]

An explorer travels 1 mile South, 1 mile East, then 1 mile North, and finds themselves back at the starting point. The explorer spots a bear, what is the color of the bear?

Solution. The bear must be white. There are a few relevant scenarios that could have resulted in the explorer starting back where they began. In the first case, they started at the North pole, then travelling South equates to travelling directly away from the Pole. They can then travel *any* distance East or West, the following mile back North will return them to their destination. There are no bears in the South Pole so we omit this, though the same logic applies (with the Eastward movement equivalent to spinning in place or not moving at all).

There are other solutions near the South Pole. If the traveler's initial one-mile journey South ends on a latitude circle with a circumference of $\frac{1}{n}$ ($n \in \mathbb{Z}^+$) miles, walking one mile East will complete n full rotations around the pole and leave them exactly where they started walking East, meaning the following mile North would place them in the same starting position.

Problem 2.3 [Easy]

There is a light bulb inside a room and three switches outside. All switches are currently in off state and only one switch controls the light bulb. You may turn any number of switches on or off any number of times you want. How many times do you need to go into the room to figure out which switch controls the light bulb?

Solution. We only need to enter the room once. The first switch is flipped and left on for enough time that the bulb becomes hot. It is then flipped off and the second switch is flipped, we then immediately enter the room. This leads to three cases:

1. Bulb is hot → the light must be off and the first switch controls the light

2. Bulb is on $\rightarrow$ the second switch controls the light
3. Bulb is off and cold $\rightarrow$ the third switch controls the light

If there were four switches, we can still infer the switch in one entry by turning on both switches 1 and 2 for a long time, then switching off switch 2 and turning on switch 3 before entering the room. This results in the following cases:

1. Bulb is on and hot $\rightarrow$ Switch 1 controls the light
2. Bulb is on and cool $\rightarrow$ Switch 3 controls the light
3. Bulb is off and hot $\rightarrow$ Switch 2 controls the light
4. Bulb is off and cool $\rightarrow$ Switch 4 controls the light

The extension to four bulbs worked because we had two binary variables (illumination and bulb temperature) which creates 4 distinct states, of which we were only using 3 in the 3-bulb case.

Problem 2.4 *[Easy]*

How do you place 50 good candies and 50 rotten candies in two boxes such that if you choose a box at random and take out a candy at random, it better be good!

We need to maximize the probability of getting a good candy when selecting a random box and a random candy from it.

Solution. On one extreme, we can place all of the good candies in one box with all of the bad candies in another. This gives us $\mathbb{E}[G] = 0.5$. Thus, we should find solutions that better than 50%. We can beat this rather simply, by placing one good candy in one box and the other 49 in the other box with all of the bad candies. This results in:

$$\mathbb{E}[G] = 0.5 * \frac{1}{1} + 0.5 * \frac{49}{99} \approx 0.747$$

Problem 2.5 *[Easy]*

In a world where everyone wants a girl child, each family continues having babies till they have a girl. What do you think will the boy-to-girl ratio be eventually?

Solution. The stopping rule shapes each family but does not affect the aggregate ratio, since it cannot bias any individual birth. Each birth is an independent fair coin, and the choice of when to stop having children cannot change the overall fraction of each sex.

Counting round by round makes this precise. Begin with N couples. In the first round all N give birth, producing $\frac{N}{2}$ girls and $\frac{N}{2}$ boys, and the couples with a girl stop. The $\frac{N}{2}$ couples with a boy have another child, producing $\frac{N}{4}$ girls and $\frac{N}{4}$ boys, and the process repeats. Each round is an independent batch of fair births and contributes equal numbers of girls and boys, so the totals remain equal and the ratio is

$$1 : 1$$

The appearance of a girl surplus comes from the family structure. Every family terminates on exactly one girl, which makes girls salient as the final child. The boys are the births preceding that girl, and their count follows a geometric distribution with $p = \frac{1}{2}$ and mean $\frac{1-p}{p} = 1$. The

average family therefore has one boy and one girl. The rule fixes the position of the girl within each family, not the proportion of each sex overall.

Problem 2.6

Hundred tigers and one sheep are put on a magic island that only has grass. Tigers can live on grass, but they want to eat sheep. If a Tiger bites the Sheep then it will become a sheep itself. If 2 tigers attack a sheep, only the first tiger to bite converts into a sheep. Tigers don't mind being a sheep, but they have a risk of getting eaten by another tiger. All tigers are intelligent and want to survive. Will the sheep survive?

Solution. This is a parity argument by backward induction, sharing the alternating logic of the pirate loot-splitting problem. The cases build from the bottom up, and the governing fact is that a tiger eats the sheep only when doing so leaves it safe, meaning the resulting one-sheep island has an outcome in which that new sheep is not itself eaten.

Consider the smallest cases. With one tiger, no other tiger threatens it, so it eats the sheep and survives. With two tigers, a tiger that eats the sheep becomes a sheep facing one remaining tiger, which is the one-tiger case in which the sheep is eaten, so neither tiger risks it and the sheep survives. With three tigers, a tiger that eats reduces the island to the two-tiger case, in which a sheep survives, so eating is safe and the sheep is eaten. With four, eating reduces to the three-tiger case, in which the sheep is eaten, so no tiger risks it and the sheep survives.

Each case is thus determined by the one below it: the sheep is eaten precisely when the case with one fewer tiger leaves the sheep safe. This alternation follows parity, so the sheep survives for an even number of tigers and is eaten for an odd number. With 100 tigers, the sheep survives.

Problem 2.7

A duck is sitting at the center of a circular lake. A fox is waiting at the shore, not able to swim, wishing to eat the duck. The Fox can move around the whole lake at a speed four times the speed at which the duck can swim. The duck can fly, but only once it reaches the shore of the lake, it can't fly from the water directly. Can the duck always reach the shore without being eaten by the fox?

Solution. Let R be the radius of the lake, v the duck's speed, and $4v$ the fox's. A direct dash across fails: the duck would swim R while the fox runs at most half the circumference, πR . Since $\pi < 4$, the fox covers its distance in less time and arrives first.

Instead the duck swims in a small circle concentric with the lake, of radius slightly less than $\frac{R}{4}$. The choice of $\frac{R}{4}$ comes from comparing angular speeds. A duck at radius r sweeps angle at rate $\frac{v}{r}$, while the fox running the shore sweeps at rate $\frac{4v}{R}$, so the duck out-rotates the fox exactly when

$$\frac{v}{r} > \frac{4v}{R} \iff r < \frac{R}{4}$$

The fox tracks the duck from the nearest point of the shore, but since the duck gains angle steadily, the separation between them grows until the duck sits diametrically opposite the fox.

At that moment the duck turns and swims straight for the shore. It must cover $R - \frac{R}{4} = \frac{3R}{4}$, while the fox must run half the circumference, πR . Their times are

$$t_{\text{duck}} = \frac{3R}{4v}, \quad t_{\text{fox}} = \frac{\pi R}{4v}$$

so the comparison reduces to $3 < \pi$, which holds. The duck reaches the shore first and escapes.

Problem 2.8

An airplane flies in a straight line from airport A to airport B , and then back in a straight line from B to A . It travels with a constant engine speed and there is no wind. Will the total travel time for the same round trip be greater, less or the same if, throughout both the flights a constant wind blows from A to B ?

Solution. The round trip takes longer with wind. Take a plane speed of 100 mph, a wind of 10 mph, and a distance of 100 miles each way. The wind adds to the ground speed on one leg and subtracts on the other, giving

$$\frac{100}{110} + \frac{100}{90} \approx 2.02 \text{ hours}$$

against 2 hours with no wind.

In general, for distance d , plane speed v , and wind $w < v$, the round trip takes

$$\frac{d}{v+w} + \frac{d}{v-w} = \frac{2dv}{v^2 - w^2}$$

which exceeds the windless time $\frac{2d}{v}$ for any $w \neq 0$, since $v^2 - w^2 < v^2$. The asymmetry is that the plane spends less time on the fast leg and more time on the slow one, so the time lost to the headwind always outweighs the time gained from the tailwind.

Problem 2.9

You are given two cords that both burn in exactly one hour, not necessarily at a uniform rate. How should you light the cords to determine a time interval of exactly 15 minutes?

Solution. We light three of the four ends at the same time. When the cord with both ends lit burns out, thirty minutes will have passed. At this point we snuff the fire on the other cord. The segment remaining can be lit from both sides and will burn for an interval of 15 minutes.

The same interval can be obtained from a single cord by maintaining four simultaneous flames, assuming that cutting and lighting take negligible time. A cord holds sixty minutes of burn time, and with k flames alight that reserve is consumed at rate k , so four flames exhaust it in $\frac{60}{4} = 15$ minutes

To begin, we cut the cord in two and light all four ends. Whenever a piece burns out and the count drops to two flames, we cut one of the remaining pieces and light the two new ends, restoring the count to four. The number of cuts required is not bounded in advance, since it depends on how unevenly the cord burns, but the flame count is held at four throughout and the cord is spent after exactly fifteen minutes.

Problem 2.10

There are 10 black socks and 10 red socks in a cupboard. Your task is to draw a few socks at random, without looking, such that you get at least one matching pair (single color). What's the least number of socks that you need to draw?

Solution. You only need to draw three. If the first two were opposite colours (not a match), then the third sock *must* match one of the first two. This is the pigeonhole principle with 3 pigeons (socks drawn) and two holes (colours).

Problem 2.11

Prove that there are two opposite points on the Earth's surface that have the same temperature. Assume that the temperature varies continuously over the surface.

Solution. Consider any great circle on the surface, such as the equator, and let $t(\theta)$ be the temperature at angle θ . Two points are opposite when their angles differ by π , so the goal is a value of θ satisfying $t(\theta) = t(\theta + \pi)$. Define $f(\theta) = t(\theta) - t(\theta + \pi)$, which is zero when the two points have equal temperatures. As a difference of continuous functions, f is continuous.

Evaluating at the ends of a half circle, and using $t(2\pi) = t(0)$ because the circle closes on itself,

$$f(0) = t(0) - t(\pi), \quad f(\pi) = t(\pi) - t(0) = -f(0)$$

so f takes values of opposite sign at 0 and π , or is zero, in which case the temperatures are equal and we are done. Otherwise, by the Intermediate Value Theorem there must be a point on the closed interval $[0, \pi]$ such that $f(\theta) = 0$, and that root is an antipodal pair at equal temperature.

This is the one-dimensional case of the Borsuk-Ulam theorem, which states that any continuous map $S^n \rightarrow \mathbb{R}^n$ admits an antipodal pair with equal values. Thematically it resembles the hairy ball theorem, in that a continuous structure on a sphere is forced into a degeneracy, though Borsuk-Ulam constrains values at antipodal pairs while the hairy ball theorem forces a tangent vector field to vanish somewhere.

Problem 2.12

The probability of having at least one accident on a road in one hour is $\frac{3}{4}$. What is the probability that at least one accident occurs in half an hour?

Solution. Assuming accidents in disjoint intervals are independent, no accident in an hour means no accident in each of its two half-hour intervals. Let q be the probability of no accident in half an hour. Then $q^2 = 1 - \frac{3}{4} = \frac{1}{4} \rightarrow q = \frac{1}{2}$. Then the probability of at least one accident in half an hour is $1 - q = \frac{1}{2}$.

Problem 2.13

Suppose you're on a game show, and you're given the choice of three doors: Behind one door is a car; behind the others, goats. You pick a door, say No. 1, and the host, who knows what's behind the doors, opens another door, say No. 3, which has a goat. Now, do you want to pick door No. 2? What is the probability to win the car if you switch?

Solution. Switching wins with probability $\frac{2}{3}$.

Suppose door 1 is chosen. The car is equally likely to be behind each door, giving three cases. If the car is behind door 1, the host opens one of the goats and switching loses. If the car is behind door 2, the host is forced to open door 3, and switching wins. If the car is behind door 3, he is forced to open door 2, and switching wins.

Switching therefore wins in two of the three equally likely cases. Equivalently, switching wins exactly when the initial pick was wrong, which occurs with probability $\frac{2}{3}$.

Problem 2.14

One hundred prisoners are lined up, facing one direction and assigned a random hat, either red or blue. Each prisoner can see the hats in front of them but not behind. Starting with the prisoner at the back of the line and moving forward, they must each, in turn, say only one word which must be "red" or "blue". If the word matches their hat color they are released, if not, they are kept imprisoned. They can hear each others' answers, no matter how far they are on the line, but they do not hear the verdict (whether the answer was correct). A friendly guard warns them one night before, giving them enough time to come up with a strategy. How many prisoners can be freed using the best strategy? Assume that there is an unknown number of red & blue hats.

Solution. Ninety-nine prisoners are guaranteed release, and the first to speak is correct with probability $\frac{1}{2}$. This is optimal, since that prisoner sees only the hats ahead and receives no information bearing on their own, so no strategy improves on a coin flip for them.

The prisoners agree in advance to encode parity. The prisoner at the back, who sees all 99 hats ahead, says "red" if the number of red hats ahead is even and "blue" if it is odd.

Each later prisoner deduces their own colour from three known quantities: the announced parity, the colours called behind them, and the hats visible ahead. Their hat is red exactly when the combined parity of the reds heard and seen differs from the announced parity, since their own hat is the only one unaccounted for and so must absorb the discrepancy. Having deduced it, they answer correctly, and their answer joins the record used by those ahead.

Problem 2.15

A traveler meets a pair of twins at a fork in the road: one path is correct and one incorrect. One of the twins always tells the truth (knight), and the other one always lies (knave). Which single question can the traveler ask one of the twins to determine the correct path?

Solution. Ask either twin: "Which path would your brother say is the good one?", then take the opposite of the path indicated.

The question works because it forces the answer to pass through both twins, one truthful and one lying, so a single inversion is applied no matter which twin is asked. Let the honest report of the good path be G . The brother being described contributes one inversion if he is the liar, and the twin answering contributes a second inversion if he is the liar. Exactly one of the two is the liar in either case, so the answer always carries exactly one inversion and points to the bad path.

Concretely, asking the knight gives a truthful report of what the knave would say, and the knave would name the bad path. Asking the knave gives a lying report of what the knight would say, and the knight would name the good path, which the knave inverts to the bad path. Both routes indicate the bad path, so the traveler takes the other one.

Problem 2.16

There are two jars, one contains water, and the other contains wine (equal volumes). A certain amount of water is transferred to the wine, then the same amount of the mixture is transferred back to the water beaker. Is there now more water in the wine than there is wine in the water?

Solution. Call the initial volume in each jar V . Each jar ends with volume V again, since equal amounts move out and back. Say the wine jar finishes with x of water, so its remaining wine is $V - x$.

Since there was initially V wine, the x missing from the wine jar must be in the water jar, with $V - x$ of water there to reach a total volume of V . Thus the amount of water in the wine jar equals the amount of wine in the water jar.

Problem 2.17

There are 100 light bulbs in a sequence, all kept in OFF state. The first person comes and toggles all the bulbs which are at a position that is a multiple of 1, i.e. switch all bulbs to ON. The second person toggles all multiples of 2, i.e. turn the even-numbered bulbs OFF and leave the odd ones as is. The third person comes and toggles all multiples of 3. This process continues until the 100th person does their part. After this, how many bulbs are in ON state?

Solution. 10 bulbs will be on. A bulb ends ON if it is toggled an odd number of times, and bulb n is toggled once by each person whose number divides n , so it ends ON exactly when n has an odd number of divisors. This happens only for perfect squares, and the number of perfect squares up to n is $\lfloor \sqrt{n} \rfloor$, which is $\lfloor \sqrt{100} \rfloor = 10$.

The prime factorization of a number n follows the form $n = p_1^{e_1} \cdot p_2^{e_2} \cdots p_k^{e_k}$, where e_i counts the copies of the prime p_i . A divisor d of n has to use a subset of that factorization: $d = p_1^{a_1} \cdot p_2^{a_2} \cdots p_k^{a_k}$, $0 \leq a_i \leq e_i$. There are $e_i + 1$ choices for each exponent, so the total number of divisors is

$$\tau(n) = \prod_{i=1}^k (e_i + 1)$$

n is a perfect square when every e_i is even, since squaring doubles each exponent. Then each $e_i + 1$ is odd, and a product of odd numbers is odd, so a perfect square has an odd number of divisors and no other number does.

Problem 2.18

We have a weighted coin that shows a Head with probability p and Tails with $1-p$, ($0.5 < p < 1$). How do we define two equally likely events using this coin? Each event is a function of the outcomes of one or more coin tosses.

Solution. We want two equally likely events, each built from coin toss outcomes. We do *not* know p , so we anchor the equality on something independent of p . Flip the coin twice and read the ordered pair. Define Event 1 as HT and Event 2 as TH , and discard HH or TT .

These are equally likely because $P(HT) = p(1-p)$ and $P(TH) = (1-p)p$ are the same product, equal for every p . Discarding the tied outcomes HH and TT leaves the remaining probability split evenly between the two.

The flips must be read in fixed, non-overlapping pairs. If instead we flipped a single stream until either HT or TH first appeared, the result would be decided by the very first flip, since the run ends as soon as the coin differs from where it started. That favors HT with probability p and breaks the fairness.

Problem 2.19

Spiderman has two close friends, Mary Jane & Gwen Stacy. After every mission, he rushes to the central subway. One line heads towards Mary's place, and another towards Stacy. Trains from each line leave every 10 minutes. Spiderman being impartial always boards the first train that leaves. However, he observes that he ends up visiting Mary Jane nine times more often than Gwen Stacy. Can you decipher why?

Solution. The key is that Spiderman boards the next train to depart, so what matters is the waiting gap *before* each train, and those two gaps are unequal. We assume his arrival time is uniform across the 10-minute cycle.

Say Mary's train departs at times 0 and 10, and Gwen's train departs at some time d in between, $0 < d < 10$. If he arrives in $(0, d)$ the next train is Gwen's, and if he arrives in $(d, 10)$ the next train is Mary's. So the interval leading to Gwen has length d and the interval leading to Mary has length $10 - d$, giving $P(\text{Mary}) = \frac{10-d}{10}$ and $P(\text{Gwen}) = \frac{d}{10}$. Concretely, we solve for a 9-to-1 ratio: $\frac{10-d}{d} = 9 \implies d = 1$.

So Gwen's train leaves just 1 minute after Mary's, which leaves a 9-minute window in which the next train goes to Mary.

Problem 2.20

An egg breaks only if dropped from above a threshold floor, within a 100-story building. Every time you drop an egg, it is counted as an attempt. You are given 2 eggs to deduce the threshold floor, and eggs can be reused if they do not break. Minimize the number of drops (trials) in the worst case.

Solution. We cannot perform a standard binary search because overestimates of the target require a linear search from the last floor to the threshold (until we find it). Considering this, we could try to optimize an equation which divides 100 into d splits and then performs $\frac{100}{d}$ trials for the linear scan:

$$\min \left(d + \frac{100}{d} \right), \quad d \in \mathbb{N}, d \in [1, 100]$$

The minimum of this can be solved using the roots of the derivative:

$$\frac{d}{dd} \left(d + \frac{100}{d} \right) = 0 \longrightarrow 1 - \frac{100}{d^2} = 0 \longrightarrow d = 10$$

which gives 19 trials in the worst case (threshold on floor 99). This is not the true optimum, the approach failed for two reasons. Firstly the function is discrete, which we only *approximated* using a derivative. Secondly, and more importantly, the equation we chose encoded a constraint that each division be the same size. To find the true optimum, we can do better by searching in cuts of *decreasing* size over time.

We use step sizes that decrease by 1. From an initial step n , we drop at floor n , then $n + (n - 1)$, then $n + (n - 1) + (n - 2)$, and so on, running a linear scan from the last passed floor whenever the egg breaks. Each band is one floor shorter than the last, so a broken first egg always leaves the worst-case total at n . The floors this covers is the triangle number $\frac{n(n+1)}{2}$, so we want the smallest n with

$$\frac{n(n+1)}{2} \geq 100$$

Which is $n = 14$. Thus, the worst case is 14 drops.

Intuitively, the decreasing steps work by *spreading* the worst case evenly across every threshold placement rather than letting it collapse onto one. In the fixed-jump attempt the cost concentrated on a single floor (99), which sat as a spike above the rest. The decreasing schedule flattens that spike, so every placement costs exactly n , and the worst case drops to meet the average. This is the general shape of a minimax problem: whenever the maximum is uneven, the tall placement has slack elsewhere to borrow from, and we keep leveling until no placement can be improved without worsening another.

Problem 2.21

1024 pirates stand in a circle. They start shooting alternately in a cycle such that the 1st pirate shoots the 2nd, the 3rd shoots the 4th and so on. The pirates who got shot are eliminated from the game. They continue in circles, shooting the next standing pirate, till only one pirate is left. Which position should someone stand to survive?

Solution. Each pass around the circle, the number of pirates decreases by a factor of 2. Since the initial total is a power of 2, every pass starts with an even number of pirates. With an even count the pirates split exactly into shooter-victim pairs, so the last pirate of the pass is a victim, and the next shooter wraps back around to position 1. Thus position 1 shoots every pass and is never shot, until they are the last one remaining. So the pirate in position 1 survives.

Problem 2.22

I bought two fair six-sided dice. Keeping the first one untouched, I modified the second die, by erasing each face and writing a number of my own. Now, the sum of outcomes of the two dice is equally likely an integer between 1 and 12. Can you deduce the numbers written on the modified die?

Solution. The modified die has three zero faces and three six faces. The key detail is that any integer from 1 to 12 is equally likely, unlike the usual two-dice sum which peaks at 7 and tapers to a single way each of making 2 or 12. Since the standard die contributes at least 1, reaching a sum of 1 forces a 0 on the modified die, and reaching 12 forces a 6. The standard die alone is already uniform over 1 to 6, so we can build a uniform 1 to 12 by taking that block and shifting a copy of it up by 6. Adding 0 leaves the sum in 1 to 6, and adding 6 moves it to 7 to 12, so to weight the two blocks equally the modified die needs three 0 faces and three 6 faces.

Problem 2.23

Assume the gender of a child can be either Boy or Girl, equally likely.

- a. Suppose I have two children. Younger one is a girl. What is the probability that both children are girls?
- b. Suppose I have two children, and atleast one of them is a boy. Find the probability that both of them are boys.
- c. Suppose I have two kids, lets call them X and Y. Suppose X is a girl. What is the probability that I have two daughters?

Solution. The three parts differ in whether the condition names a specific child or only makes a claim about the pair. Naming a specific child leaves the other as an independent fair coin. An existential “at least one” condition does not.

- a. We condition on a specific child, the younger, being a girl. Writing each outcome as (older, younger), the condition leaves the two equally likely cases $\{(B, G), (G, G)\}$, of which (G, G) is both girls. So the probability is $\frac{1}{2}$, which is just the independent chance that the older child is also a girl.
- b. Saying *at least* one is a boy names no particular child, so it removes only (G, G) . This leaves three equally likely cases $\{(B, G), (G, B), (B, B)\}$, of which only (B, B) is both boys, giving $\frac{1}{3}$. The two mixed cases both satisfy the condition, which is why the denominator is 3 rather than 2.
- c. Labeling the children and telling us X is a girl fixes a specific child, exactly as in (a). The other child is an independent fair coin, so the probability of two daughters is $\frac{1}{2}$.

Problem 2.24

In a dark room, there is a deck of 52 cards, with exactly 10 cards facing up, rest are facing down. You can touch the cards with your hands but cannot sense which cards are facing up/down. You need to split all these cards into two decks with equal number of cards facing up.

Solution. We first note that the two decks only need an *equal* number of face-up cards between them, not a count matching the original 10. The two decks also need not be the same size.

Using this, we draw 10 cards at random, matching the number of face-up cards in the deck. Suppose this pile contains x face-up cards, so the remaining 42 cards hold the other $10 - x$ face up. Since our pile has 10 cards total, it also holds $10 - x$ face-down cards. We then flip every card in the pile, which turns those $10 - x$ face-down cards up and the x face-up cards down, leaving exactly $10 - x$ face up. Both piles now show $10 - x$ face-up cards.

Problem 2.25

A certain town comprises of 100 married couples. Some husbands secretly cheat on their wives. All wives know about the nature of every husband except their own. When a wife concludes that her husband cheated, she kicks her husband into the street at midnight. All husbands remain silent about their secret. One day, the mayor of the town announces to the whole town that there is at least 1 cheating husband in the town. After announcement, no one talks, waiting for someone to get kicked. Till 9th night from announcement, no husband was kicked, but on the 10th night, some husbands got kicked out simultaneously. How many are they?

Solution. 10 husbands were kicked out. Each wife saw either 9 cheating husbands (her husband cheats) or 10 (her husband is faithful), so each knows her count is either correct or lower by 1.

A wife who sees c cheaters therefore knows every other wife sees at least $c - 1$, so no one can act before night c , and the earlier nights tell her nothing. She watches only night c . If those c are all of them, they are kicked that night. When night c passes with no one kicked, the count must instead be $c + 1$, so her own husband is the extra cheater, and she kicks him on night $c + 1$. The 10 cheaters' wives each saw 9, ignored everything up to night 9, and when it stayed quiet they kicked their husbands together on night 10.

Problem 2.26

A father claims that it snowed last night. His first daughter says that the probability of snowfall on any given night is $\frac{1}{8}$. His second daughter says that the father lies 5 out of every 6 times. Given that the father claims it snowed, what is the probability that it actually did?

Solution. We can use Bayes' Theorem directly to solve for $P(S)$, that snowfall occurred given C , the father claimed it snowed. Using $P(C|S) = 1 - \frac{5}{6} = \frac{1}{6}$:

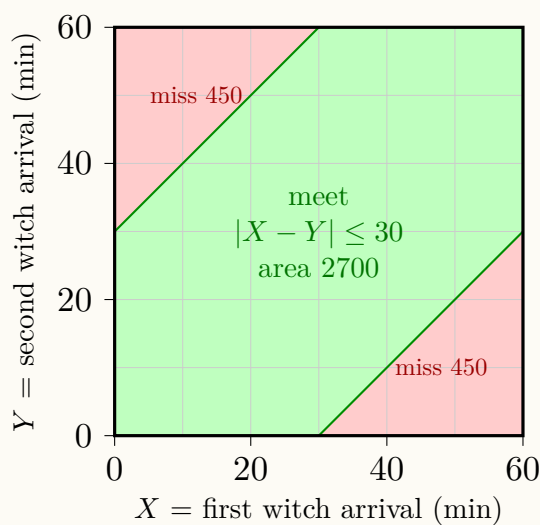
$$P(S|C) = \frac{P(C|S) \cdot P(S)}{P(C)} = \frac{\frac{1}{6} \cdot \frac{1}{8}}{\frac{1}{6} \cdot \frac{1}{8} + \frac{5}{6} \cdot \frac{7}{8}} = \frac{1}{36}$$

Problem 2.27

Two witches make a nightly visit to an all-night coffee shop. Each arrives at a random time between 0:00 and 1:00. Each one of them stays for exactly 30 minutes. On any one given night, what is the probability that the witches will meet at the coffee shop?

Solution.

The answer is $\frac{3}{4}$. If we plot each witches arrival as X and Y , then we can solve $|X - Y| \leq 30$:



Problem 2.28

Someone breaks into a king's wine cellar where he stores 1000 bottles of wine. This person proceeds to poison one of the 1000 bottles, but gets away too quickly for the king's guard to see which one he poisoned or to catch him.

The king needs the remaining 999 safe bottles for his party in 4 weeks. The king has 10 prisoners who deserve execution. The poison takes about 3 weeks to take effect, and any amount of it will kill whoever drinks it. How can he figure out which bottle was poisoned in time for the party?

Solution. We can use each prisoner's death as one bit of binary information. With 10 prisoners we can encode $2^{10} = 1024$ unique states, more than enough for 1000 bottles. We number the bottles in binary and assign each prisoner a bit position, and a prisoner drinks from a bottle whenever its bit is a 1. After 3 weeks, the set of prisoners who died reads off the poisoned bottle directly. For example, bottle 579 is 1001000011 in binary, so prisoners 1, 4, 9, and 10 drank from it, and they are the only ones who die.

Problem 2.29

Let X be a random variable that takes value 0 or 1 with 50% probability each. You need to define a new random variable Y as a function of X , such that Y has the value 1 with 25% probability and 0 otherwise.

Solution. A single draw of X can only give probabilities of 0, $\frac{1}{2}$, or 1, so we need two independent draws to reach $\frac{1}{4}$. We make Y the product of two independent instances of X :

$$Y = X_1 \cdot X_2$$

The product is 1 only when both draws are 1, which by independence happens with probability $\frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}$.

Problem 2.30

You have 25 horses and can race 5 at a time. Each race gives you the ranking of those 5 but no times. What's the minimum number of races to find the top 3 horses?

Solution. It takes seven races at minimum. We first split the 25 horses into five races of five, which we call A through E .

For the sixth race, the five group winners compete. We relabel the groups by this result, so that A_1 beats B_1 beats C_1 beats D_1 beats E_1 , where X_i is the i -th place horse in group X . This confirms A_1 as the best horse overall.

We eliminate D_1 and E_1 along with their whole groups. Each already has three proven-faster horses above it (A_1, B_1, C_1), so it cannot make the top three, and everyone behind it in its group is slower.

We prune the remaining groups by the same rule: a horse is out once three horses are proven faster than it. That eliminates C_2 and below, B_3 and below, and A_4 and below. We also set A_1 aside as the known winner. This leaves A_2, A_3, B_1, B_2 , and C_1 , exactly five horses.

We race these in the seventh race and take the top two as the second and third best horses overall.

Problem 2.31

There are 21 flags. Two players take turns removing 1, 2, or 3 flags. The player who takes the last flag wins. You go first. What's your strategy?

Solution. We work backwards from the goal. We want to leave the opponent's turn with exactly 4 flags remaining (so every move loses: whatever they take, we take the rest and win). To land them on 4, we want to have left them on 8 on the previous round (for any n they remove, we remove $4 - n$ to bring them back to 4). Using this same repeating logic, we keep leaving them on multiples of 4, up to 20. Thus, on the first turn, we remove 1 flag.

Problem 2.32

A line of 100 airline passengers is waiting to board a plane. They each hold a ticket to one of the 100 seats on that flight. For convenience, let's say that the n_{th} passenger in line has a ticket for seat number ' n '. Being drunk, the first person in line picks a random seat (equally likely for each seat). All of the other passengers are sober, and will go to their assigned seats unless it is already occupied; If it is occupied, they will then find a free seat to sit in, at random. What is the probability that the last (100th) person to board the plane will sit in their own seat (#100)?

Solution. Label seats so passenger n owns seat n , with passenger 1 drunk.

First we pin down where the 100th passenger can end up. Take any passenger k with $2 \leq k \leq 99$. When k boards, seat k is either free, in which case k takes it, or already occupied. Either way seat k is occupied once k has boarded. So when passenger 100 boards, seats $2, \dots, 99$ are full and the one free seat is 1 or 100.

Now we show these are equally likely. Seats 1 and 100 are the only seats no passenger in $\{2, \dots, 99\}$ owns. Both begin free, and every passenger who chooses at random chooses uniformly among the free seats. Swap the labels 1 and 100. Passenger 1 chooses at random no matter what he owns, passenger 100 takes the leftover no matter what she owns, and the owners of seats $2, \dots, 99$ are unaffected. So the swap sends the boarding process to an equally likely one with the two seats exchanged, and the two outcomes have equal probability.

So, passenger 100 gets seat 100 with probability $\frac{1}{2}$.

Problem 2.33

A stick is broken into 3 parts, by choosing 2 points randomly along its length. With what probability can it form a triangle?

Solution. Call the length ℓ and set $\ell = 1$. To form a triangle every piece must be shorter than $\frac{1}{2}$, since a piece of length $\frac{1}{2}$ or more equals or exceeds the sum of the other two.

Fix the first cut and measure it from its nearer end, calling that distance X , so X is uniform on $[0, \frac{1}{2}]$. Let Y be the second cut, uniform on $[0, 1]$.

If $Y < X$ the pieces are Y , $X - Y$, and $1 - X$. Here $1 - X \geq \frac{1}{2}$, so this always fails.

If $Y > X$ the pieces are X , $Y - X$, and $1 - Y$. We already have $X < \frac{1}{2}$. The remaining piece $1 - Y$ stays below $\frac{1}{2}$ when $Y > \frac{1}{2}$, and the middle piece $Y - X$ stays below $\frac{1}{2}$ when $Y < X + \frac{1}{2}$. So we succeed exactly when

$$\frac{1}{2} < Y < X + \frac{1}{2}$$

a window of width X . Given X , the success probability is therefore X .

Since the first cut is uniform on $[0, \frac{1}{2}]$, we can average with $\frac{1}{b-a} \int_a^b g(x)dx$,

$$P(\text{triangle}) = \frac{1}{\frac{1}{2} - 0} \int_0^{1/2} x \, dx = 2 \left[\frac{x^2}{2} \right]_0^{1/2} = \left(\frac{1}{2} \right)^2 - 0 = \frac{1}{4}$$

So the probability of forming a valid triangle is $\frac{1}{4}$.

Problem 2.34

A rabbit sits at the bottom of a staircase with n stairs. The rabbit can hop up only one or two stairs at a time. What kind of sequence is depicted by the different ways possible for the rabbit to ascend to the top of the stairs of length $n = 1, 2, 3 \dots$?

Solution. We'll call the number of paths to a certain stair n as $f(n)$. We can compute $f(1)$ to $f(5)$ as:

n	$f(n)$	paths
1	1	1
2	2	11, 2
3	3	111, 12, 21
4	5	1111, 112, 121, 211, 22
5	8	11111, 1112, 1121, 1211, 2111, 122, 212, 221

Examining each level from $n = 3$ onwards, we can see that each path can be constructed as the paths to $n - 2$ plus a double hop, or the paths to $n - 1$ plus a single hop. Thus, the number of paths to n is equal to the number of paths to $n - 2$ plus the number of paths to $n - 1$, which is the Fibonacci recurrence. So the pattern that arises is the Fibonacci sequence.

Problem 2.35

A , B , and C live together and share everything equally. One day A brings home 5 logs of wood, B brings 3 logs and C brings none. Then they use the wood to cook together and share the food. Since C did not bring any wood, he gives \$8 instead. How much to A and how much to B ?

Solution. The three of them consume 8 logs total, so each eats $\frac{8}{3}$ logs. Measuring what each contributed beyond his own share, A has a surplus of $5 - \frac{8}{3} = \frac{7}{3}$ logs and B has a surplus of $3 - \frac{8}{3} = \frac{1}{3}$ logs.

C 's \$8 pays for the $\frac{8}{3}$ logs he ate without providing any. Those logs were supplied entirely out of the two surpluses, so the money splits in proportion to surplus:

$$\frac{7}{3} : \frac{1}{3} = 7 : 1$$

Thus A receives \$7 and B receives \$1.

Problem 2.36

Suppose you have a hotel which has one floor with infinite number of rooms in a row and all of them are occupied. A new customer wants to check in, how will you accommodate her? What if infinite number of people want to check in, how will you accommodate them?

Lastly, suppose infinite number of buses arrive at the hotel, each having infinite number of people, how will you accommodate them?

Solution. This is a question of counting infinities, or more specifically, of finding a 1-1 mapping that includes the new incoming people. Since all of these are based on integer amounts of people, each is countable (even an infinite number of buses of infinite people).

In the first case, we can simply ask each person in room k to move to room $k + 1$. Similarly for the second case, we can ask each k_{th} person to move to room $2 \cdot k$. This would leave open every $2 \cdot k - 1$ room open for the infinite number of people arriving on the bus.

For the final case, identify each person by a pair. An original occupant is labeled by their room number $k \geq 1$. A bus passenger is labeled (i, j) , meaning seat $j \geq 1$ of bus $i \geq 1$. We assign rooms in successive shells, indexed by $n = 1, 2, 3, \dots$, where shell n contains:

- the n th original occupant,
- seats 1 through n of bus n , that is the pairs $(n, 1), \dots, (n, n)$,
- seat n of every earlier bus, that is the pairs $(1, n), \dots, (n - 1, n)$.

The bus pairs in shell n are exactly those with $\max(i, j) = n$, so every pair (i, j) lands in shell $\max(i, j)$ and lands in no other. Every occupant k lands in shell k . Each shell holds $2n$ people, a finite count, so processing the shells in order $1, 2, 3, \dots$ reaches every person after finitely many rooms are filled. No bus is skipped, since a fixed pair (i, j) is served by the time we finish shell $\max(i, j)$.

Problem 2.37

A group has 70 members. For any two members X and Y , there is a language that X speaks but Y does not, and there is a language that Y speaks but X does not. At least how many different languages are spoken by this group?

Solution. Start by considering a smaller group, say $n = 6$ with members labelled 1 to 6. We show this is supportable by 4 languages, denoted A, B, C , and D . Assign languages as sets:

$$\begin{array}{lll} 1 = \{A, B\} & 2 = \{A, C\} & 3 = \{A, D\} \\ 4 = \{B, C\} & 5 = \{B, D\} & 6 = \{C, D\} \end{array}$$

The condition is that for any two members X and Y , each speaks a language the other does not. In set terms, neither set may contain the other. Here all six sets have size 2, and no size-2 set can contain another (collections of distinct but differently sized sets could contain subsets), so any two distinct sets satisfy the condition automatically.

This suggests giving every member a set of the same size r , which is both convenient and optimal. Convenient, because the number of available sets is then just $\binom{n}{r}$. Optimal, because nothing is gained by mixing sizes: enlarging one member's set only shrinks the pool of sets that can coexist with it. By Sperner's theorem, the largest antichain (a collection of sets where none contains another) built from n elements is obtained by taking all subsets of a single size $\lfloor n/2 \rfloor$, giving $\binom{n}{\lfloor n/2 \rfloor}$ of them.

Finally, the smallest n for which $\binom{n}{\lfloor n/2 \rfloor} \geq 70$ is $n = 8$ languages.

Problem 2.38

Two players, A and B , form a team playing against an opponent. To enter, the team pays \$1 at the start. If the team wins it receives \$3, and nothing otherwise.

A and B are placed in two separate rooms with no communication between them. Each flips a fair coin. A must guess the outcome of B 's flip, and B must guess the outcome of A 's flip. The team wins only if both guesses are correct, and wins nothing if either guess is wrong. The two may agree on a strategy in advance, before being separated.

Should the team pay \$1 to play?

Solution. The two coins land in one of four equally likely states:

$$\{H, H\} \quad \{H, T\} \quad \{T, H\} \quad \{T, T\}$$

With no strategy, each player guesses the other coin at random. Each guess is correct with probability $\frac{1}{2}$, and the two guesses are independent, so both are correct with probability $\frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}$. The expected return is $\mathbb{E}[\text{Play}] = \frac{1}{4} \cdot 3 + \frac{3}{4} \cdot 0 - 1 = -\frac{1}{4}$, which is negative, so random guessing is not worth playing.

Instead, the strategy is for each player to guess that the other coin *equals their own*. Under this rule, A 's guess is correct exactly when B 's coin equals A 's, and B 's guess is correct exactly when A 's coin equals B 's. These are the same condition: both coins show the same face. So the two guesses are no longer independent, they are both correct together or both wrong together.

The team therefore wins precisely when the coins match. Of the four states, two match and two do not:

$$\underbrace{\{H, H\}, \{T, T\}}_{\text{match, win}} \quad \underbrace{\{H, T\}, \{T, H\}}_{\text{no match, lose}}$$

so the win probability is $\frac{2}{4} = \frac{1}{2}$. The expected return becomes

$$\mathbb{E}[\text{Play}] = \frac{1}{2} \cdot 3 + \frac{1}{2} \cdot 0 - 1 = \frac{1}{2} \quad \longrightarrow \quad \text{which is positive, so they should play.}$$

Problem 2.39

Snow particles fall to the ground one after another. Each snowflake is of type “Stellar Dendrite” with probability p if the preceding particle was also a Stellar Dendrite, and with probability q if the preceding particle was some other type. If a snowflake is picked at random from the ground, what is the probability that it is a Stellar Dendrite?

Solution. Let x be the probability that a snowflake picked up is an SD. We set this up as a Markov chain and use first-step analysis, conditioning on the previous flake.

Assume the system has reached a steady state, so the probability of an SD does not change from one flake to the next. Then the previous flake is an SD with the same probability x . The current flake is an SD in two ways: the previous was an SD (probability x) and it continues (probability p), or the previous was not (probability $1 - x$) and it starts (probability q). Since the current flake is itself a picked-up flake, its SD probability is also x :

$$x = x \cdot p + (1 - x) \cdot q$$

Solving,

$$x = xp + q - qx \implies x - xp + qx = q \implies x = \frac{q}{1 - p + q}$$

Problem 2.40

A drawer contains red socks and black socks. When two socks are drawn at random, the probability that both are red is $\frac{1}{2}$. (a) How small can the number of socks in the drawer be? (b) How small if the number of black socks is even?

Solution. Draws are without replacement, so for red socks r and black socks b the probability that both are red factors as

$$p = \frac{r}{r+b} \times \frac{r-1}{r+b-1} = \frac{1}{2}$$

We can test this with $r = 3$ and $b = 1$:

$$\frac{3}{4} \times \frac{2}{3} = \frac{1}{2}$$

As desired. For (b), the second factor is smaller than the first when $b > 0$, so $\frac{r}{r+b} > \frac{r-1}{r+b-1}$. Multiplying this inequality by each factor and using $p = \frac{1}{2}$ gives

$$\left(\frac{r}{r+b}\right)^2 > \frac{1}{2} > \left(\frac{r-1}{r+b-1}\right)^2$$

We solve these for bounds on r in terms of b , then check the admissible candidate r for each even b until the product equals $\frac{1}{2}$.

Problem 2.41

Four people, A , B , C and D need to get across a river. The only way to cross the river is by an old bridge, which holds at most 2 people at a time. Being dark, they can't cross the bridge without a torch, of which they only have one. So each pair can only walk at the speed of the slower person. They need to get all of them across to the other side as quickly as possible. A is the slowest and takes 10 minutes to cross; B takes 5 minutes; C takes 2 minutes; and D takes 1 minute. What is the minimum time to get all of them across to the other side?

Solution. We could first try using D to 'ferry' everyone across, coming back each time for the next person, but this takes 19 minutes and never uses people on *both* sides of the bridge.

Instead, send C and D over (2 min), D back (3 min), A and B over (13 min), C back (15 min), then C and D over again for 17 minutes total. The trick is to pair the two slowest so we pay A 's 10 minutes on the same trip as B 's, leaving a fast runner across to carry the torch back.

This beats ferrying whenever $2t_2 < t_1 + t_3$, where the t_i are the sorted times: two crossings by the second fastest must cost less than one each by the fastest and second slowest. Here $2C < D + B$, or $4 < 6$, saving 2 minutes.

Problem 2.42

Boss A 's birthday is one of the following 10 dates:

- Mar 4, Mar 5, Mar 8
- Jun 4, Jun 7
- Sep 1, Sep 5
- Dec 1, Dec 2, Dec 8

A tells you only the month and tells colleague C only the day.

1. You state: "I do not know A 's birthday, and C does not know it either."
2. C replies: "I did not know A 's birthday, but now I do."
3. You state: "Now I know it, too."

An assistant overhears the exchange and correctly determines the birthday. What is A 's birthday?

Solution. Since we know the month, our personal list of candidate birthdays is restricted to that single month. For our colleague C , the same principle applies with respect to the day.

From (1), we announce that we do not know the birthday and that C cannot possibly know it either. This means our month cannot contain any unique day numbers (7 in June, 2 in December), which eliminates both June and December entirely. The candidate list is reduced to the March and September dates:

- March: 4, 5, 8
- September: 1, 5

From (2), C announces that they now know the birthday. This rules out day 5, since 5 appears in both March and September and would leave C uncertain. The remaining candidate dates are March 4, March 8, and September 1.

From (3), we announce that we now know the birthday as well. If our month were March, we would still have two possibilities (March 4 and March 8). Because we can determine the exact date, our month must be September, making the boss's birthday September 1.

Problem 2.43

A casino offers a card game using a normal deck of 52 cards. The rule is that you turn over two cards each time. For each pair, if both are black, they go to the dealer's pile; if both are red, they go to your pile; if one black and one red, they are discarded. The process is repeated until you two go through all 52 cards. If you have more cards in your pile, you win \$100; otherwise (including ties) you get nothing. The casino allows you to negotiate the price you want to pay for the game. How much would you be willing to pay to play this game?

Solution. The objective is to have more cards in our pile than the house, so we want some way to "burn" black cards. However, the only way that black cards (and symmetrically, red cards) can be used up without awarding points to the house is if they are discarded. Note, however, that this requires a red card burn too. Thus, there is *no* way to build an advantage. In fact, every game will always end in a tie (though the amount of discarded and held cards may differ game to game). The game is unwinnable, so no amount of money makes it worth playing.

3 Code

3.1 SQL Patterns (PostgreSQL)

Reference for recurring query shapes. Each entry states the problem phrasing that signals it and the mechanism that makes it work.

Reading a problem

Most prompts map to a small set of shapes, and the wording is the tell:

- “per group”, “for each” → **GROUP BY**, or a **PARTITION BY** window when the whole row must survive.
- “top- N per group”, “most recent per” → rank-then-filter, or **DISTINCT ON** when $N = 1$.
- “change from the previous”, “period-over-period”, “return” → **LAG** / **LEAD**.
- “running”, “cumulative”, “rolling”, “moving average” → an ordinary aggregate with **OVER**.
- “consecutive”, “streak”, “gap” → gaps and islands.
- “one column per category”, “pivot” → conditional aggregation.

The first decision on any prompt is whether the answer collapses rows (**GROUP BY**) or keeps every row and appends a computed column (a window function).

Logical processing order

A query is evaluated in this order, regardless of written order:

FROM/JOIN → **WHERE** → **GROUP BY** → **HAVING** → **SELECT** → **DISTINCT** → **ORDER BY** → **LIMIT**

Two consequences drive most of the patterns below. **WHERE** runs before aggregation, so it cannot see aggregates or **SELECT** aliases. Window functions are computed at the **SELECT** step, so they also cannot appear in **WHERE** and must be wrapped in a subquery or CTE to filter on.

3.1.1 Filtering and Joins

The foundation. Every later pattern assumes you can already restrict rows and combine tables.

WHERE vs HAVING. **WHERE** filters individual rows before grouping. **HAVING** filters whole groups after aggregation and is the only place an aggregate predicate is legal. A filter on a plain (non-aggregated) column belongs in **WHERE** so rows are dropped before the grouping work happens. Reserve **HAVING** for conditions like **COUNT(*) > 5**.

```
1 SELECT department, COUNT(*) AS headcount
2 FROM employees
3 WHERE active = true           -- row filter, before grouping
4 GROUP BY department
5 HAVING COUNT(*) > 5;         -- group filter, on an aggregate
```

Listing 1: **WHERE** prunes rows, **HAVING** prunes groups

Joins. The recognition cue is which side you want to keep when a match is missing.

- **INNER:** rows with a match on both sides.
- **LEFT / RIGHT:** keep all rows from one side, NULL-fill the other.
- **FULL OUTER:** keep unmatched rows from both sides.
- **CROSS:** Cartesian product, every left row against every right row.
- **SELF:** a table joined to itself under an alias (employee to manager, row to prior row).
- **Semi-join (EXISTS, IN):** keep left rows with at least one match. No row duplication, no right-side columns.
- **Anti-join (NOT EXISTS, or LEFT JOIN ... WHERE right.key IS NULL):** keep left rows with no match.

Semi- and anti-joins filter the left table on whether a match exists, without pulling right-side columns or duplicating rows. Prefer NOT EXISTS over NOT IN for the anti-join, since a NULL on the right poisons NOT IN to zero rows.

```
1  -- Semi-join: customers with at least one order
2  SELECT c.customer_id, c.name
3  FROM customers c
4  WHERE EXISTS (SELECT 1 FROM orders o
5                  WHERE o.customer_id = c.customer_id);
6
7  -- Anti-join: customers with no orders
8  SELECT c.customer_id, c.name
9  FROM customers c
10 WHERE NOT EXISTS (SELECT 1 FROM orders o
11                     WHERE o.customer_id = c.customer_id);
12
13 -- Anti-join, LEFT JOIN form: unmatched rows are the NULL-filled ones
14 SELECT c.customer_id, c.name
15 FROM customers c
16 LEFT JOIN orders o ON o.customer_id = c.customer_id
17 WHERE o.customer_id IS NULL;
```

Listing 2: Semi-join keeps matches, anti-join keeps non-matches

3.1.2 Aggregation

Conditional aggregation (pivot). The signal is a request for one column per category on a single row. Fold a categorical column into counts by wrapping a CASE in an aggregate. Rows the CASE does not match contribute 0 to SUM. In Postgres the FILTER clause does the same thing and reads more cleanly.

```
1  SELECT
2      inspection_type,
3      COUNT(*) FILTER (WHERE risk_category = 'Low Risk')      AS low_risk
4      ,
5      COUNT(*) FILTER (WHERE risk_category = 'High Risk')     AS
6      high_risk,
7      COUNT(*)                                                AS total
8  FROM sf_restaurant_health_violations
9  GROUP BY inspection_type
10 ORDER BY total DESC;
```

Listing 3: Pivot risk categories to columns

The `SUM(CASE WHEN ... THEN 1 ELSE 0 END)` form is equivalent and portable across engines. `COUNT` ignores `NULL`, so `COUNT(CASE WHEN cond THEN 1 END)` (no `ELSE`) also counts only the matches.

3.1.3 NULL Handling

`NULL` means unknown, so it propagates through arithmetic and poisons comparisons. The two functions below are the standard guards.

COALESCE and NULLIF. `COALESCE(a, b, c, ...)` returns the first non-`NULL` argument, evaluated left to right, or `NULL` if all are `NULL`. The common use is a fallback that keeps a `NULL` from poisoning arithmetic: `salary + commission` is `NULL` whenever `commission` is, so write `salary + COALESCE(commission, 0)`. It also supplies a default down a priority list, as in `COALESCE(nickname, first_name, 'Unknown')`. It is ANSI standard, unlike the vendor-specific `IFNULL` (MySQL) and `ISNULL` (SQL Server), and it accepts any number of arguments.

`NULLIF(a, b)` is the mirror image: it returns `NULL` when `a = b`, otherwise `a`. Its standard use guards division, `x / NULLIF(y, 0)` yields `NULL` instead of raising a divide-by-zero when `y` is 0.

```
1 SELECT
2     order_id,
3     revenue / NULLIF(quantity, 0) AS unit_price, -- NULL, not an
         error, when quantity = 0
4     COALESCE(discount, 0) AS discount -- NULL discount
         treated as 0
5 FROM orders;
```

Listing 4: `NULLIF` guards division, `COALESCE` supplies defaults

A subtlety with aggregates: `SUM`, `AVG`, and `COUNT(col)` skip `NULL`s. So `AVG(commission)` averages only the non-`NULL` rows, while `AVG(COALESCE(commission, 0))` counts the `NULL`s as zeros and enlarges the denominator.

Gotchas.

- **NOT IN with NULLs:** if the subquery returns any `NULL`, `x NOT IN (...)` is never true and the query returns zero rows. `x NOT IN (1, NULL)` expands to `NOT (x=1 OR x=NULL)`, and the `NULL` comparison poisons the result to unknown. Use `NOT EXISTS`, which is null-safe.
- **Integer division truncates:** `5 / 2` yields 2 because the fractional part is discarded. Cast one operand: `5::numeric / 2` or `5.0 / 2`. Bites hardest in ratio and percentage columns.
- **COUNT(col) vs COUNT(*):** the former skips `NULL`s in `col`, the latter counts every row. A common source of off-by-*N* errors.

3.1.4 Window Functions

The signal for a window function is a per-row result that still depends on other rows, such as a rank, a neighbor value, or a running total. Where `GROUP BY` would collapse the group to one row, a window function keeps every row and appends the computed column.

Every window function shares the same shape: `f() OVER (PARTITION BY ... ORDER BY ...)`. `PARTITION BY` splits the rows into independent groups, `ORDER BY` sets the order within

each group, and the function is evaluated per row. All of them are computed at the `SELECT` step, so to filter on their output you wrap them in a subquery or CTE.

Rank-then-filter with `ROW_NUMBER()`. Use this for top- N per group when you need to keep the full row, since `GROUP BY` would collapse the other columns away. Number rows inside each partition, then filter on that number in an outer query.

Choose the ranker by tie policy: `ROW_NUMBER` assigns distinct numbers and breaks ties arbitrarily, `RANK` repeats a rank and leaves gaps, `DENSE_RANK` repeats with no gaps.

```
1 SELECT department, name, salary
2 FROM (
3     SELECT department, name, salary,
4            ROW_NUMBER() OVER (PARTITION BY department
5                               ORDER BY salary DESC) AS rn
6     FROM employees
7 ) ranked
8 WHERE rn = 1;
```

Listing 5: Highest-paid employee per department

Offset access with `LAG()` and `LEAD()`. The signal is a comparison against the previous or next row: a delta, a period-over-period change, or a return. These read a value from another row relative to the current one, without a self-join. `LAG(col, k, default)` returns the value k rows earlier in the ordered partition, `LEAD` looks k rows ahead. Both default to $k = 1$ and return `NULL` past the partition edge unless a fill default is supplied. The `ORDER BY` inside `OVER` defines what “previous” means and is mandatory. `PARTITION BY` resets the sequence at each group boundary, so a lag never bleeds from one partition into the next.

```
1 SELECT ticker, trade_date, close,
2        close / LAG(close) OVER (PARTITION BY ticker
3                                 ORDER BY trade_date) - 1 AS
4                                daily_return
5 FROM prices;
```

Listing 6: Daily returns per ticker with `LAG`

The first row of each partition has no predecessor, so its `LAG` and the derived return are `NULL`, which is correct since there is no prior price to compare against. `LEAD` is the same tool aimed forward, used below for gap detection.

Running and rolling aggregates. The signal is “running”, “cumulative”, or “moving average”. An ordinary aggregate gains a window when you attach `OVER (... ORDER BY ...)`. With an `ORDER BY` and no explicit frame, the frame defaults to everything from the partition start up to the current row, giving a running total. Add an explicit `ROWS BETWEEN` frame to get a fixed moving window instead.

```
1 SELECT ticker, trade_date, close,
2        AVG(close) OVER (PARTITION BY ticker ORDER BY trade_date
3                          ROWS BETWEEN 19 PRECEDING AND CURRENT ROW) AS
4                                ma_20,
5        SUM(volume) OVER (PARTITION BY ticker ORDER BY trade_date) AS
6                                cum_volume
7 FROM prices;
```

Listing 7: 20-day moving average and running volume

`ma_20` is the current row plus the 19 before it, and `cum_volume` uses the default frame for a running total. These are the SQL forms of a pandas `.rolling(20).mean()` and `.expanding().sum()`.

One subtlety: the default frame is `RANGE UNBOUNDED PRECEDING`, which includes every row tied on the `ORDER BY` key, not just rows physically up to the current one. When ties on the order key exist and you want a row-precise window, state `ROWS BETWEEN` explicitly.

Gaps and islands. The signal is any mention of consecutive values, streaks, or runs. An *island* is a maximal run of consecutive values. The trick: subtract a dense row number from the value. Across a consecutive run both sides increase by one per step, so the difference stays constant, and it jumps exactly at a gap. Group by the difference to collapse each run.

```
1 WITH numbered AS (  
2     -- DISTINCT first so duplicate ids do not desync the row number  
3     SELECT DISTINCT log_id,  
4         ROW_NUMBER() OVER (ORDER BY log_id) AS rn  
5     FROM logs  
6 )  
7 SELECT MIN(log_id) AS island_start,  
8        MAX(log_id) AS island_end,  
9        COUNT(*)    AS length  
10 FROM numbered  
11 GROUP BY log_id - rn  
12 ORDER BY island_start;
```

Listing 8: Islands of consecutive ids

The `DISTINCT` is load-bearing. If `log_id` repeats, `rn` still advances on the duplicate while the value does not, so `log_id - rn` shifts and splits an island that should stay whole. For consecutive *dates* rather than integers, offset by an interval: `log_date - rn * INTERVAL '1 day'` is constant within a run.

```
1 SELECT log_id + 1 AS missing_from,  
2        next_id - 1 AS missing_to  
3 FROM (  
4     SELECT log_id,  
5         LEAD(log_id) OVER (ORDER BY log_id) AS next_id  
6     FROM logs  
7 ) t  
8 WHERE next_id - log_id > 1;    -- values missing between two present ids
```

Listing 9: Gaps via LEAD

3.1.5 Deduplication

DISTINCT. `DISTINCT` compares the entire select list as a tuple. `SELECT DISTINCT a, b` returns every distinct *pair*, so a value in `a` can still repeat as long as the paired `b` differs. It is a whole-row operator with no way to say “dedupe on this column and keep the rest of the row.” It runs after `SELECT` and before `ORDER BY`, which is why every `ORDER BY` expression must also appear in the select list, and it treats two `NULL`s as duplicates. `COUNT(DISTINCT col)` is a separate mechanism that dedupes the input to one aggregate and has no effect on the rows returned.

DISTINCT ON. A PostgreSQL extension that keeps the first row of each group sharing the same value of the parenthesized expressions, returning all selected columns of that surviving row. “First” is defined by `ORDER BY`, whose leading terms must match the `DISTINCT ON` expressions. Omit the `ORDER BY` and the surviving row is arbitrary.

```
1 SELECT DISTINCT ON (ticker)
2     ticker, trade_date, close
3 FROM prices
4 ORDER BY ticker, trade_date DESC;    -- leading term must be ticker
```

Listing 10: Most recent close per ticker

This is the compact form of the rank-then-filter pattern from the window section. The `ROW_NUMBER` equivalent needs a subquery and an outer `WHERE rn = 1`. Which to reach for:

- **Top-1 per group in Postgres:** `DISTINCT ON` is shorter and generally at least as fast, since it stops at the first row of each group.
- **Top-*N* for *N* > 1, or portability:** `ROW_NUMBER` extends to `rn <= 3` and runs on any engine. `DISTINCT ON` is Postgres-only.
- **Ties:** `DISTINCT ON` always emits exactly one row per group, settled by the remaining `ORDER BY` terms or arbitrarily. To keep every tied row, use `RANK() = 1`.

Because the `ORDER BY` is dictated by the deduplication key, presenting the result in a different order means wrapping the query in an outer `SELECT ... ORDER BY`.

3.1.6 String Manipulation

Positions are 1-based, and every function returns a new value rather than mutating in place. The recurring uses are normalizing a dirty join key, parsing a delimited field, and folding a group of strings into one.

Concatenation. The operator `a || b` propagates `NULL`: if either side is `NULL` the whole expression is `NULL`. `CONCAT(a, b, ...)` treats `NULL` as an empty string. `CONCAT_WS(sep, ...)` takes the separator first, joins the rest with it, and skips `NULL` arguments without leaving a doubled separator. Choosing `||` for user-facing text is a common way to silently blank out a whole column.

Case and whitespace. `UPPER` and `LOWER` are the standard folds, `INITCAP` capitalizes the first letter of each word and lowercases the rest. `TRIM` strips characters from the ends only, spaces by default, with `LEADING`, `TRAILING`, or `BOTH` to pick a side. Interior whitespace survives, so collapsing double spaces needs `REGEXP_REPLACE(s, '\s+', ' ', 'g')`. `LPAD(s, n, fill)` and `RPAD` pad to a fixed width and truncate when the input is already longer than `n`.

```
1 SELECT
2     LOWER(TRIM(email))                AS email_key,
3     SPLIT_PART(LOWER(TRIM(email)), '@', 2) AS domain,
4     LPAD(account_id::text, 8, '0')      AS padded_id,
5     CONCAT_WS(' ', last_name, first_name) AS sort_name
6 FROM users;
```

Listing 11: Normalizing a join key

Extraction and search. SUBSTRING(*s* FROM *a* FOR *b*) (or SUBSTR(*s*, *a*, *b*)) takes *b* characters starting at position *a*. LEFT(*s*, *n*) and RIGHT(*s*, *n*) take from the ends, and a negative *n* drops that many characters from the opposite end. STRPOS(*s*, *sub*) returns the 1-based index of the first occurrence and 0 when the substring is absent, so a found/not-found test compares against 0. SPLIT_PART(*s*, *delim*, *n*) pulls the *n*th field of a delimited string and returns an empty string when that field does not exist, worth wrapping in NULLIF(..., "").

```

1 SELECT full_name,
2        SPLIT_PART(full_name, ' ', 1)           AS first_name,
3        NULLIF(SPLIT_PART(full_name, ' ', 2), '') AS last_name,
4        LENGTH(full_name)
5        - LENGTH(REPLACE(full_name, ' ', ''))    AS space_count
6 FROM people;
```

Listing 12: Parsing a delimited name field

The last expression is the occurrence-counting idiom: remove every copy of the target and measure how much the string shrank. For a needle longer than one character, divide the difference by LENGTH(needle).

Replacement. REPLACE(*s*, *from*, *to*) substitutes every literal occurrence. TRANSLATE(*s*, *from*, *to*) maps character to character by position and deletes any character in *from* that has no counterpart in *to*, a fast way to strip a known character set. REGEXP_REPLACE(*s*, *pattern*, *repl*, *flags*) handles the general case and replaces only the *first* match unless the 'g' flag is passed, a frequent source of half-cleaned output.

Pattern matching. LIKE uses % for any run of characters and _ for exactly one, matches case-sensitively, and takes an ESCAPE clause to match a literal % or _. ILIKE is the case-insensitive form. SIMILAR TO combines the two conventions: it reads % and _ as wildcards and also accepts the regex metacharacters |, *, +, and grouping, anchoring the pattern to the whole string. The POSIX operators cover the general case. ~ matches, ~* matches case-insensitively, and !~ and !~* are their negations. Each returns a boolean. A POSIX pattern is unanchored and matches anywhere in the string unless ^ or \$ fixes it to an end.

<i>Construct</i>	<i>Meaning</i>
^ \$	Start and end of the string.
.	Any single character.
[abc] [^abc] [a-z]	A character in the set, outside the set, or in the range.
* + ?	The preceding item zero or more, one or more, or zero or one times.
{n} {n,} {n,m}	Exactly n, at least n, or between n and m repetitions.
	Alternation between the branches on either side.
() (?:)	A capturing group, or a non-capturing group.
\d \s \w	A digit, a whitespace character, or a word character. The uppercase form negates each.
\y	A word boundary, the Postgres spelling.

POSIX regex constructs used by ~ and the REGEXP_ functions. Quantifiers are greedy unless followed by ?.

Match tests as indicator flags. A POSIX match returns a boolean, so casting it with `::int` produces a 0/1 column per pattern. `^p` matches a string that begins with `p`, `p$` one that ends with `p`, and an unanchored `p` one that contains `p` at any position. Alternation inside a group, `(a|b|c)`, matches any branch, and anchoring the group applies the test at that position.

```

1 SELECT sample_id,
2        (dna_sequence ~ '^ATG')::int           AS has_start,    -- begins
          with ATG
3        (dna_sequence ~ '(TAA|TAG|TGA)$')::int AS has_stop,    -- ends
          with a stop codon
4        (dna_sequence ~ 'ATAT')::int          AS has_atat,    -- occurs
          anywhere
5        (dna_sequence ~ 'GGG')::int           AS has_ggg
6 FROM samples
7 ORDER BY sample_id;
```

Listing 13: Presence flags from anchored and unanchored patterns

Extracting with regex. `SUBSTRING(s FROM pattern)` returns the first substring the pattern matches, or the first capture group when the pattern contains one, and `NULL` on no match. `REGEXP_SUBSTR(s, pattern, ...)` returns a single match as a scalar. `REGEXP_MATCHES(s, pattern, flags)` returns the capture groups as an array, and under the 'g' flag returns one row per match, which places it in the `FROM` clause. `REGEXP_COUNT(s, pattern)` returns the number of matches.

```

1 SELECT email,
2        SUBSTRING(email FROM '@(.+)$')         AS domain,      -- the
          capture group
3        REGEXP_SUBSTR(ip_port, '[0-9.]+') AS ip,
4        REGEXP_COUNT(csv, ',') + 1            AS field_count
5 FROM records;
```

Listing 14: Pulling fields out with a pattern

The presence of a capture group determines what `SUBSTRING` returns: with a group it returns the group, and without one it returns the whole match. `'@(.+)$'` therefore returns the domain without the `@`.

Greediness and flags. Quantifiers are greedy and match the longest possible span. A trailing `?` makes a quantifier lazy, so `.*?` matches as few characters as possible. An operation acts on the first match alone unless the 'g' flag is passed, the default that leaves `REGEXP_REPLACE` applied to a single occurrence without it. Word boundaries are written `\b` in Postgres, since `\b` denotes a backspace inside a bracket expression.

Splitting and aggregating. `STRING_AGG(expr, delim ORDER BY ...)` collapses a group into one delimited string and is the string analogue of `SUM`. The `ORDER BY` lives inside the call, so the output order is deterministic. The inverse direction expands one row into many: `UNNEST(STRING_TO_ARRAY(s, delim))` or `REGEXP_SPLIT_TO_TABLE(s, pattern)` in the `FROM` clause.

```

1 SELECT department,
2     STRING_AGG(name, ', ' ORDER BY name) AS members
3 FROM employees
4 GROUP BY department;
5 -- one row per tag stored in a comma-separated column
6 SELECT p.id, TRIM(tag) AS tag
7 FROM products p,
8     UNNEST(STRING_TO_ARRAY(p.tags, ',')) AS tag;

```

Listing 15: Collapse a group, then expand it back

3.1.7 Command Reference

Signatures are PostgreSQL. Optional arguments are shown in brackets.

<i>Command</i>	<i>Arguments</i>	<i>What it does</i>
WHERE	boolean predicate	Filters individual rows before grouping. Cannot see aggregates, window functions, or select aliases.
GROUP BY	column or expression list	Collapses rows to one output row per distinct key combination.
HAVING	boolean predicate	Filters whole groups after aggregation. The only legal home for an aggregate predicate.
DISTINCT	applies to the select list	Removes duplicate rows compared across the entire select list. Treats two NULLs as equal.
DISTINCT ON	(expr, ...)	Postgres. Keeps one row per distinct value of the expressions, chosen by the leading <code>ORDER BY</code> terms, which must match.
ORDER BY	expr [ASC DESC] [NULLS FIRST LAST]	Sorts the output. Postgres places NULLs last under <code>ASC</code> and first under <code>DESC</code> .
LIMIT / OFFSET	n	Caps the number of rows returned, or skips the first n.
WITH	name AS (query)	Common table expression. Names a subquery for reuse. <code>WITH RECURSIVE</code> walks hierarchies.
CASE	WHEN cond THEN val [ELSE val] END	Row-level branching. Returns the first match, or NULL when nothing matches and <code>ELSE</code> is absent.
FILTER	(WHERE cond) after an aggregate	Restricts which rows the aggregate consumes. Postgres shorthand for <code>SUM(CASE ...)</code> .
EXISTS / NOT EXISTS	subquery	Semi-join and anti-join. Null-safe, unlike <code>IN</code> and <code>NOT IN</code> .

Query clauses and structure

<i>Command</i>	<i>Arguments</i>	<i>What it does</i>
COUNT(*)	none	Counts every row, including all-NULL ones.
COUNT(col)	one column	Counts rows where <code>col</code> is non-NULL.
COUNT(DISTINCT col)	one column	Counts distinct non-NULL values.
SUM / AVG / MIN / MAX	one expression	Skip NULLs. AVG divides by the non-NULL count.
STRING_AGG	(<code>expr</code> , <code>delim</code> [<code>ORDER BY ...</code>])	Concatenates a group into one delimited string.
ARRAY_AGG	(<code>expr</code> [<code>ORDER BY ...</code>])	Collects group values into an array.
COALESCE	any number of values	Returns the first non-NULL argument, left to right.
NULLIF	two values	Returns NULL when the arguments are equal, otherwise the first. Guards division by zero.
GREATEST / LEAST	list of values	Largest or smallest value across columns within one row. Ignores NULLs.
IS [NOT] DISTINCT FROM	two values	Null-safe comparison that treats two NULLs as equal.
IS [NOT] NULL	one value	The only valid null test. <code>= NULL</code> is always unknown.

Aggregates and NULL handling

<i>Command</i>	<i>Arguments</i>	<i>What it does</i>
OVER	(<code>PARTITION BY ...</code> <code>ORDER BY ...</code> [<code>frame</code>])	Turns a function into a window function evaluated per row without collapsing the result set.
ROW_NUMBER()	none	Sequential number within the partition. Ties broken arbitrarily.
RANK()	none	Competition rank. Ties share a rank and leave gaps afterward.
DENSE_RANK()	none	Ties share a rank with no gaps.
NTILE(n)	bucket count	Splits the partition into <code>n</code> roughly equal buckets and labels each row.
LAG(col, k, default)	<code>k</code> defaults to 1	Value <code>k</code> rows earlier in the ordered partition. NULL past the partition edge unless a default is supplied.
LEAD(col, k, default)	same as LAG	Value <code>k</code> rows ahead.
FIRST_VALUE / LAST_VALUE	<code>col</code>	First or last value in the current frame. LAST_VALUE needs an explicit frame to see the full partition.
NTH_VALUE(col, n)	column, position	The <code>n</code> th value in the frame.
SUM / AVG / COUNT ... OVER	aggregate plus window spec	Running or rolling aggregate computed per row.
ROWS BETWEEN	<code>a PRECEDING</code> to <code>b FOLLOWING</code> , <code>CURRENT ROW</code> , <code>UNBOUNDED</code>	Physical frame counted in rows. Use it for fixed moving windows.
RANGE BETWEEN	same bound keywords	Value-based frame that includes every row tied on the <code>ORDER BY</code> key. This is the default frame.

Window functions and frames

<i>Command</i>	<i>Arguments</i>	<i>What it does</i>
 	two strings	Concatenation. Yields NULL if either side is NULL.
CONCAT	any number of values	Concatenation treating NULL as an empty string.
CONCAT_WS	separator, then values	Joins arguments with a separator and skips NULLs.
LENGTH / CHAR_LENGTH	one string	Character count. OCTET_LENGTH gives bytes.
UPPER / LOWER /INITCAP	one string	Case folding. INITCAP capitalizes each word and lowercases the rest.
TRIM	[LEADING TRAILING BOTH] [chars] FROM s	Strips characters from the ends only, spaces by default. LTRIM , RTRIM , BTRIM are shorthands.
LPAD / RPAD	(s, n [, fill])	Pads to width n, truncating when s is already longer.
LEFT / RIGHT	(s, n)	First or last n characters. A negative n drops that many from the opposite end.
SUBSTRING	(s FROM a FOR b) or SUBSTR(s, a, b)	Slice of length b starting at 1-based position a.
STRPOS / POSITION	(s, sub) / (sub IN s)	1-based index of the first occurrence, 0 when absent.
REPLACE	(s, from, to)	Replaces every literal occurrence.
TRANSLATE	(s, from_set, to_set)	Maps characters by position and deletes those with no counterpart.
SPLIT_PART	(s, delim, n)	The nth delimited field. Returns an empty string when the field is missing.
STRING_TO_ ARRAY / UNNEST	(s, delim) / (array)	Splits into an array, then expands an array into rows.
REVERSE / REPEAT	(s) / (s, n)	Reverses a string, or repeats it n times.
LIKE / ILIKE	pattern with % and _	Wildcard match, case sensitive and case insensitive respectively.
~ ~* !~	POSIX regex	Regex match, case-insensitive match, and negated match.
TO_CHAR	(value, format)	Formats a number or timestamp as text, e.g. 'YYYY-MM-DD'.
CAST / ::	(expr AS type) / expr::type	Type conversion. ::numeric also fixes integer division.

String functions

3.2 Python Patterns

Good solutions to interview problems are often determined by the choice of container and knowledge of which built-in primitives express the required operation.

Containers and cost profiles

Selecting a container is selecting a cost profile. That decision precedes the algorithm and frequently dominates the runtime.

	Ordered	Mutable	Lookup	Use
<code>list</code>	yes	yes	$O(n)$	ordered sequence, duplicates permitted
<code>tuple</code>	yes	no	$O(n)$	fixed record, or a dictionary key
<code>set</code>	no	yes	$O(1)$	membership, deduplication
<code>dict</code>	insertion	yes	$O(1)$	mapping from key to value

Membership cost

`x in some_list` is $O(n)$. `x in some_set` is $O(1)$ on average. A membership test inside a loop over n elements is therefore $O(n^2)$ against a list and $O(n)$ against a set. Converting the list to a set once, before the loop, removes a factor of n . This substitution accounts for a large share of naive-to-optimal improvements.

Hashability. Set elements and dictionary keys must be hashable, which in practice requires immutability. A `tuple` is a valid key. A `list` is not, since mutation would invalidate its position in the hash table. Converting a list to a tuple to form a composite key is therefore a standard construction, as in grouping anagrams by their sorted letters.

```
1 seen = set()
2 seen.add((row, col))           # legal, tuple is hashable
3 # seen.add([row, col])         # TypeError: unhashable type
```

Listing 16: Tuple as a composite key

Dictionary access. Counting, grouping, and seen-before problems reduce to a dictionary. Three access forms differ in their treatment of a missing key.

```
1 d[k]                           # raises KeyError if k is absent
2 d.get(k, default)              # returns default instead of raising
3 d.setdefault(k, [])            # inserts default if absent, then returns it
4
5 for k in d:                    # iterates over keys
6 for k, v in d.items():          # iterates over pairs
```

Listing 17: Dictionary access

Built-in functions

An accumulator loop is often a built-in written out longhand. The following cover most cases.

Built-in	Expresses
<code>len, sum, min, max</code>	size and aggregation
<code>sorted</code>	ordering, parameterized by key and reverse
<code>any(...)</code>	existential quantifier: at least one element satisfies p
<code>all(...)</code>	universal quantifier: every element satisfies p
<code>enumerate</code>	index paired with value
<code>zip</code>	parallel iteration over several sequences
<code>reversed, range</code>	traversal order
<code>abs, round, divmod</code>	numeric utilities

`any` and `all` short-circuit. The expression `any(x < 0 for x in xs)` terminates at the first negative value.

The key argument. `sorted`, `min`, and `max` accept a key function applied to each element. Ranking is performed on the image of that function while the original element is returned. The consequence is that a record can be ranked on one of its fields and recovered whole.

```
1 min(scores)                                # returns a number
2 min(records, key=lambda r: r[1])           # returns the full record
3
4 sorted(words, key=len)                     # rank by length
5 sorted(records, key=lambda r: (-r[1], r[0])) # score desc, name asc
```

Listing 18: Ranking on a field, returning the record

A tuple key is the analogue of a multi-column `ORDER BY`. Negating a single numeric field reverses that field alone. `reverse=True` cannot express this, since it inverts every field simultaneously.

Comprehensions

A comprehension expresses selection and projection in one expression. The form `[f(x) for x in xs if p(x)]` corresponds to selecting $f(x)$ over the elements for which $p(x)$ holds. Four bracket types produce four containers.

```
1 [x*x for x in xs if x > 0]                # list
2 {x*x for x in xs}                         # set, deduplicates
3 {k: v for k, v in pairs}                  # dict
4 (x*x for x in xs)                         # generator, lazy, builds no intermediate list
```

Listing 19: The four forms

The generator form is appropriate when the result is consumed once, for example inside `sum(...)` or `max(...)`, since it avoids materializing a list. Nesting beyond two levels is generally less readable than an explicit loop.

Sorting

- `sorted(xs)` returns a new list. `xs.sort()` sorts in place and returns `None`. The assignment `xs = xs.sort()` binds `None` and fails silently.
- Python's sort is **stable**. Elements comparing equal retain their original relative order. Stability licenses sorting in multiple passes: sorting by the secondary field first and by the primary field second preserves the secondary ordering within each group.
- Sorting a sequence of lists or tuples compares elementwise from the left. The result depends on field order, so an explicit **key** should be supplied whenever the intent is nonobvious.
- Sorting costs $O(n \log n)$, which is negligible at the input sizes these problems use.

The collections module

Four types cover nearly every use. Three are specializations of the dictionary that eliminate the same recurring boilerplate: the check for whether a key already exists before writing to it. The fourth replaces the list when insertion or removal happens at the front.

Counter: frequency. A `Counter` is a dictionary from element to count. It is constructed in one pass over any iterable and answers the question “how many times does each value occur.” Frequency is the most common intermediate object in string and array problems, and constructing one by hand requires an existence check on every element.

```
1 from collections import Counter
2
3 c = Counter("mississippi") # Counter({'i': 4, 's': 4, 'p': 2, 'm': 1})
4 c['s']                    # 4
5 c['z']                    # 0, a missing key reads as zero and does NOT raise
6
7 c.most_common(2)          # [('i', 4), ('s', 4)], top 2 by count
8 c.most_common()           # every pair, ordered by count descending
9 c.total()                 # sum of all counts
```

Listing 20: Counter

A missing key evaluates to 0 rather than raising `KeyError`, which is what permits `c[x] += 1` to be written unconditionally. The read does not insert the key, so the counter does not grow through inspection.

`most_common(k)` selects the top k with a heap in $O(n \log k)$. Calling `most_common()` with no argument sorts the whole counter at $O(n \log n)$, so the argument should be supplied whenever k is known.

Counters support multiset arithmetic, which expresses several problems directly.

```
1 a, b = Counter("aabbcc"), Counter("abbbd")
2
3 a + b          # counts added
4 a - b          # counts subtracted, results <= 0 are dropped
5 a & b          # intersection: elementwise minimum
6 a | b          # union: elementwise maximum
```

Listing 21: Counter arithmetic

Two canonical Counter identities

Anagram test. Two strings are anagrams if and only if their counters are equal, so `Counter(s) == Counter(t)` settles the question in $O(n)$. Sorting both strings answers the same question in $O(n \log n)$.

Sliding-window matching. To locate every permutation of a pattern p inside a text t , maintain a counter over a window of length $|p|$ and compare it against `Counter(p)` after each shift. Incrementing the entering character and decrementing the leaving one updates the window in $O(1)$.

defaultdict: grouping. A `defaultdict` is constructed with a zero-argument factory. Accessing a missing key calls that factory, inserts the result, and returns it. The effect is to remove the initialization branch from every accumulation loop.

```
1 from collections import defaultdict
2
3 groups = defaultdict(list)           # missing key -> []
4 for name, dept in employees:
5     groups[dept].append(name)        # no existence check required
6
7 counts = defaultdict(int)           # missing key -> 0
8 adj     = defaultdict(set)          # missing key -> set(), an
    adjacency list
9 grid    = defaultdict(lambda: defaultdict(int)) # a nested, sparse 2D
    table
```

Listing 22: defaultdict

The factory is any callable taking no arguments. `int()` returns 0, `list()` returns [], and `set()` returns an empty set, which is why the bare type name is passed rather than a call to it. A nested default requires a `lambda`, since `defaultdict(defaultdict(int))` would pass an instance where a factory is expected.

`defaultdict(list)` performs a GROUP BY that omits the aggregation step. Every member of each group survives, which is the property that `ROW_NUMBER() OVER (PARTITION BY ...)` supplies in SQL. Building a graph's adjacency list is the same operation applied to edges.

Reading a defaultdict mutates it

Any access to a missing key inserts it. The expression `if d[k]:` therefore creates k with an empty default and the dictionary grows through inspection alone. This corrupts a subsequent iteration over `d.items()` and inflates `len(d)`.

For a read that must not write, use `k in d` or `d.get(k)`. Neither triggers the factory.

The alternative to a `defaultdict` is `d.setdefault(k, []).append(x)` on an ordinary dictionary. It produces the same result and constructs a fresh empty list on every call, including the calls where the key is already present. The `defaultdict` factory runs only on a miss.

deque: a queue. A `deque` is a double-ended queue backed by a doubly linked list of fixed-size blocks. Appending and popping at either end is $O(1)$. The relevant comparison is against a `list`, which stores its elements contiguously, so removing the front element requires shifting every remaining element left by one position.

Operation	list	deque
Append or pop at the right	$O(1)$	$O(1)$
Append or pop at the left	$O(n)$	$O(1)$
Index at position i	$O(1)$	$O(n)$

The costs are complementary. A **list** is the correct stack, since a stack only touches the right end. A **deque** is the correct queue. Random access into a **deque** is a sign that the wrong container was chosen.

```

1 from collections import deque
2
3 q = deque([1, 2, 3])
4 q.append(4)           # right
5 q.appendleft(0)       # left
6 q.pop()              # from the right
7 q.popleft()          # from the left,  $O(1)$  where  $list.pop(0)$  is  $O(n)$ 
8 q.rotate(1)          # shift every element right by one, cyclically

```

Listing 23: deque

Breadth-first search is the standard application. The frontier is processed in insertion order, which requires removal from the front on every iteration of the main loop.

```

1 q = deque([start])
2 seen = {start}
3 while q:
4     node = q.popleft()      # FIFO, so nodes are visited by distance
5     for nxt in adj[node]:
6         if nxt not in seen:
7             seen.add(nxt)
8             q.append(nxt)

```

Listing 24: BFS frontier

Bounded length. A deque constructed with `maxlen=k` discards an element from the opposite end whenever a full deque is appended to. This maintains a rolling window over the last k items with no explicit eviction logic, which is useful for moving averages and for any statistic over a fixed lookback.

```

1 w = deque(maxlen=3)
2 for x in stream:
3     w.append(x)           # the oldest element is evicted automatically
4     if len(w) == 3:
5         print(sum(w) / 3)

```

Listing 25: A rolling window of the last 3 values

namedtuple: a labeled record. A tuple carries no information about what its positions mean, so `r[1]` requires the reader to remember the field order. A **namedtuple** attaches names while remaining a tuple, so it stays immutable, hashable, unpackable, and usable as a dictionary key or a set element.

```

1 from collections import namedtuple
2
3 Student = namedtuple("Student", ["name", "score"])
4 s = Student("Ada", 92.0)
5
6 s.score          # 92.0, and s[1] still works
7 name, score = s  # unpacking is unchanged
8 min(students, key=lambda s: s.score) # the key function reads clearly

```

Listing 26: namedtuple

The cost over a plain tuple is negligible and the gain is that a key function or a sort becomes self-documenting. When mutability is required, `dataclasses.dataclass` provides the equivalent with assignable fields.

Iteration

The following forms eliminate manual index arithmetic, which is the usual source of off-by-one errors.

```

1 for i, x in enumerate(xs):          # index and value together
2 for a, b in zip(xs, ys):            # parallel, halts at the shorter
    sequence
3 for a, b in zip(xs, xs[1:]):        # consecutive pairs
4 for x in reversed(xs):              # reverse traversal without index
    arithmetic
5
6 name, score = record                # tuple unpacking
7 first, *rest = xs                   # starred unpacking

```

Listing 27: Iteration forms

The construction `zip(xs, xs[1:])` yields every adjacent pair and underlies run detection, difference sequences, and monotonicity checks. It is the analogue of `LAG` in SQL.

Problem structure

The four-move skeleton

A typical warm-up problem decomposes into four steps.

1. **Parse** the input into a list of records.
2. **Reduce** the records to a target key: a minimum, a maximum, a count, or the k -th distinct value.
3. **Filter** the records against that key.
4. **Order and format** the output.

Step 2 accounts for most errors. The characteristic failure is fusing steps 2 and 3 into a single pass that mishandles an edge case.

Ranking and ties. The most frequent error in step 2 concerns ties. Whether tied elements share a rank or consume one is the same decision in Python as in SQL.

Tie policy	Python	SQL
Ties consume a rank	<code>sorted(xs)[k-1]</code>	<code>ROW_NUMBER() = k</code>
Ties share a rank	<code>sorted(set(xs))[k-1]</code>	<code>DENSE_RANK() = k</code>

For scores [20, 20, 30], the second lowest is 30. Both elements equal to 20 hold rank 1 and neither occupies rank 2, so `sorted(xs)[1]` evaluates to 20 and is incorrect. The `set` converts ROW_NUMBER semantics into DENSE_RANK semantics.

Rank then filter. Once the key is identified, it is computed in one pass and the records are filtered against it in a second. The structure mirrors the necessity of wrapping a window function in a CTE, since a window function cannot appear in `WHERE`.

```
1 target = sorted({score for _, score in records})[1]    # rank
2 winners = sorted(name for name, score in records if score == target)
3                                     # filter, then order
```

Listing 28: Rank then filter

Common errors

- `sorted(set(xs))[1]` raises `IndexError` when all values are identical.
- Slicing copies. `xs[a:b]` costs $O(b - a)$, so a slice inside a loop makes the loop quadratic.
- `list.insert(0, x)` and `list.pop(0)` are $O(n)$, since every subsequent element shifts. A `deque` provides $O(1)$ operations at both ends.
- Integer division `//` floors toward negative infinity, so `-7 // 2` evaluates to `-4`. `int(-7 / 2)` truncates toward zero.
- A mutable default argument (`def f(xs=[])`) is created once at definition time and persists across calls. The default should be `None`, with the container constructed inside the func.
- Strings are immutable. Accumulating with `s += c` in a loop is $O(n^2)$. Collect the pieces in a list and apply `".join(parts)`.

3.3 Algorithmic Paradigms

Each paradigm is identified by a recognition cue in the problem statement.

- **Hashing.** Cue: find a pair, count occurrences, seen before, contains a duplicate. Space is traded for $O(1)$ lookup. The canonical construction tests whether the complement of the current element already appears in a map of previously visited elements.
- **Two pointers.** Cue: a sorted sequence, or a pairing problem. Pointers converging from both ends solve “two elements summing to t ” in $O(n)$ with constant extra space. Fast and slow pointers detect a cycle in a linked list.
- **Sliding window.** Cue: a contiguous subarray or substring subject to a constraint. The right edge expands to admit elements and the left edge contracts while the constraint is violated. Each element enters and leaves once, giving $O(n)$.
- **Prefix sums.** Cue: repeated range-sum queries over a static array. Cumulative sums are precomputed once, after which any range query is a difference of two entries. This is the array analogue of `SUM()` `OVER`.
- **Binary search on the answer.** Cue: the smallest x such that a condition is feasible. Requires a monotone predicate, meaning that feasibility at x implies feasibility at every larger x . The search ranges over the answer space rather than the input.
- **Sorting as preprocessing.** Cue: intervals, scheduling, or grouping equal elements. Sorting converts a global problem into a local one in which each element need only be compared against its neighbor.

Complexity anchors

Operation	Cost
<code>x in set, x in dict</code>	$O(1)$ average
<code>x in list</code>	$O(n)$
<code>list.append</code>	$O(1)$ amortized
<code>list.insert(0,x), list.pop(0)</code>	$O(n)$
<code>deque.appendleft, deque.popleft</code>	$O(1)$
<code>sorted(xs)</code>	$O(n \log n)$
<code>xs[a:b]</code>	$O(b - a)$, copies

3.4 Recursion and Memoization

A recursive function is defined by three obligations.

The recursion contract

1. **Base case.** At least one input for which the answer is returned without a recursive call.
2. **Recursive case.** The answer for input n expressed in terms of the answers for strictly smaller inputs.
3. **Progress.** Every recursive call moves the argument closer to a base case, so the recursion terminates.

Correctness follows by induction. Assuming the recursive call returns the correct answer on the smaller input, verify only that the combination step is correct. Tracing the full call tree by hand is unnecessary.

The call stack. Each pending call occupies a stack frame, so the space cost of a recursion is $O(d)$ where d is the maximum depth. Python imposes a default recursion limit of 1000 frames and raises `RecursionError` beyond it. A recursion whose depth scales with n is therefore unsafe for large n unless it is rewritten as a loop with an explicit stack.

Overlapping subproblems. Naive recursion recomputes identical subproblems. The Fibonacci recurrence $F(n) = F(n-1) + F(n-2)$ generates a call tree with $O(\varphi^n)$ nodes, since $F(n-2)$ is evaluated once by the $F(n-1)$ branch and again directly. Caching each result by its argument collapses the tree to $O(n)$ distinct evaluations.

```
1 from functools import lru_cache
2
3 @cache(maxsize=None)
4 def fib(n):
5     if n < 2:                                # base case
6         return n
7     return fib(n-1) + fib(n-2)               # recursive case
```

Listing 29: Memoization by decorator

Arguments to a memoized function must be hashable, which is the reason memoized state is passed as an integer, a string, or a tuple rather than as a list or a set.

Top-down and bottom-up. Memoized recursion is the top-down form. It computes only the states that are reachable, at the cost of a call stack. Tabulation is the bottom-up form. It fills an array in an order that guarantees every dependency is already resolved, using no stack and no recursion.

```
1 def fib(n):
2     if n < 2:
3         return n
4     prev, curr = 0, 1
5     for _ in range(n - 1):
6         prev, curr = curr, prev + curr    # only two states are ever live
7     return curr
```

Listing 30: The same recurrence, tabulated

The final line illustrates a general reduction. When a recurrence depends only on the previous k states, the table collapses from $O(n)$ space to $O(k)$.

Dynamic programming

Dynamic programming applies when a problem has **optimal substructure**, meaning the optimal solution is composed of optimal solutions to subproblems, together with **overlapping subproblems**, meaning those subproblems recur. Absent overlap, plain divide and conquer suffices. Absent optimal substructure, a greedy or exhaustive method is required.

Formulating a DP

Five questions, answered in order:

1. **State.** What arguments fully determine a subproblem? This is the signature of the function or the index set of the table.
2. **Recurrence.** How is a state's value expressed in terms of smaller states?
3. **Base cases.** Which states are known outright?
4. **Order.** In what order can states be evaluated so that dependencies are resolved first? Top-down recursion answers this automatically.
5. **Answer.** Which state holds the final result?

The complexity is the number of states multiplied by the cost of evaluating one state.

Canonical state definitions. Most problems reuse a small number of state shapes.

State	Meaning	Typical problem
$\text{dp}[i]$	best over the prefix ending at i	maximum subarray, house robber
$\text{dp}[i][j]$	best over two prefixes	edit distance, longest common subsequence
$\text{dp}[i][w]$	first i items, capacity w	0/1 knapsack
$\text{dp}[i][j]$	best over the interval $[i, j]$	matrix chain, palindrome partition

Backtracking

Backtracking is recursion over a search tree with an explicit undo step. It enumerates permutations, subsets, and combinations, and prunes branches that cannot yield a valid solution.

```

1 def solve(path, choices, results):
2     if is_complete(path):
3         results.append(path[:]) # copy, since path is mutated in place
4         return
5     for c in choices:
6         if not is_valid(path, c):
7             continue           # prune
8         path.append(c)         # choose
9         solve(path, remaining(choices, c), results)
10        path.pop()             # undo

```

Listing 31: Backtracking skeleton

The `path[:]` copy is mandatory. Appending `path` itself stores a reference to a list that subsequent mutation will alter, so every recorded result ends up identical.

For unconstrained enumeration, the standard library is preferable to a hand-written recursion.

```

1 from itertools import permutations, combinations, product
2
3 permutations(xs, r)      # ordered arrangements of length r
4 combinations(xs, r)     # unordered selections of length r
5 product(xs, repeat=r)   # r-tuples with repetition, the r-fold
                        Cartesian power

```

Listing 32: Enumeration without recursion

These correspond exactly to the counting formulas $P(n, r) = n!/(n - r)!$, $\binom{n}{r}$, and n^r .

3.5 Heaps and Priority Queues

A binary heap maintains the minimum of a collection under insertion and deletion at logarithmic cost. Python's `heapq` operates on an ordinary list interpreted as a heap, so no dedicated type is involved.

Operation	Call	Cost
Convert a list in place	<code>heapify(xs)</code>	$O(n)$
Insert	<code>heappush(h, x)</code>	$O(\log n)$
Remove the minimum	<code>heappop(h)</code>	$O(\log n)$
Inspect the minimum	<code>h[0]</code>	$O(1)$
Push then pop	<code>heappushpop(h, x)</code>	$O(\log n)$, one pass

A heap provides no ordering beyond its root. Iterating over the underlying list does not produce sorted output.

Minimum only. `heapq` implements a min-heap exclusively. A max-heap is obtained by negating the keys on insertion and negating again on extraction. For records, push the tuple `(-priority, item)`, since tuples compare lexicographically and the first field dominates.

Top- k under a stream

To retain the k largest of n elements, maintain a min-heap of size k . Each new element is compared against the root, which is the smallest of the current k . If the element is larger, the root is evicted and the element inserted.

The cost is $O(n \log k)$ and the memory is $O(k)$. Sorting the full input costs $O(n \log n)$ and requires all n elements in memory at once, so the heap is superior when $k \ll n$ and is the only option when the input arrives as a stream of unknown length.

```
1 import heapq
2
3 def top_k(stream, k):
4     h = []                                     # min-heap holding the k best
5     so far
6     for x in stream:
7         if len(h) < k:
8             heapq.heappush(h, x)
9         elif x > h[0]:                         # h[0] is the weakest survivor
10            heapq.heapreplace(h, x)           # pop the root, then push x
11    return sorted(h, reverse=True)
```

Listing 33: The k largest elements

When the whole input already fits in memory, the library provides this directly as `heapq.nlargest(k, xs)` and `heapq.nsmallest(k, xs)`, both of which accept a `key` argument.

Two heaps for a running median. Maintain a max-heap over the lower half of the values and a min-heap over the upper half, rebalancing after each insertion so their sizes differ by at most one. The median is then read from the root of the larger heap, or is the mean of the two roots when the sizes are equal. Insertion costs $O(\log n)$ and the median query is $O(1)$.

3.6 Sliding Window

A sliding window is a contiguous span of a sequence, delimited by two indices ℓ and r , together with a state summarizing the elements inside it. The window advances by admitting the element at r and evicting the element at ℓ . The technique applies when the state can be updated in $O(1)$ under those two operations, so that recomputing it from scratch is never necessary.

The recognition cue is a request for a *contiguous* subarray or substring subject to a constraint. Contiguity is required. A problem about subsequences or subsets admits reordering and belongs to a different paradigm.

Why the cost is linear

The amortization argument

Both indices are monotone. r advances once per iteration of the outer loop and ℓ never decreases, so each index takes at most n values over the entire run. The inner **while** loop is therefore bounded by n increments *in total* rather than n increments per outer iteration. The nesting is syntactic. The work is $O(n)$: every element is admitted exactly once and evicted at most once. The naive alternative enumerates all $O(n^2)$ spans and pays the cost of the state on each.

The precondition is that ℓ may safely move forward. This holds when the constraint is **monotone in the window**: if a window violates the constraint, every window containing it also violates it. Under that property, once $[\ell, r]$ fails, no window starting before ℓ and ending at r or later can succeed, so the discarded prefixes are provably useless.

Fixed-size windows

When the length k is given, there is no contraction step. The window is initialized once and then translated, one position at a time, by a paired admit and evict.

```
1 def max_window_sum(xs, k):
2     if len(xs) < k:
3         return None
4     total = sum(xs[:k])          # the window is built once, in O(k)
5     best = total
6     for r in range(k, len(xs)):
7         total += xs[r] - xs[r-k] # admit xs[r], evict xs[r-k]
8         best = max(best, total)
9     return best
10
11 max_window_sum([2, 1, 5, 1, 3, 2], 3)    # 9, from the span [5, 1, 3]
```

Listing 34: Maximum sum over any window of length k

The single line `total += xs[r] - xs[r-k]` is the whole method. It replaces an inner loop of length k , reducing $O(nk)$ to $O(n)$. The index $r - k$ is the element leaving the window, since the window covering positions $r - k + 1$ through r has just replaced the one covering $r - k$ through $r - 1$.

A counter as the window state. The state need not be a number. Matching a pattern's permutations, described earlier under **Counter**, carries a frequency table as the state and compares it against a fixed target.

```

1 from collections import Counter
2
3 def find_anagrams(t, p):
4     k = len(p)
5     need = Counter(p)
6     have = Counter(t[:k])
7     out = [0] if have == need else []
8     for r in range(k, len(t)):
9         have[t[r]] += 1           # admit
10        have[t[r - k]] -= 1       # evict
11        if have[t[r - k]] == 0:
12            del have[t[r - k]]    # see the note below
13        if have == need:
14            out.append(r - k + 1)  # the window's left index
15    return out
16
17 find_anagrams("cbaebabacd", "abc") # [0, 6]

```

Listing 35: Every starting index at which a permutation of p occurs in t

Each shift performs a constant amount of work and one dictionary comparison over an alphabet of bounded size, giving $O(n)$.

Zero-valued keys in a Counter

Decrementing to zero leaves the key present with value 0. Two consequences follow, and they behave differently.

Equality. On Python 3.10 and later, `Counter.__eq__` compares counts over the union of the keys and treats a missing key as zero, so `Counter({'a':1,'b':0}) == Counter({'a':1})` evaluates to `True`. On Python 3.9 and earlier the class inherits `dict.__eq__`, which compares key sets, and the same expression is `False`.

Size. `len(counter)` counts keys on every version. A key held at zero still contributes to `len`, so any window that tests the number of *distinct* elements must delete the key on reaching zero.

Deleting the exhausted key satisfies both, on every version.

Variable-size windows

When the length is unknown, r expands the window on every iteration and an inner loop moves ℓ forward. The two problem shapes differ in the test governing that inner loop and in the point at which the answer is recorded.

	Longest valid window	Shortest sufficient window
Inner loop	<code>while</code> invariant is broken	<code>while</code> condition holds
Effect	restore validity	test whether ℓ can be given up
Record at	after the loop, on a valid window	inside the loop, before evicting
Answer	<code>max(best, r - 1 + 1)</code>	<code>min(best, r - 1 + 1)</code>

Longest valid window. The window is repaired after each expansion, so it is valid at the bottom of every outer iteration, which is where its length is measured.

```

1 def longest_unique(s):
2     seen = set()                # the state: which characters are in
    the window
3     left = best = 0
4     for right, c in enumerate(s):
5         while c in seen:        # admitting c would break the
    invariant
6             seen.remove(s[left]) # evict from the left until it would
    not
7             left += 1
8             seen.add(c)         # admit
9             best = max(best, right - left + 1) # the window is valid here
10    return best
11
12 longest_unique("abcabcbb")      # 3, from "abc"
13 longest_unique("pwwkew")       # 3, from "wke"

```

Listing 36: Longest substring with no repeated character

The eviction loop is written before the admission so that the invariant is never violated inside the set. Admitting first and then repairing is equally correct provided the repair condition is written to match, and the ordering must be chosen once and held to.

```

1 from collections import defaultdict
2
3 def longest_k_distinct(s, k):
4     count = defaultdict(int)    # the state: a frequency table over
    the window
5     left = best = 0
6     for right, c in enumerate(s):
7         count[c] += 1           # admit
8         while len(count) > k:   # invariant: at most k
    keys
9             count[s[left]] -= 1
10            if count[s[left]] == 0:
11                del count[s[left]] # required, or len(count)
    is wrong
12            left += 1
13            best = max(best, right - left + 1)
14    return best
15
16 longest_k_distinct("eceba", 2)  # 3, from "ece"

```

Listing 37: Longest substring with at most k distinct characters

Here the admission precedes the repair, because the invariant is stated over the window's state rather than over the incoming element. The `del` is what makes `len(count)` equal to the number of distinct characters currently present.

Shortest sufficient window. The inner loop contracts while the target condition is still satisfied, which searches for the smallest left index that keeps it satisfied. The length is recorded before each eviction, since the window may cease to qualify immediately afterward.

```

1 def min_length(xs, t):
2     total = 0
3     left = 0
4     best = float('inf')
5     for right, x in enumerate(xs):
6         total += x                                # admit
7         while total >= t:                          # the window already
8             suffices
9             best = min(best, right - left + 1)    # record, then try to
10            shrink
11            total -= xs[left]                      # evict
12            left += 1
13    return 0 if best == float('inf') else best
14 min_length([2, 3, 1, 2, 4, 3], 7)    # 2, from [4, 3]
15 min_length([1, 1, 1], 7)           # 0, no window qualifies

```

Listing 38: Shortest subarray with sum at least t , over positive values

`float('inf')` as the initial value of a minimum removes the special case for an empty candidate set. The sentinel is then translated to whatever the problem specifies for “no such window,” which is commonly 0.

When the window does not apply

The monotonicity precondition

`min_length` above is correct only because every element is positive. Positivity makes `total` nondecreasing in the window’s length, so `total >= t` is monotone and a failing prefix can be discarded permanently.

Introduce a negative value and that reasoning collapses. Extending a window may now decrease its sum, and a left index abandoned earlier could belong to the optimal answer. On `[3, -2, 5]` with $t = 6$, the window `[3, -2, 5]` qualifies while neither the prefix `[3, -2]` nor the suffix `[5]` taken alone reveals it by contraction.

The replacement is prefix sums with a hash map, which considers every left endpoint rather than only the ones a monotone rule preserves. The check for a negative value in the constraints is therefore the first thing to perform on any subarray-sum problem.

Common errors

- The window length is `right - left + 1`. Omitting the `+1` is the standard off-by-one, since both endpoints are inclusive.
- Recording the answer at the wrong point. A longest-window problem measures after the repair loop. A shortest-window problem measures inside the contraction loop, before the eviction.
- An `if` where a `while` is required. One eviction restores the invariant only when a single admission can break it by exactly one unit. The general repair is a loop.
- Failing to evict from the state. Advancing `left` without removing `xs[left]` from the set, counter, or running sum leaves the state describing a window that no longer exists.
- Retaining zero-valued keys in a `Counter` or `defaultdict` while testing `len` against a distinct-count bound.
- Recomputing the state over the span, as in `sum(xs[left:right+1])` inside the loop. This restores the $O(n^2)$ cost the window exists to remove.

3.7 Pandas

3.7.1 The Data Model

A DataFrame looks like a table, but two properties make it behave unlike a SQL table or a raw array. Every DataFrame carries a labeled axis called the index, and every column holds values of a single resolved dtype. The index drives automatic alignment across operations, and the dtype decides what an operation means and how fast it runs.

Definition 3.1 (Series). A Series is a one-dimensional labeled array of a single dtype. It pairs an ordered sequence of values with an **index** of labels, so every element is addressable by its label as well as by its position.

Definition 3.2 (DataFrame). A DataFrame is a two-dimensional structure whose columns are Series sharing one common row index. Each column carries its own dtype, so a DataFrame has no single dtype of its own. The column labels form a second axis, itself an **Index**, which makes the structure symmetric: rows and columns are both labeled.

A DataFrame behaves much like a dictionary of aligned Series keyed by column name. Selecting a single column returns the underlying Series, and that Series keeps the frame's row index.

```
1 df = pd.DataFrame({
2     'city': ['Austin', 'Dallas', 'Austin'],
3     'temp': [98, 101, 96],
4 })
5
6 df['temp']           # a Series, dtype int64, indexed 0,1,2
7 df.index            # RangeIndex(start=0, stop=3, step=1)
8 df.columns          # Index(['city', 'temp'], dtype='object')
9 df.dtypes           # city -> object, temp -> int64
```

Listing 39: A frame is columns of typed Series over a shared index

The index. A fresh DataFrame receives a default **RangeIndex** running from 0, so the labels coincide with positions and the distinction stays invisible. The index becomes load-bearing the moment it holds anything else, such as a datetime, a string key, or the grouping key that a **groupby** leaves behind. An index may be of any dtype and is not required to be unique. Two consequences shape later sections. Label-based access through **loc** and positional access through **iloc** diverge as soon as the index departs from a clean **0..n** range. And the index survives operations, so a filter or a sort carries the original labels forward rather than renumbering, which is the reason **reset_index** exists to install a fresh range when the old labels have served their purpose.

Alignment on labels

Binary operations align their operands on the index before computing. pandas matches labels, evaluates on the ones the operands share, and fills every label present in only one operand with a missing value. Position plays no part in this matching. The same rule governs assignment and boolean-mask selection, so a mask built on one frame lands on the correct rows of another as long as the two share an index, whatever their row order.

```

1 a = pd.Series([1, 2, 3], index=['x', 'y', 'z'])
2 b = pd.Series([10, 20], index=['y', 'z'])
3
4 a + b
5 # x      NaN      <- 'x' has no match in b
6 # y      12.0
7 # z      23.0

```

Listing 40: Addition aligns by label and fills the gaps

A boolean Series used as a mask, `df.loc[mask]`, is matched to the frame by label, so the mask and the frame need not share a row order for the selection to land on the right rows. The same rule cuts the other way: combining two columns taken from different frames introduces `NaN` at every label where their indices disagree, even when the columns appear to line up by position.

Dtypes. Each column resolves to a single dtype, and that dtype determines both the operations available and the speed at which they run. Vectorized arithmetic and the `.str` and `.dt` accessors all require a genuine numeric, datetime, or string dtype underneath. The core dtypes are collected below.

<i>dtype</i>	<i>Stores</i>	<i>Missing</i>	<i>Note</i>
int64, float64	Integers and floating point	<code>NaN</code> (float only)	The numpy-backed numeric types.
bool	Booleans	<code>none</code>	The result of every comparison.
object	Arbitrary Python objects, usually strings	<code>None</code> , <code>NaN</code>	Python-speed, the untyped fallback.
datetime64[ns]	Timestamps	<code>NaT</code>	Unlocks the <code>.dt</code> accessor.
timedelta64[ns]	Durations	<code>NaT</code>	Produced by subtracting timestamps.
category	A fixed set of repeated labels	<code>NaN</code>	Compact, and can carry an order.
Int64, string, boolean	As their lowercase forms	<code>pd.NA</code>	Nullable, capitalized, missing-aware.

Core dtypes. The lowercase numpy-backed types are the default. The capitalized nullable types carry `pd.NA` and preserve integer and boolean columns across missing values.

The `object` dtype is the fallback for anything without a native type, which in practice is mostly Python strings. It stores pointers to Python objects, so work on it runs at Python speed rather than vectorized speed. Converting an `object` column of strings to a real `string` or `category` dtype, or a column of date-like text to `datetime64[ns]`, is usually the first step.

Missing values force a dtype

numpy `NaN` is a floating-point value, and numpy integer arrays have no representation for a missing entry. A single missing value in an integer column therefore promotes the whole column to `float64`. This appears constantly after a left join or a `reindex`, where unmatched rows introduce `NaN` and an `int64` count column comes back as `float64`. Datetime and timedelta columns use `NaT` in the same role. The nullable dtypes, written with a capital initial (`Int64`, `boolean`, `string`), instead carry `pd.NA` as a type-agnostic missing marker and hold an integer column at integer dtype through the gap.

```

1 pd.Series([1, 2, 3]).dtype           # dtype('int64')
2 pd.Series([1, 2, np.nan]).dtype      # dtype('float64'), the gap
   forces the promotion
3
4 pd.Series([1, 2, pd.NA], dtype='Int64')
5 # 0          1
6 # 1          2
7 # 2      <NA>           # integer dtype preserved,
   missing marked by pd.NA

```

Listing 41: A NaN forces float64, a nullable dtype keeps the integer

3.7.2 Selection and Filtering

Three access needs recur: selecting by label, selecting by position, and selecting by predicate. pandas keeps the first two on separate accessors, `loc` for labels and `iloc` for positions, and expresses the third through a boolean mask. Restricting rows is the pandas **WHERE**, and every later pattern builds on it.

Bracket indexing. The bare `[]` operator on a DataFrame is overloaded by the type of its argument. A string selects a column and returns a Series, a list of strings selects several columns and returns a DataFrame, and a boolean Series filters rows. The overloading is convenient but ambiguous, and a key that could name either a row or a column is resolved as a column. For anything past a plain column pick, `loc` states the intent without ambiguity.

```

1 df['temp']           # a Series (one column)
2 df[['city', 'temp']] # a DataFrame (a column list)
3 df[df['temp'] > 97]  # row filter by boolean mask

```

Listing 42: The overloaded bracket

Definition 3.3 (`loc` and `iloc`). `loc` selects by label and `iloc` by integer position. Both take a row selector and an optional column selector, `df.loc[rows, cols]`, and each selector accepts a single key, a list, a slice, or a boolean mask. A `loc` label slice includes both endpoints, whereas an `iloc` position slice excludes the stop in the usual Python way.

The inclusive label slice is a frequent trap. `df.loc['a':'c']` returns rows `a` through `c` inclusive, while `df.iloc[0:3]` returns three rows and stops before position 3.

```

1 df.loc[3, 'temp']           # row label 3, column 'temp'
2 df.loc[df['temp'] > 97, ['city', 'temp']] # mask picks rows, list
   picks columns
3 df.iloc[0]                  # first row by position
4 df.iloc[-1]                 # last row by position

```

Listing 43: Label access, position access, filtered selection

Boolean masks. A comparison on a Series returns a boolean Series of the same length, and passing it to `loc` keeps the `True` rows. Compound conditions combine with the bitwise operators `&`, `|`, and `~`, and each comparison must be wrapped in parentheses because the bitwise operators bind tighter than the comparisons. The Python keywords `and`, `or`, and `not` do not work on a Series, since they demand a single truth value and raise on an array.

```

1 mask = (df['temp'] > 97) & (df['city'] == 'Austin')
2 df.loc[mask]
3
4 df.loc[~df['city'].isin(['Dallas', 'Houston'])]    # NOT IN

```

Listing 44: Combine masks with bitwise operators, parenthesize each side

Membership and range. `isin(values)` is a vectorized membership mask, the pandas IN, and negating it with `~` gives the NOT IN. `between(lo, hi)` tests a closed interval. Both read more cleanly than the equivalent chain of `|` clauses or the paired inequalities they stand in for.

The query string. `query` filters from a string expression, naming columns bare and outer variables with a leading `@`. It reads well for a long compound predicate and spares the repetition of the frame name. Being parsed at call time, it runs slower than a mask and accepts only expressions it can interpret, so a mask stays the general tool.

```

1 threshold = 97
2 df.query('temp > @threshold and city == "Austin"')

```

Listing 45: query names columns bare and locals with `@`

Chained assignment is unreliable

A selection can return a view onto the original's data or an independent copy, and which one arises is not predictable from the expression. Assigning through a chained selection, `df[df['t'] > 100]['city'] = 'hot'`, may therefore write into a discarded temporary and leave the original untouched, which is what raises the `SettingWithCopyWarning`. The reliable form selects the rows and the column together in a single `loc`, so the write lands on the frame itself.

```

1 df[df['temp'] > 100]['city'] = 'hot'          # unreliable, warns
2 df.loc[df['temp'] > 100, 'city'] = 'hot'      # correct, one selection

```

Listing 46: The chained write warns and may no-op, the single loc is definite

3.7.3 Missing Data

Missing entries are carried by a sentinel rather than a separate flag: `NaN` in a float column, `NaT` in a datetime column, and `pd.NA` in a nullable column. Because the sentinel lives in the data, detecting, dropping, and filling it are ordinary column operations, and the arithmetic rules around it decide how a reduction or a comparison treats a gap.

NaN never equals itself

`NaN` compares unequal to everything, itself included, so `s == np.nan` is `False` on every row and never finds a missing value. Detection goes through `isna` and `notna` instead, which test the sentinel directly. This mirrors SQL, where `x = NULL` is never true and the test is `IS NULL`. One consequence is useful: `s != s` is `True` exactly where `s` is `NaN`, which is how a duplicate-tolerant equality check spots gaps.

Detecting and dropping. `isna` returns a boolean mask of the missing cells and `notna` its complement, both feeding a filter directly. `dropna` removes rows holding any missing value, restricted to certain columns with `subset` or loosened with `thresh`, which keeps a row that

clears a minimum count of non-missing values. Aggregations skip NaN on their own, so `mean` and `sum` need no explicit filtering.

```
1 df[df['manager_id'].notna()]           # keep rows with a manager
2 df.dropna(subset=['salary'])          # drop rows missing salary
3 df.dropna(thresh=3)                   # keep rows with >= 3 non-
   missing fields
```

Listing 47: Detect, filter, and drop missing values

Filling. `fillna(value)` substitutes a constant or an aligned Series, the pandas COALESCE. Passing a dict fills each column with its own default. A group-aware fill runs through `groupby(...)` and `.transform`, which replaces a gap with a statistic of its own group, such as the group mean.

```
1 df['manager_id'].fillna(0)
2
3 # fill each department's missing salaries with that department's mean
4 df['salary'] = df['salary'].fillna(
5     df.groupby('dept')['salary'].transform('mean'))
```

Listing 48: Constant fill and group-relative fill

Forward fill carries the last observation

`ffill` propagates the last valid value forward into the gaps that follow it, and `bfill` pulls the next valid value backward. On an ordered series this is carrying the last observation forward, the standard way to hold a price or a level across timestamps that report no change. The fill follows the current row order, so a time series must be sorted by time first, and a per-entity fill runs through `groupby` so one entity's last value cannot leak into the next.

Combining sources. `combine_first` fills the missing cells of one frame from another aligned on the index, taking the first non-missing value at each position. It is the COALESCE across two whole frames, used to layer a fallback table under a primary one.

3.7.4 Column Operations and Vectorization

pandas operations run array-at-a-time. A column expression applies to the whole Series in one pass over the underlying array, which runs far faster than a Python loop and removes the need for a loop in most transformations. This vectorized style is the default, and the row-wise `apply` is the fallback for the rare case that resists it.

Building columns. Arithmetic and comparisons between columns produce new Series aligned on the index, assigned back by name. `assign` does the same inside a chain, returning a copy with columns added or overwritten. Its callable form receives the frame, which lets a new column read from the frame as it is being built.

```
1 df['ratio'] = df['rev'] / df['units']
2
3 df = df.assign(
4     ratio=lambda d: d['rev'] / d['units'],
5     is_big=lambda d: d['rev'] > 1000)
```

Listing 49: Direct assignment and the chainable assign

Prefer vectorized operations over apply

Arithmetic, comparisons, numpy ufuncs, and the `.str` and `.dt` accessors are all vectorized and act on the whole column at compiled speed. A Python-level `apply` runs the function once per element or row and is slower by one to two orders of magnitude. Nearly every transformation has a vectorized form, and finding it repays the effort at any nontrivial size.

Conditional columns. A two-way choice is `np.where(cond, a, b)`, taking `a` where the condition holds and `b` elsewhere. A multi-way choice is `np.select(conditions, choices, default)`, which takes the first matching condition in order. `Series.where(cond, other)` keeps the value where the condition holds and substitutes `other` elsewhere, and `mask` is its complement. Each of these replaces a conditional `apply` with a single vectorized call.

```
1 df['flag'] = np.where(df['temp'] > 97, 'hot', 'mild')
2
3 conditions = [df['score'] >= 90, df['score'] >= 80]
4 grades     = ['A', 'B']
5 df['grade'] = np.select(conditions, grades, default='C')
```

Listing 50: Two-way and multi-way conditionals without apply

The `str` and `dt` accessors. A Series of string dtype exposes vectorized string methods under `.str`, and a datetime Series exposes calendar and clock fields under `.dt`. Both return aligned Series, so their results drop straight into masks and new columns. `pd.to_datetime` parses text or epoch values into `datetime64[ns]`, the prerequisite for `.dt`.

```
1 df['user'].str.lower().str.startswith('a')      # chainable, vectorized
2 df['email'].str.contains('@company', na=False)  # na=False maps missing
          to False
3
4 df['ts'] = pd.to_datetime(df['ts'])
5 df['year'] = df['ts'].dt.year
6 df['weekday'] = df['ts'].dt.dayofweek           # Monday = 0
```

Listing 51: Vectorized string and datetime work

The workhorses are `.str.contains`, `.str.extract` for a regex capture, `.str.split`, `.str.len`, and positional slicing through `.str[:3]`, alongside `.dt.year`, `.dt.month`, `.dt.date`, and `.dt.floor('D')`. The `na` argument on the string predicates fixes how a missing value scores in the resulting mask.

Element mapping. `map` on a Series applies a dict or function element by element, which suits recoding a small fixed set of values. `replace` substitutes chosen values throughout a column, `astype` casts to a target dtype, and `round` sets the number of decimals on a numeric column.

Binning into categories. `pd.cut` slices a numeric column at explicit edges into labeled bands, the vectorized form of a CASE ladder over ranges. `pd.qcut` instead cuts at quantiles, producing bins of roughly equal count, which is the NTILE operation. Both return a `category` column that feeds `value_counts` or a `groupby` directly.


```

1 df['band'] = pd.cut(df['salary'],
2                     bins=[0, 50_000, 80_000, np.inf],
3                     labels=['<50k', '50-80k', '80k+'])
4
5 df['quartile'] = pd.qcut(df['salary'], 4, labels=False)  # 0..3 by
    quantile

```

Listing 52: Fixed-edge bins and equal-frequency bins

apply and the axis argument

`Series.apply(f)` calls `f` once per element, and `DataFrame.apply(f, axis=1)` calls it once per row, handing each row over as a `Series`. Both run at Python speed and are the last resort, kept for logic with no vectorized form. The `axis` argument is a recurring source of confusion: `axis=0` runs down the index and operates on each column, while `axis=1` runs across the columns and operates on each row. A row-wise `apply` spanning several columns is the one case where `axis=1` earns its cost.

3.7.5 String Matching with Regex

The `.str` accessor backs its matching methods with regular expressions. The three predicates differ only in where the pattern is anchored, which decides whether it must match the whole string or merely appear inside it. Patterns are written as raw strings, `r'...'`, so backslashes reach the regex engine intact.

<i>Method</i>	<i>Anchoring</i>	<i>True when</i>
<code>str.match</code>	Start only, an implicit <code>^</code>	The pattern matches a prefix of the string.
<code>str.fullmatch</code>	Both ends, an implicit <code>^...\$</code>	The pattern matches the entire string.
<code>str.contains</code>	None	The pattern is found anywhere in the string.

The three regex predicates on `.str`. All return a boolean `Series` and read `pat` as a regex. `contains` is the pandas REGEXP or RLIKE, and its unanchored search subsumes LIKE `'%x%'`.

Substitution and extraction. `str.replace(pat, repl, regex=True)` substitutes on a pattern. From pandas 2.0 the `regex` flag defaults to `False` and treats `pat` as literal text, so a pattern replacement must set `regex=True`. `str.extract(pat)` returns one column per capture group in the pattern, pulling structured fields out of a string, and `str.extractall` returns every match rather than only the first.

Match methods return NaN on missing input

On a row whose string is missing, `match`, `fullmatch`, and `contains` return `NaN` in place of a boolean. A mask carrying `NaN` raises when passed to `loc`, so a matching predicate used as a filter takes `na=False` to score missing strings as non-matches. Case sensitivity is set through `case=False` or `flags=re.IGNORECASE`.

```

1 # fullmatch checks the entire string; the ^ and $ are implied
2 valid = users['email'].str.fullmatch(r'[\w]+@[A-Za-z]+\com', na=False)
3 users.loc[valid].sort_values('user_id')
4
5 # one column per capture group
6 users['email'].str.extract(r'@([A-Za-z]+\com') # the domain

```

Listing 53: Validate whole strings, then extract a field

Two habits keep patterns tight. `fullmatch` already anchors both ends, so a leading `^` or trailing `$` adds nothing. And `\w` already covers letters, digits, and the underscore, so a class such as `[\w_]` repeats the underscore it already contains.

3.7.6 Grouping and Aggregation

Grouping follows the split-apply-combine shape: split the rows by a key, apply a reduction within each group, and combine the results into one row per group. It is the pandas `GROUP BY`, with the wrinkle that the grouping is a distinct lazy object holding the split, and no work happens until a reduction is named.

Definition 3.4 (`GroupBy`). `df.groupby('k')` returns a `GroupBy` object that partitions the rows by the values of `k` without computing anything. A reduction called on it, such as `.sum()` or `.agg(...)`, evaluates per group and combines to one row each. The key may be a list of columns for a composite grouping.

Named aggregation. The clearest reduction form names each output column and gives its source column and function as a tuple, `out=('col', 'func')`. It maps onto `func(col)` AS `out` directly, and it extends to several outputs in one call.

```

1 g = (df.groupby('customer_id', as_index=False)
2       .agg(n_orders=('order_id', 'count'),
3            total=('amount', 'sum')))

```

Listing 54: Named aggregation, one row per group

The older forms remain available. A dict `agg({'amount': 'sum'})` maps a column to a function, and a list `agg(['sum', 'mean'])` applies several functions at once and returns a frame with a `MultiIndex` on its columns. The named form sidesteps that nested column index.

The group key becomes the index. A `groupby` puts the key on the index of the result, one level per key column. `reset_index()` moves it back to an ordinary column and installs a clean range index, and passing `as_index=False` to `groupby` does the same inline. The key needs to leave the index whenever a later step selects it as a column or expects a plain integer row label.

count versus size versus nunique

`count` tallies non-null values in the group, `size` tallies rows including nulls, and `nunique` tallies distinct non-null values. The gap between `count` and `size` is where left-join placeholders hide: an unmatched row carries `NaN`, so `size` scores it as one row while `count` scores it as zero. Choosing the intended one of these is a routine source of off-by-*N* errors.

<i>Reduction</i>	<i>SQL analog</i>	<i>What it computes</i>
count	COUNT(col)	Non-null values in the group.
size	COUNT(*)	Every row in the group, nulls included.
nunique	COUNT(DISTINCT col)	Distinct values, nulls excluded.
sum	SUM(col)	Total of the values. An empty group sums to 0.
mean	AVG(col)	Arithmetic mean.
median	PERCENTILE_CONT(0.5)	Middle value by order.
min	MIN(col)	Smallest value.
max	MAX(col)	Largest value.
std	STDDEV_SAMP(col)	Sample standard deviation, ddof = 1.
var	VAR_SAMP(col)	Sample variance, ddof = 1.
first	FIRST_VALUE (window)	First value in the current row order.
last	LAST_VALUE (window)	Last value in the current row order.
any / all	BOOL_OR / BOOL_AND	Whether any, or all, values are truthy.

Common reductions for `agg`. All skip NaN except `size`. `std` and `var` default to the sample estimator, unlike SQL engines that default to population.

transform versus agg. `agg` returns one row per group. `transform` returns a result the same length as the input, broadcasting each group’s statistic back onto every member row. It is the tool for a group-relative column, where each row is measured against a summary of its own group.

```
1 dept_mean = df.groupby('dept')['salary'].transform('mean')
2 df['above_dept_avg'] = df['salary'] > dept_mean
```

Listing 55: A group-relative column via transform

Filtering whole groups. `groupby(...).filter(pred)` keeps or drops entire groups by a group-level predicate, the reading of HAVING. The predicate runs once per group in Python, so on a wide key the vectorized equivalent through `transform('size')` and a mask runs faster.

```
1 # groups of size greater than one
2 df.groupby('email').filter(lambda g: len(g) > 1)
3
4 # vectorized equivalent
5 df[df.groupby('email')['email'].transform('size') > 1]
```

Listing 56: Keep groups by a group predicate (HAVING)

groupby drops null keys by default

A row whose grouping key is NaN is excluded from every group, so a null-keyed record drops out of the result. Passing `dropna=False` retains the null-key group. Set it whenever a grouping key may be missing and those rows still belong in the output.

Frequencies and distinct values. `value_counts` on a Series tallies how often each value occurs and returns the counts sorted high to low, with `normalize=True` giving proportions instead. `unique` lists the distinct values in order of appearance and `nunique` counts them.

`drop_duplicates` removes repeated rows, keyed to chosen columns through `subset` and keeping the first or last of each run through `keep`. A grouped string concatenation, the `GROUP_CONCAT`, falls out of `agg` with a join function.

```
1 df['dept'].value_counts(normalize=True)      # share of rows per dept
2 df.drop_duplicates(subset=['email'], keep='first')
3
4 df.groupby('dept')['name'].agg(', ', '.join)  # GROUP_CONCAT
```

Listing 57: Frequencies, distinct rows, and joined strings

3.7.7 Joining and Combining

Two operations bring separate frames together. `merge` combines columns by matching keys, the pandas JOIN. `concat` stacks frames along an axis without matching. Reach for `merge` when rows are paired by a key and for `concat` when frames are laid end to end.

Definition 3.5 (merge). `merge` joins two frames on one or more key columns. `on` names a key shared by both, while `left_on` and `right_on` pair keys that differ in name. `how` selects which unmatched rows survive, taking `inner`, `left`, `right`, `outer`, or `cross`, and defaults to `inner`.

The `how` argument carries the same meaning as the SQL join it names. `inner` keeps rows matched on both sides. `left` and `right` keep every row from one side and fill the other with NaN. `outer` keeps unmatched rows from both. `cross` forms the Cartesian product and takes no key.

```
1 orders.merge(customers, on='customer_id', how='left')
2 a.merge(b, left_on='cid', right_on='id', how='inner')
```

Listing 58: A keyed merge and a rename-free key pairing

Join type decides which unmatched rows survive

The `how` argument decides whether an unmatched row survives as a hole or disappears. A left join keeps every left row and fills the unmatched right columns with NaN, while an inner join drops those rows outright. The choice carries a dtype consequence: filling an integer column with NaN promotes it to float64, so an int64 count returns as float64 after a left join.

Overlapping column names. When both frames carry a non-key column of the same name, `merge` disambiguates the two with the default suffixes `_x` and `_y`. The result then requires tracking which side is which, so passing an explicit `suffixes=` pair, or renaming beforehand, keeps the output readable.

Joining on the index. `join` is a convenience wrapper over `merge` that matches on the index of the right frame by default, which suits attaching a lookup table already indexed by its key. `merge` stays the explicit tool for arbitrary key columns, and it reads more clearly whenever the key is an ordinary column rather than the index.

Duplicate keys multiply rows

A merge emits one row per matching pair of keys. When a key repeats on both sides, the matches fan out as their product, so a many-to-many merge can return far more rows than either input. Passing `validate='one_to_one'` or `'one_to_many'` makes `merge` assert the expected multiplicity and raise on a violation.

Stacking with `concat`. `concat` joins a list of frames along one axis. `axis=0` stacks rows and aligns on the columns, `axis=1` stacks columns and aligns on the index. `ignore_index=True` discards the source indices and lays down a fresh range index, which matters when the pieces carry overlapping or meaningless labels.

```
1 pd.concat([jan, feb, mar], ignore_index=True)    # rows, fresh index
2 pd.concat([features, labels], axis=1)           # columns, aligned on
    index
```

Listing 59: Stack rows, then stack columns

Semi- and anti-joins through `isin`. Filtering a frame by whether its key appears in another needs no materialized join. `isin` against the other frame's key column keeps the matching rows, a semi-join, and negating it keeps the non-matching rows, an anti-join. Neither widens the frame nor pulls columns from the other side.

```
1 customers[customers['id'].isin(orders['customer_id'])]    # has orders
2 customers[~customers['id'].isin(orders['customer_id'])]  # has no
    orders
```

Listing 60: Semi-join keeps matches, anti-join keeps non-matches

Filter anti-joins on a key

An exclusion should test the column that identifies a row uniquely, which is the primary key. Filtering on a display attribute such as a name folds together distinct records that happen to share it, so one person's match can drop an unrelated person of the same name from the result. The membership form also holds an advantage over the SQL `NOT IN`: `isin` returns an independent boolean per row, so a missing value among the excluded keys cannot void the whole result the way a single `NULL` collapses `NOT IN` to zero rows.

As-of joins. `merge_asof` matches each left row to the single right row whose key is nearest without exceeding it, the backward direction being the default. It answers point-in-time questions, where a fact is read as of a moment rather than on an exact match. Both frames must be sorted on the join key, since the algorithm advances through them in order. The `by` argument confines matching within a group, pairing a row only with keys from the same group.

```

1 prices = prices.sort_values('start_date')
2 sales = sales.sort_values('purchase_date')
3
4 pd.merge_asof(sales, prices, by='product_id',
5               left_on='purchase_date', right_on='start_date')

```

Listing 61: Match each sale to the price in effect at its date

The join consults the start key alone and never a matching end key, so it is not a between-dates join and will attach a stale value to a left row falling past a window's close. It fits when each key marks the start of an interval running until the next one.

3.7.8 Reshaping

Reshaping moves data between the row axis and the column axis. In long form each row is one observation, keyed by its identifying columns. In wide form the values of one key spread across the columns, one column per distinct value. `pivot` carries a frame from long to wide, and `melt` carries it back.

Definition 3.6 (`pivot` and `pivot_table`). `pivot(index, columns, values)` lifts the distinct entries of the `columns` field onto the column axis, indexes the result by `index`, and fills each cell from `values`. It requires exactly one value per `(index, columns)` cell and raises on a duplicate. `pivot_table` adds an `aggfunc` that reduces colliding values, so it tolerates duplicates by aggregating them.

Pivoting to pair split rows. A single logical record is sometimes spread across several rows, distinguished by a type column with one measured value each. Pivoting that type column lifts its values into columns, bringing the pieces of one record onto a single row where they combine through column arithmetic. This replaces a self-join for the common case of a small fixed set of types, such as a start and an end.

```

1 wide = events.pivot(index=['machine_id', 'process_id'],
2                      columns='activity_type',
3                      values='timestamp')
4
5 wide['duration'] = wide['end'] - wide['start']

```

Listing 62: Pivot a start/end type column into aligned columns

pivot versus pivot_table

`pivot` reshapes without aggregating and raises when a cell would receive more than one value. `pivot_table` aggregates collisions through its `aggfunc`, which defaults to the mean. Use `pivot` when uniqueness is guaranteed by the data, so an unexpected duplicate surfaces as an error rather than being averaged away. Reach for `pivot_table` when collisions are expected and a summary of them is the intent.

Summarizing with `pivot_table`. `pivot_table` aggregates as it reshapes, so it takes several arguments a bare `pivot` does not. `aggfunc` sets the reduction, defaulting to the mean and accepting a list for several at once. `values` names the column or columns to summarize, `fill_value` replaces the empty cells, and `margins=True` appends row and column totals. Passing a list to `index` or `columns` builds a `MultiIndex` on that axis.

```

1 # long: one row per (date, ticker) -> wide: dates down, tickers
  across
2 panel = returns.pivot_table(index='date', columns='ticker', values='ret
  ')
3
4 pd.pivot_table(sales, index='region', columns='quarter',
5               values='revenue', aggfunc='sum',
6               fill_value=0, margins=True)

```

Listing 63: A long panel to a date-by-entity grid, and a summary with totals

The date-by-entity grid is the standard layout for time-series work: each column is one entity's series aligned to a shared date index, ready for column-wise returns, correlations, and rolling statistics.

Cross-tabulation. `pd.crosstab` counts the co-occurrences of two categoricals, a frequency table over their pairs. It is `pivot_table` specialized to counting, and `normalize=True` turns the counts into proportions across the whole table, a row, or a column.

```

1 pd.crosstab(df['department'], df['hire_year'])

```

Listing 64: Frequency of department against hire year

Melting wide to long. `melt` is the inverse operation. The `id_vars` columns stay fixed as identifiers, and the `value_vars` columns collapse into two columns, one holding the former column name and one holding its value. It normalizes a wide frame into the long form that grouping and plotting expect.

```

1 long = wide.melt(id_vars='region',
2                 value_vars=['q1', 'q2', 'q3', 'q4'],
3                 var_name='quarter', value_name='revenue')

```

Listing 65: Collapse quarter columns into rows

stack and unstack. `stack` moves a column level down into a row-index level, and `unstack` lifts a row-index level up into columns. They are the index-level counterparts of `melt` and `pivot`, and `pivot` itself is `set_index` followed by `unstack`. Reshaping often leaves a `MultiIndex` on the rows or columns, which `reset_index`, `droplevel`, or `rename_axis` flatten back to plain labels.

Exploding a list column. `explode` turns a column of lists into one row per element, repeating the other columns across the pieces. It expands a nested field into flat long form, inverting a group-and-collect, and is the pandas reading of a `JSON_TABLE` over an array column.

```

1 df.explode('skills')      # a row per skill, other columns repeated

```

Listing 66: One row per element of each list

3.7.9 Ordering and Row-Relative Logic

Some results depend on the order of the rows or on values in other rows. Ordering operations sort or rank the frame, and row-relative operations read a neighbor or a running window. These are the pandas counterpart of SQL window functions, and as with a SQL window the order must be stated, since a frame carries no inherent one.

Sorting. `sort_values(by, ascending, na_position)` orders rows by one or more columns, with `by` taking a list for a multi-key sort and `ascending` a matching list of directions. `na_position` places NaN at the start or the end. `sort_index` orders by the index labels instead.

```
1 df.sort_values(['dept', 'salary'], ascending=[True, False])
```

Listing 67: Multi-key sort with a direction per key

Extremes and top-N. `nlargest(n, col)` and `nsmallest(n, col)` return the top or bottom rows by a column and cost less than a full sort for a handful of rows. `idxmax` and `idxmin` return the index label of the extreme value, an `argmax`, which `loc` then reads as a row. Both break ties by first occurrence.

```
1 df.nlargest(3, 'salary')
2 df.loc[df['salary'].idxmax()]      # the single row holding the max
```

Listing 68: Top rows and the row of the maximum

The Nth distinct value. The second-highest and Nth-highest questions read the value sitting at a chosen rank. Deduplicating the values first keeps ties from consuming a rank, and the result resolves to a missing value when the data holds fewer than N distinct entries, which is the required answer to an absent Nth place.

```
1 def nth_highest(s, n):
2     u = s.drop_duplicates().nlargest(n)
3     return u.iloc[-1] if len(u) >= n else np.nan
```

Listing 69: Nth-highest distinct value, NaN when it does not exist

Ranking. `rank(method, ascending)` numbers rows by value, with the tie policy set by `method`: `dense`, `min`, and `first` correspond to the SQL `DENSE_RANK`, `RANK`, and `ROW_NUMBER`. Applied after a `groupby` it ranks within each group, the top-N-per-group tool. Sorting and then `groupby(...).head(n)` reaches the same result by keeping the first `n` rows of each group. Passing `pct=True` scales the ranks to the interval (0, 1], giving a percent rank.

```
1 df['rk'] = df.groupby('dept')['salary'].rank(method='dense', ascending=
    False)
2
3 (df.sort_values('salary', ascending=False)
4     .groupby('dept').head(1))      # highest-paid row per dept
```

Listing 70: Rank within a group, and top-N per group by slicing

Position within a group. `groupby(...).cumcount()` numbers the rows of each group by their current order from 0 and resets at every group edge, the within-group `ROW_NUMBER` carrying no value to rank on. `ngroup()` labels the groups themselves 0, 1, 2 in order of appearance. Neither reads a column, so both rest on a prior sort the way `rank` does.

```
1 df['seq'] = df.sort_values('ts').groupby('user').cumcount()
```

Listing 71: A 0-based sequence number within each group

Row-relative operations need an explicit order

`shift`, `diff`, `rank`, `rolling`, and the cumulative methods all read the current row order and nothing else. Without a preceding `sort_values` the previous row is whatever order the rows arrived in. This is the `ORDER BY` inside a SQL window, and a per-group version needs the operation applied through `groupby` so it resets at each group boundary rather than bleeding across.

Neighbor access with `shift` and `diff`. `shift(n)` moves a column down `n` positions and fills the vacated top with `NaN`, so `shift(1)` sets the previous row's value beside the current one, the `LAG`. `shift(-1)` looks ahead, the `LEAD`. `diff()` equals `current - shift(1)`, so `diff() > 0` tests whether a value rose. The first row of an ordering has no predecessor, so its shifted value is `NaN` and every comparison against it is `False`, which drops it with no explicit guard. Applying the shift through `groupby` resets it at each group edge. The one-period fractional change, `close / close.shift(1) - 1`, is packaged as `pct_change`, the simple return.

```
1 df = df.sort_values(['ticker', 'date'])
2
3 prev = df.groupby('ticker')['close'].shift(1)
4 df['ret'] = df['close'] / prev - 1 # explicit, via shift
5 df['ret'] = df.groupby('ticker')['close'].pct_change() # same return
```

Listing 72: Per-group return against the previous row

Running and rolling windows. A cumulative method such as `cumsum` or `cummax` runs from the start up to the current row, giving a running total. `rolling(k)` fixes a window of `k` consecutive rows for a moving statistic, and `expanding` grows the window from the start. `min_periods` sets how many observations a window needs before it returns a value in place of `NaN`. Wrapping these in `groupby(...).transform(...)` keeps the result aligned to the original rows while resetting each group.

```
1 df['ma20'] = (df.groupby('ticker')['close']
2              .transform(lambda s: s.rolling(20).mean()))
```

Listing 73: A per-group moving average aligned to the rows

`rolling` accepts any reduction, so `.rolling(20).std()` is a moving volatility, and a moving correlation between two aligned series would be `.rolling(60).corr(other)`. `ewm` weights recent observations more heavily under an exponential decay set by `span`, `halflife`, or `alpha`, giving the exponentially weighted mean and variance used in volatility estimates. Compounding runs through `cumprod`, so `(1 + ret).cumprod() - 1` accumulates a return series.

```
1 g = df.groupby('ticker')
2 df['vol20'] = g['ret'].transform(lambda s: s.rolling(20).std())
3 df['ewma20'] = g['close'].transform(lambda s: s.ewm(span=20).mean())
4 df['cum_ret'] = g['ret'].transform(lambda s: (1 + s).cumprod() - 1)
```

Listing 74: Moving volatility, exponential mean, and compounded return

Time-based and custom windows. An integer `rolling(k)` counts `k` rows and ignores their timestamps. An offset string, `rolling('7D')`, counts a span of calendar time instead and needs a `DatetimeIndex` or an `on=` column naming the time field. The two coincide only when the rows sit one period apart with no gap between them. A window takes any reduction, so `sum`, `min`, `max`, `median`, and `quantile` all apply, alongside `center=True` for a window centered on

each row. `rolling(k).apply(f, raw=True)` runs an arbitrary `f` over each window as a numpy array, the route for a statistic with no built-in form.

```

1 df = df.set_index('date').sort_index()
2
3 df['roll7'] = df['amount'].rolling('7D').sum()           # seven calendar
    days back
4 df['rng10'] = df['amount'].rolling(10).apply(lambda w: w.max() - w.min
    (), raw=True)
5 df['med5'] = df['amount'].rolling(5, center=True).median()

```

Listing 75: A calendar window, a custom window, and a centered median

A row window is not a day window

`rolling(7)` spans seven rows, which equals seven days only when every day is present. On a series with missing dates it reaches further back in time than seven days. Two fixes hold the window to calendar length: `reindex` to a complete date range so each day is a row, or use the offset form `rolling('7D')`, which measures the span directly and covers the rows whose timestamps fall within seven days back of the current one.

Time indexing and resampling. A `DatetimeIndex` unlocks time-aware operations the ordinary index lacks. Partial-string slicing selects by calendar span, so `df.loc['2023']` takes the whole year and `df.loc['2023-01': '2023-06']` a range of months. `resample` is a groupby keyed by a time frequency: it bins the rows into calendar buckets and reduces each, down-sampling a series to a coarser frequency or up-sampling to a finer one that then needs filling. Frequency aliases name the bucket, among them 'B' business day, 'W' week, 'ME' month-end, and 'QE' quarter-end, the last two written 'M' and 'Q' before pandas 2.2.

```

1 px = px.set_index('date').sort_index()
2
3 px['close'].resample('ME').last()           # month-end close (downsample)
4 px['close'].resample('W').ohlc()           # weekly open-high-low-close
    bars
5 px['close'].resample('B').ffill()           # to business days, carry last
    forward
6
7 px.groupby('ticker')['close'].resample('ME').last() # per entity

```

Listing 76: Downsample, build OHLC bars, and align to business days

3.7.10 Consecutive Runs

Some results turn on stretches of adjacent rows that share a property: the same value repeated, or keys that step by a fixed amount with no break between them. A maximal such stretch is a run. The pattern, known as gaps and islands, labels every run with an identifier, after which an ordinary `groupby` measures each run's length or reduces its rows. As with any row-relative work the rows must be sorted first, since a run is defined by adjacency in the current order.

Two run labels

A run of a repeated value ends wherever the value changes, so `(s != s.shift()).cumsum()` labels the runs: the comparison is `True` at each change, and

the cumulative sum steps up there and holds flat between the changes. A run of consecutive keys uses a different invariant. Number the rows 0, 1, 2 in order, and inside a run both the key and the counter rise by one at every step, so their difference `key - counter` stays constant and jumps exactly where the run breaks. For a key stepping by `d`, subtract `counter * d`.

Equal-value runs. A label built from the change points groups adjacent equal values, which answers questions about a value repeating across consecutive rows. `transform('size')` broadcasts each run's length back onto its rows, and a mask keeps the runs that reach a threshold.

```
1 logs = logs.sort_values('id')
2
3 run    = (logs['num'] != logs['num'].shift()).cumsum()
4 length = logs.groupby(run)['num'].transform('size')
5 result = logs.loc[length >= 3, 'num'].unique()
```

Listing 77: Numbers appearing on three or more consecutive rows

Consecutive keys. When the run is a stretch of consecutive integers or consecutive dates, the `key - counter` label applies. A per-entity streak resets the counter at each entity through `groupby(...).cumcount()`, and the label becomes the pair of the entity and the offset key. Duplicate keys within an entity break the unit step, so collapse them with `drop_duplicates` before counting.

```
1 logins = (logins.drop_duplicates(['user_id', 'day'])
2           .sort_values(['user_id', 'day']))
3
4 counter = logins.groupby('user_id').cumcount()
5 streak  = logins['day'] - counter * pd.Timedelta('1D')    # constant
               along a run of consecutive days
6
7 length = logins.groupby(['user_id', streak])['day'].transform('size')
8 active = logins.loc[length >= 5, 'user_id'].unique()
```

Listing 78: Users active five or more consecutive days

A predicate can gate the run before the label is built. Filtering to the qualifying rows first, then labeling the survivors by `id - counter`, finds consecutive runs among rows that pass a test, such as three or more successive records above a threshold.

```
1 busy = stadium[stadium['people'] >= 100].sort_values('id')
2
3 grp    = busy['id'] - np.arange(len(busy))    # consecutive ids
               collapse to one value
4 length = busy.groupby(grp)['id'].transform('size')
5 result = busy[length >= 3]
```

Listing 79: Three or more consecutive ids clearing a threshold

The longest run is the largest group under the same label, `groupby(run).size().max()`, and the run's own rows come back by selecting the label that attains it.

<i>Method</i>	<i>Arguments</i>	<i>What it does</i>
shift	periods, default 1	Value k rows earlier in the current order, NaN past the edge. A negative k looks ahead, the LEAD .
diff	periods, default 1	Current value minus the value k rows back.
pct_change	periods, fill_method	Fractional change from the previous row, the one-period return.
cumsum /cumprod	none	Running sum or product from the start to the current row.
cummax /cummin	none	Running maximum or minimum seen so far.
rank	method, ascending, pct	Rank by value. dense , min , first map to DENSE_RANK , RANK , ROW_NUMBER .
cumcount	none	0-based row number within each group in the current order.
ngroup	none	Group label 0, 1, 2 in order of appearance.
rolling	window as rows or an offset like '7D'	Fixed moving window for a reduction. The offset form needs a DatetimeIndex .
expanding	min_periods	Window growing from the start, a running reduction.
ewm	span, halflife, alpha, com	Exponentially weighted mean and variance under a decay.
rolling(w).apply	func, raw	Arbitrary reduction over each window, as a numpy array when raw=True .
nlargest /nsmallest	(n, col)	Top or bottom n rows by a column, cheaper than a full sort.
idxmax /idxmin	none	Index label of the extreme value, an argmax read back by loc .
resample	frequency 'B', 'W', 'ME', 'QE'	A groupby keyed by a time frequency on a DatetimeIndex .
transform	func	Group reduction broadcast back to every input row.

Row-relative and window methods. All read the current row order, so a **sort_values** precedes them, and a **groupby** resets them at each group edge.

<i>Method</i>	<i>Arguments</i>	<i>What it does</i>
str.contains	pat, case, na, regex	Boolean, True where pat is found anywhere. Reads pat as a regex.
str.match / fullmatch	pat, na	Anchored at the start, or over the whole string.
str.extract / extractall	pat, expand	One column per capture group of the first match. extractall returns every match.
str.replace	pat, repl, regex	Substitutes on a pattern. regex defaults to False from pandas 2.0.
str.split / rsplit	pat, n, expand	Splits each string. expand=True spreads the parts into columns.
str.len	none	Character count per string.
str.lower / upper / title / capitalize	none	Case folding. title upper-cases each word, capitalize only the first.
str.strip / lstrip / rstrip	to_strip	Strips characters from the ends, whitespace by default.
str.pad / zfill	width, side / width	Pads to a width. zfill left-pads with zeros.
str.cat	others, sep, na_rep	Concatenates element-wise, or joins the whole Series when others is omitted.
str.get / str[a:b]	index or slice	Element or slice of each string by position.
str.startswith / endswith	prefix or suffix, na	Boolean test at either end, literal text.
str.find	sub	0-based index of the first occurrence, -1 when absent.
str.count	pat	Number of non-overlapping matches per string.

Vectorized string methods on the **.str** accessor. The predicates take **na** to score a missing string, and the pattern methods read a regex unless told otherwise.

<i>Method</i>	<i>Arguments</i>	<i>What it does</i>
<code>pd.to_datetime</code>	<code>format, errors, unit</code>	Parses text or epoch values into <code>datetime64[ns]</code> , the prerequisite for <code>.dt</code> .
<code>pd.date_range</code>	<code>start, end, periods, freq</code>	Builds a <code>DatetimeIndex</code> at a fixed frequency, used to reindex to a complete span.
<code>pd.Timedelta</code>	<code>'1D', '2h', ...</code>	A duration, added to or subtracted from timestamps.
<code>dt.year / month / day</code>	<code>none</code>	Calendar components as integers.
<code>dt.hour / minute / second</code>	<code>none</code>	Clock components as integers.
<code>dt.dayofweek / day_name</code>	<code>none</code>	Weekday as <code>0..6</code> with Monday 0, or its name.
<code>dt.quarter</code>	<code>none</code>	Calendar quarter <code>1..4</code> .
<code>dt.date / time</code>	<code>none</code>	The date or clock part, dropping the other.
<code>dt.days_in_month</code>	<code>none</code>	Length of the month in days.
<code>dt.is_month_end</code>	<code>none</code>	Boolean calendar-position flag, with <code>is_month_start</code> and the quarter and year forms.
<code>dt.floor / ceil / round</code>	<code>freq</code> like <code>'D', 'h'</code>	Snaps each timestamp down, up, or to nearest at a frequency.
<code>dt.normalize</code>	<code>none</code>	Sets the time to midnight, keeping the date.
<code>dt.to_period</code>	<code>'M', 'Q', 'Y'</code>	Converts to a period, a whole calendar span rather than an instant.
<code>dt.strftime</code>	<code>format</code>	Formats each timestamp as text, e.g. <code>'%Y-%m-%d'</code> .

The `.dt` accessor and the datetime constructors. The accessor requires a `datetime64[ns]` column, which `pd.to_datetime` produces from text or epoch input.